Школа анализа данных

Арифметика глубокого обучения

Михаил Хрущев
Руководитель группы претрейна Alice AI

Познакомимся

Я – Михаил Хрущёв, руковожу группой претрейна Alice AI.

Более 4 лет я и моя команда обучаем большие языковые модели и разрабатываем инфраструктуру для их обучения.

Основная часть LLM моделей Яндекса были обучены нами или на нашей инфраструктуре.

Наши open source:

- YaFSDP
- YaLM 100B
- Yandex GPT5 Lite

Содержание

01 История 1

02 История 2

История 1

Перед итерацией обучения нам нужно вычитать и подготовить батч.
Обычно подготовка происходит на CPU:

История 1

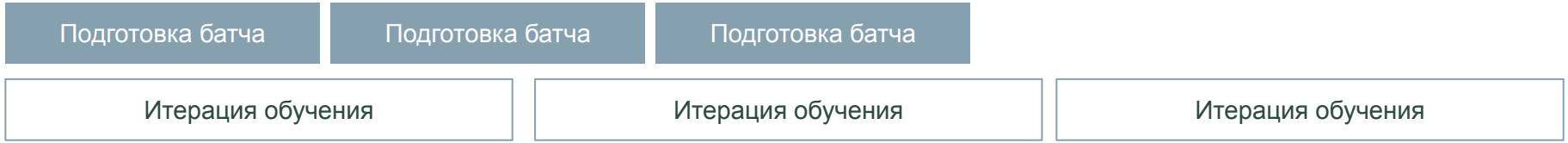
Перед итерацией обучения нам нужно вычитать и подготовить батч.
Обычно подготовка происходит на CPU:

Проблема: Из-за подготовки батча простаивает GPU – это очень дорогой ресурс.

История 1

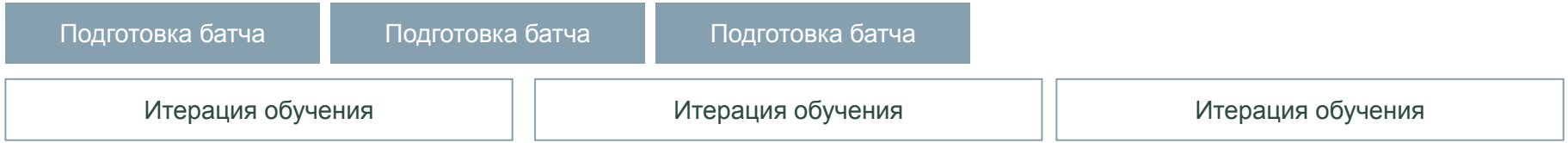
Перед итерацией обучения нам нужно вычитать и подготовить батч.
Обычно подготовка происходит на CPU:

Что можно сделать: Выносим чтение и подготовку батча в отдельный процесс.



История 1. Выводы

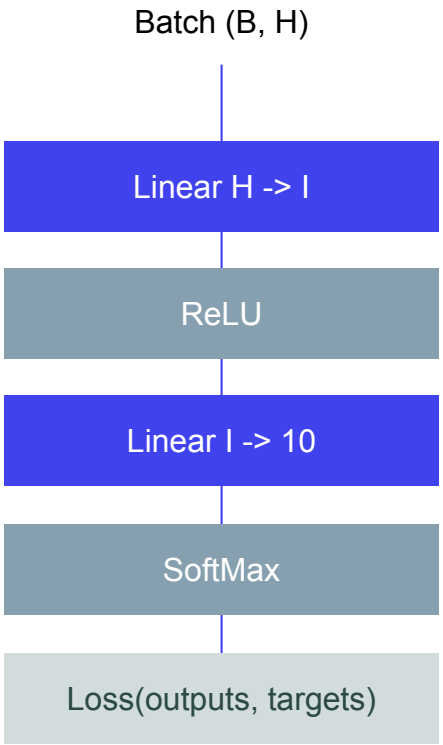
- Подготовка данных может привести к замедлению обучения и простою GPU.
- GPU – самый дорогой ресурс, его простой нежелателен.
- Чтобы избежать простоя, можно сделать загрузку данных асинхронной.



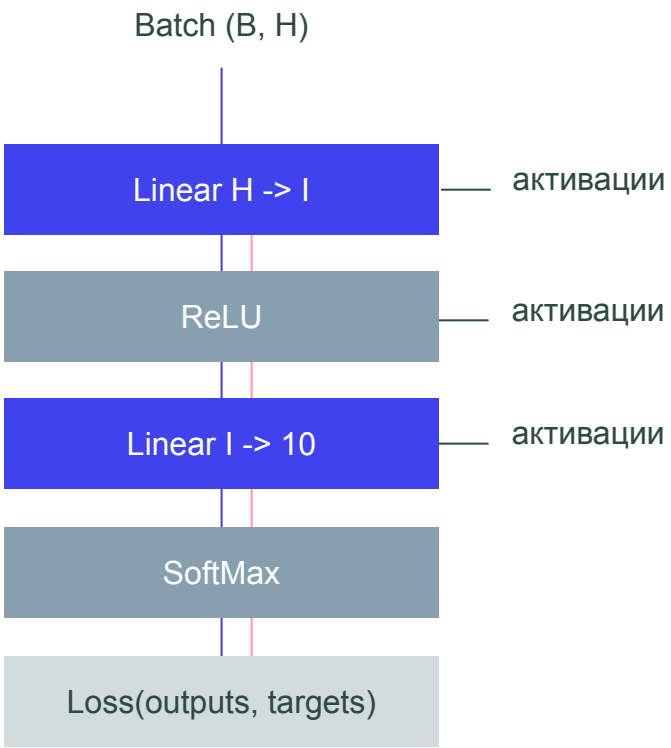
История 2. Внутри итерации

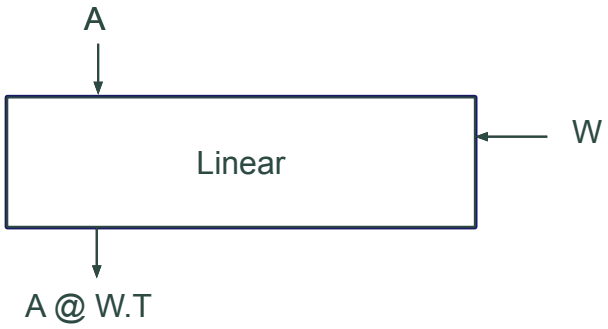
Итерация обучения

История 2. Небольшой экскурс в DL

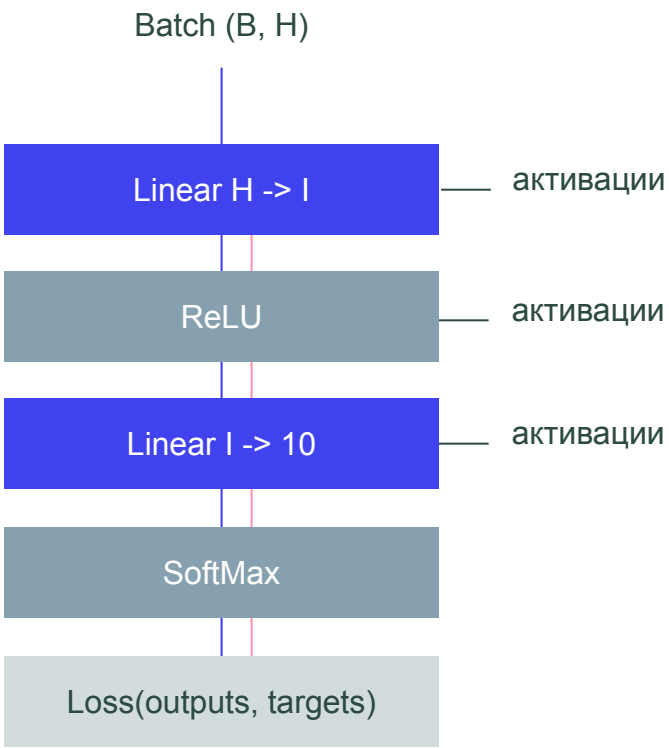


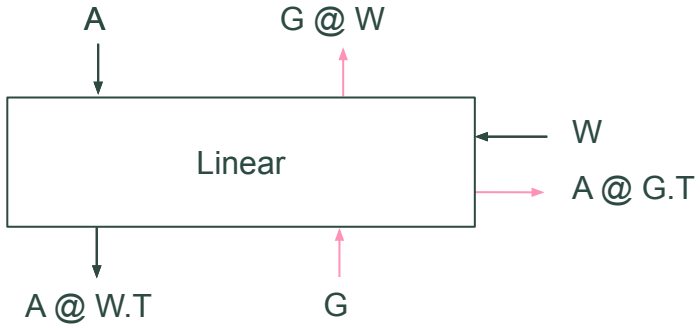
История 2. Небольшой экскурс в DL



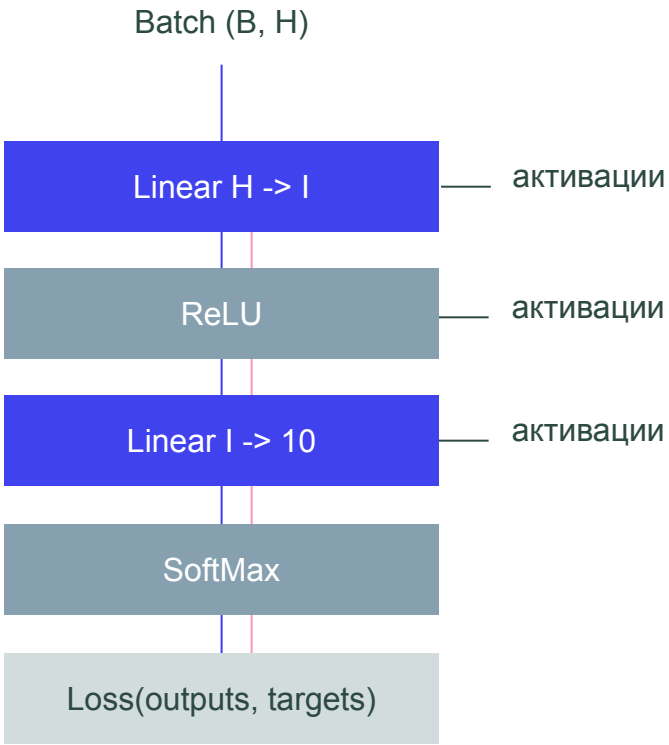


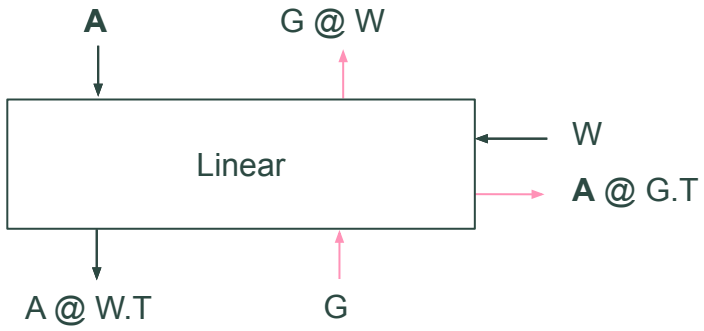
История 2. Небольшой экскурс в DL



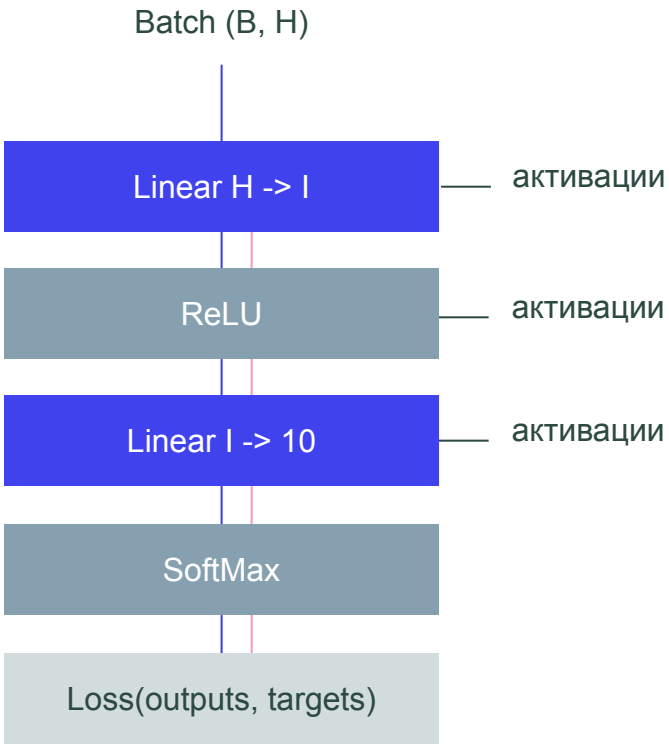


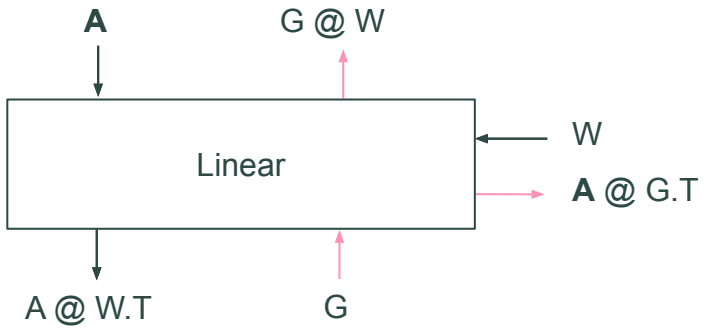
История 2. Небольшой экскурс в DL

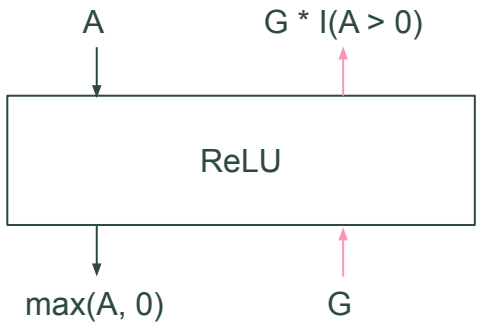




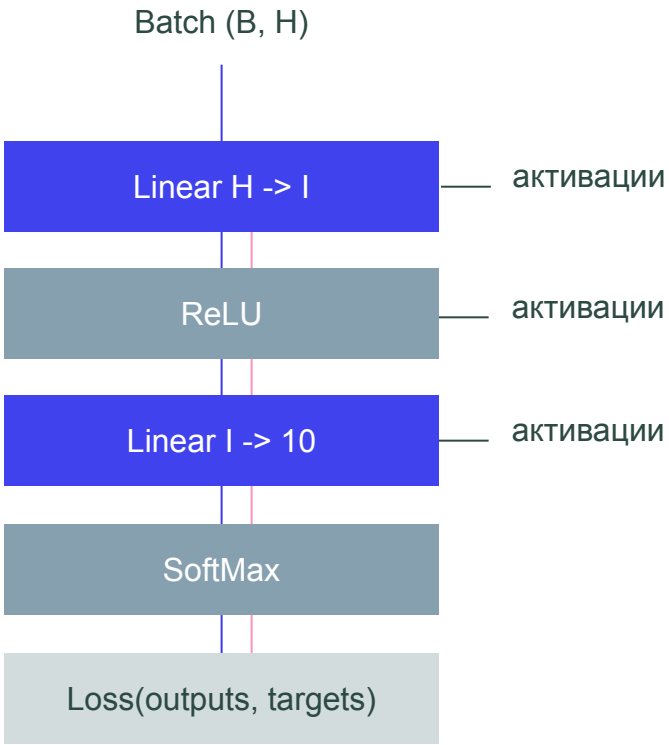
История 2. Небольшой экскурс в DL

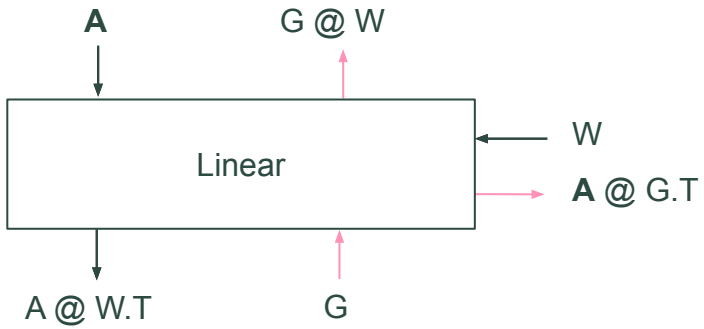


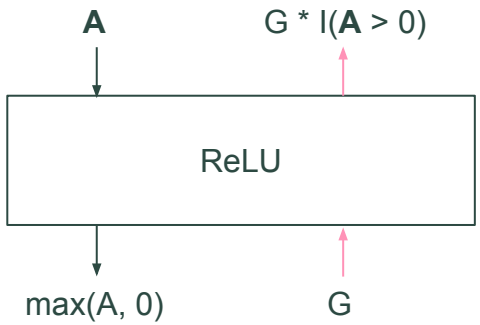




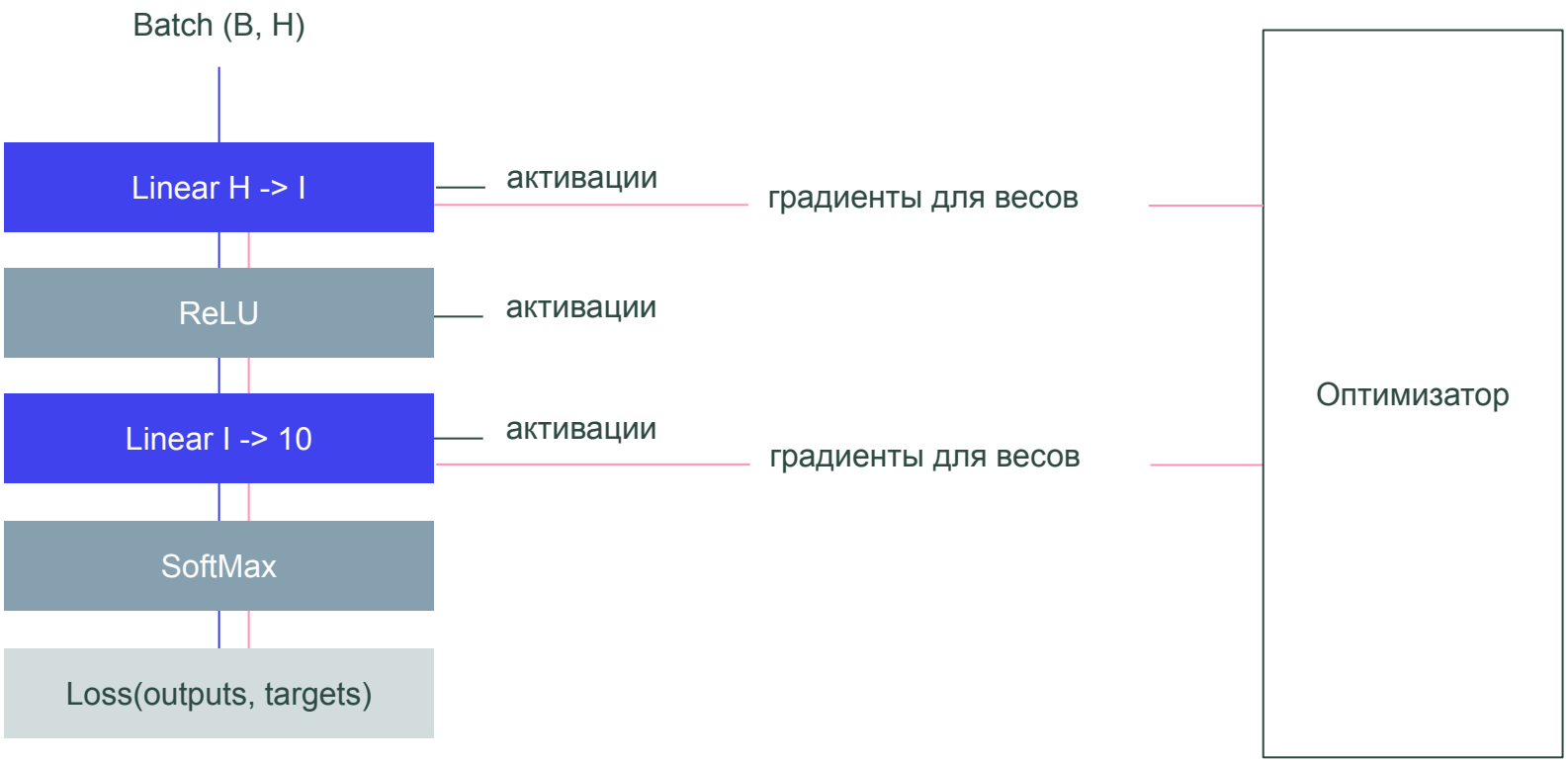
История 2. Небольшой экскурс в DL



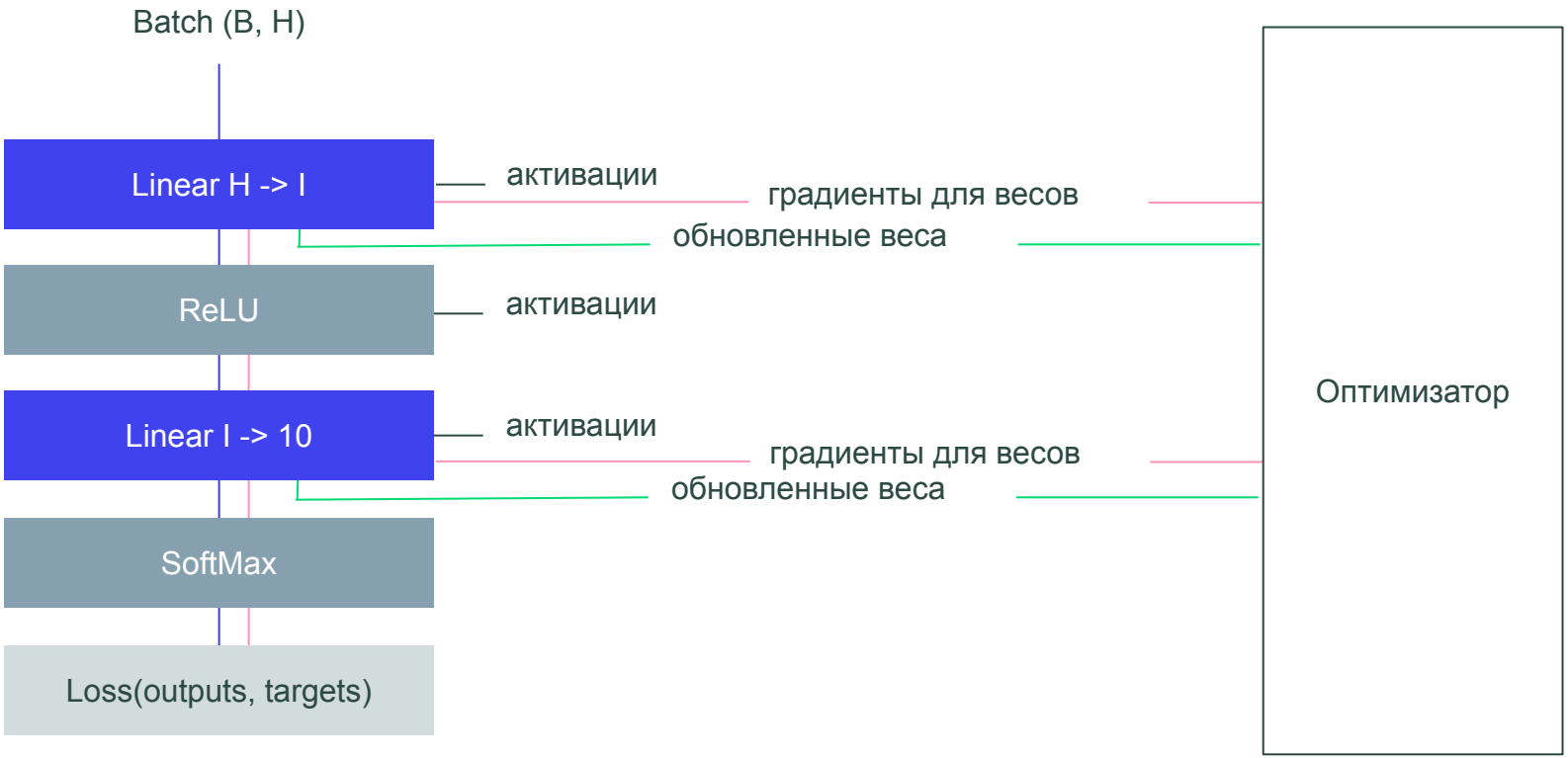




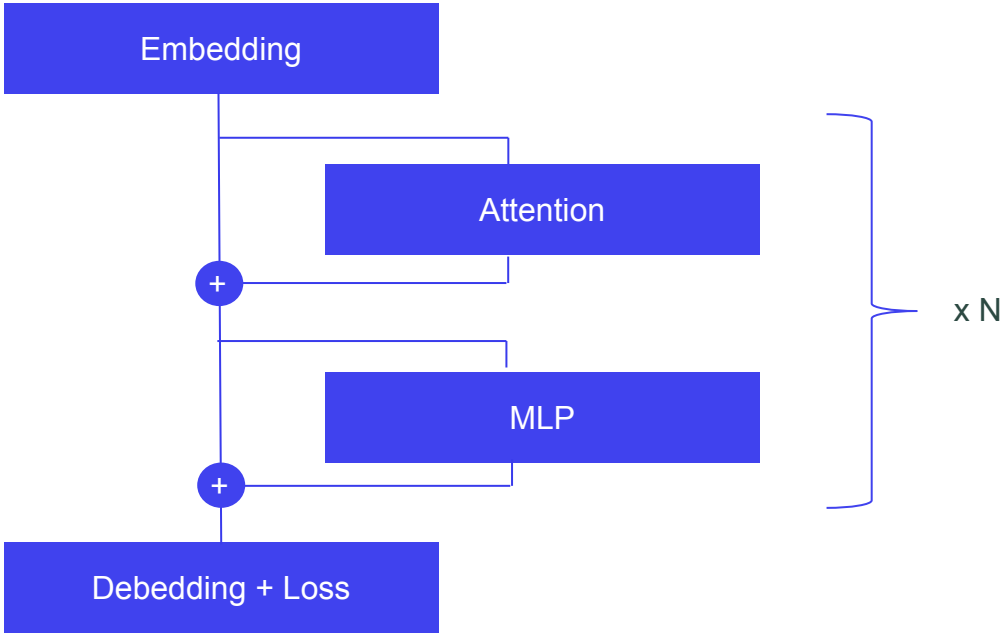
История 2. Небольшой экскурс в DL



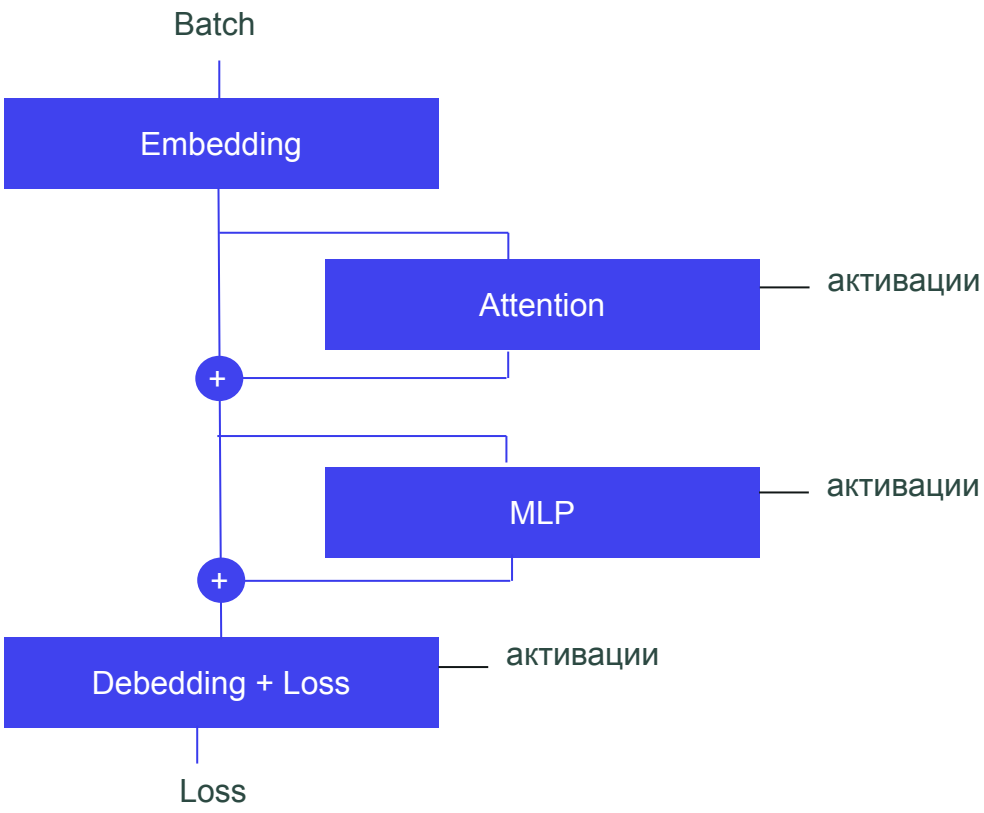
История 2. Небольшой экскурс в DL



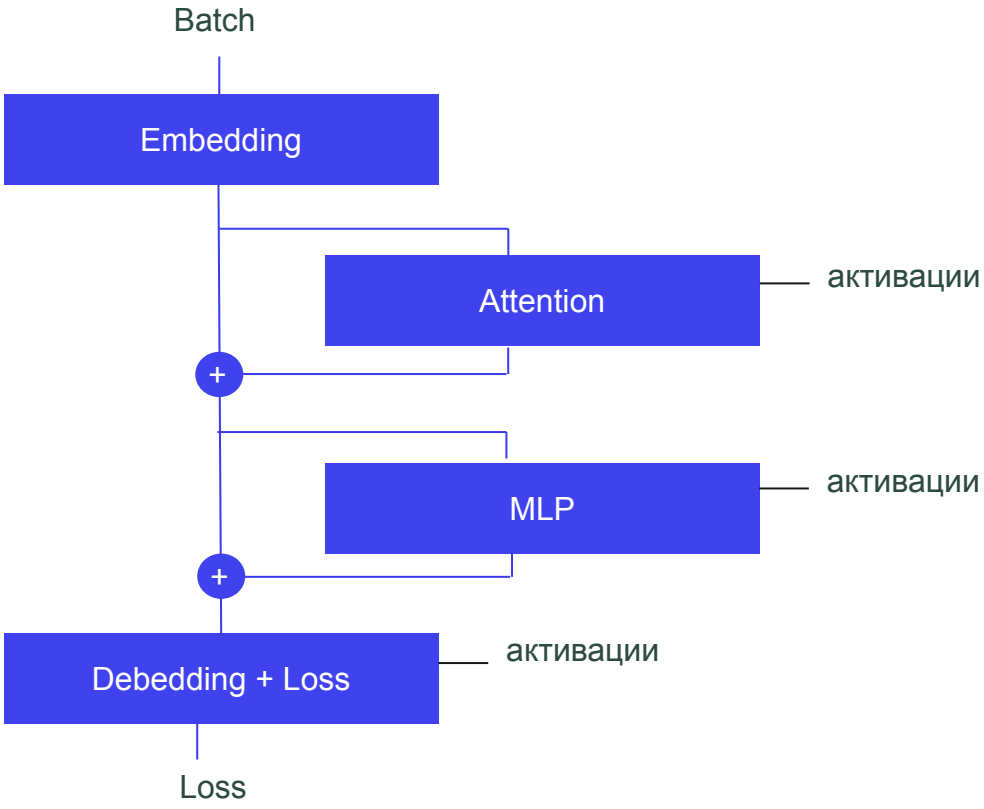
История 2

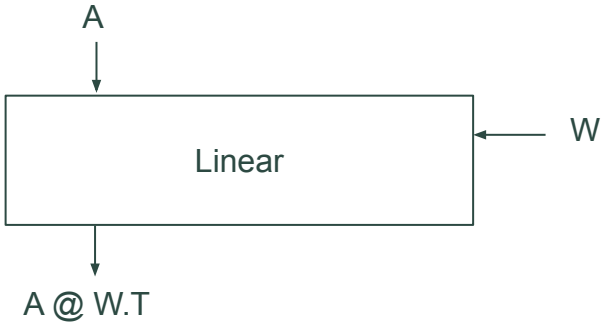


История 2

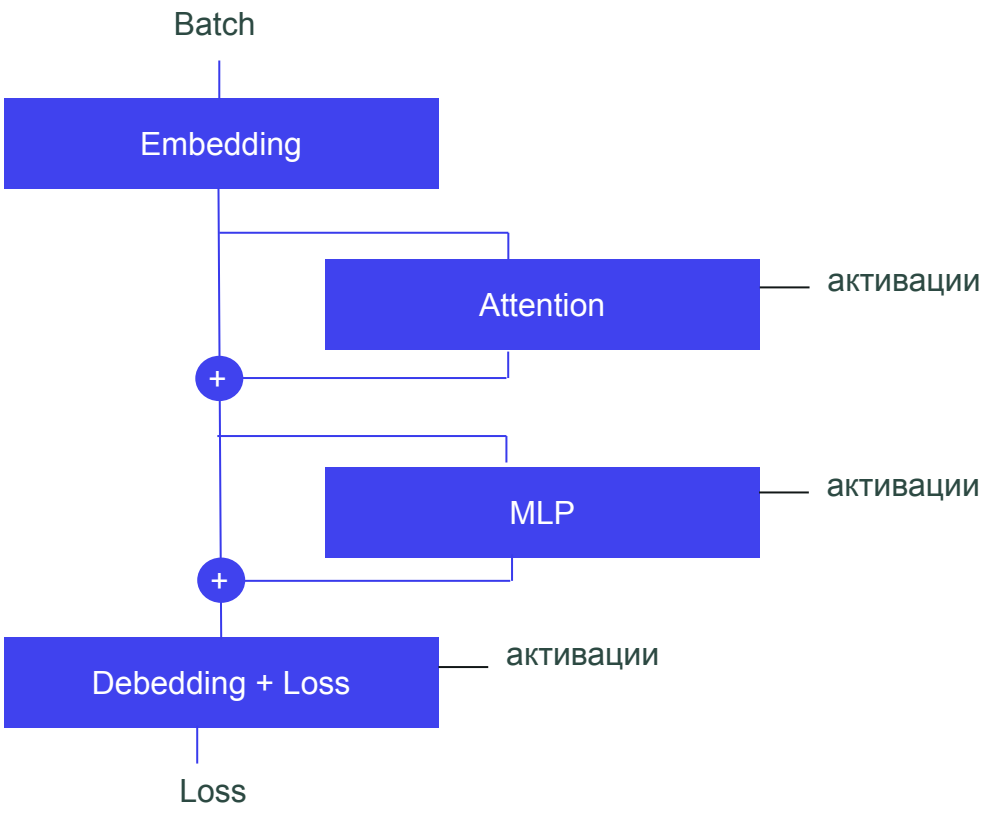

Batch
Embedding
Attention
активации
+
MLP
активации
+
Debedding + Loss
активации
Loss

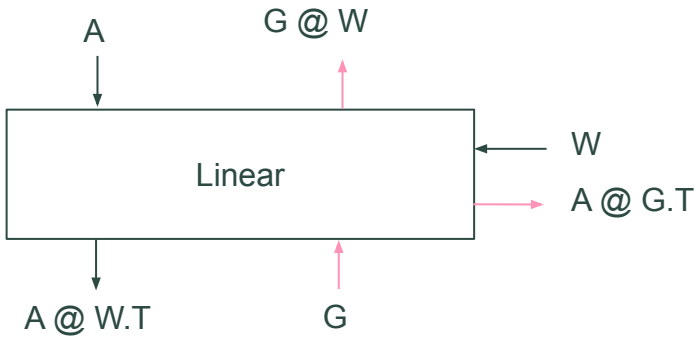
История 2


Batch
Embedding
Attention
активации
+
MLP
активации
+
Debedding + Loss
активации
Loss

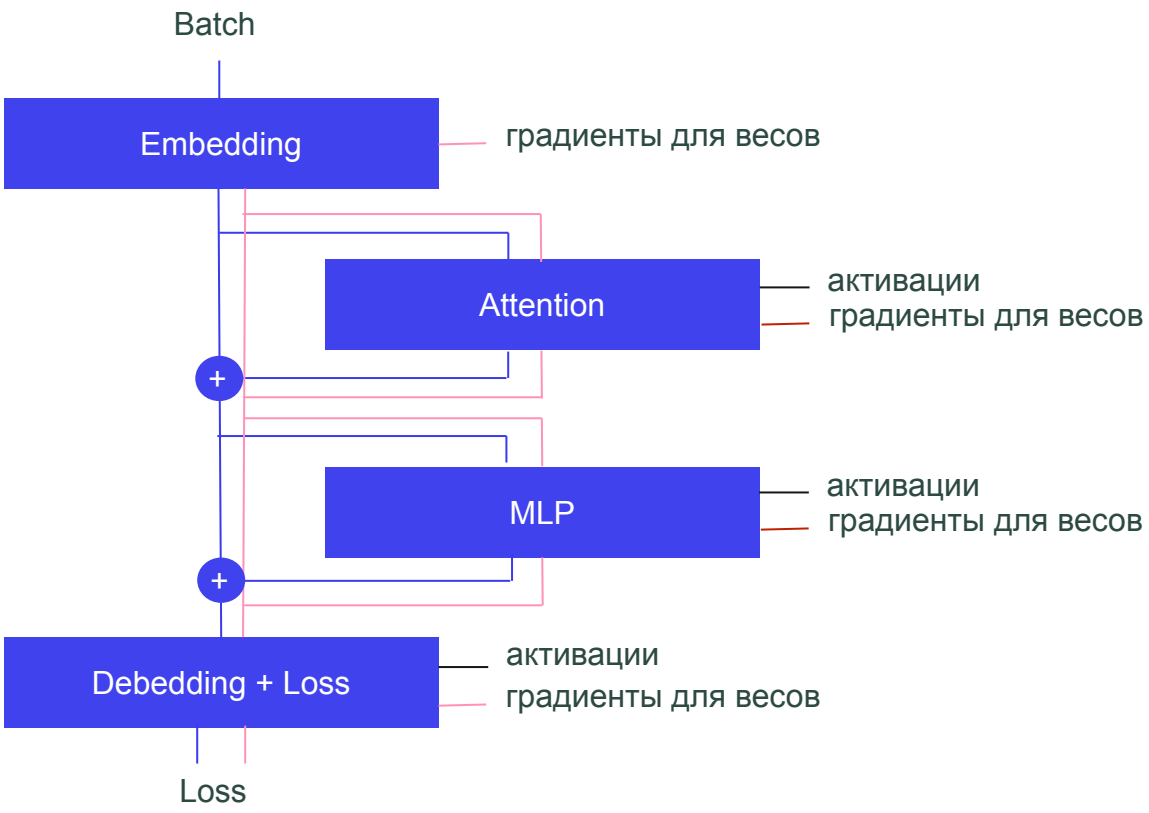

A
Linear
W
A @ W.T

История 2

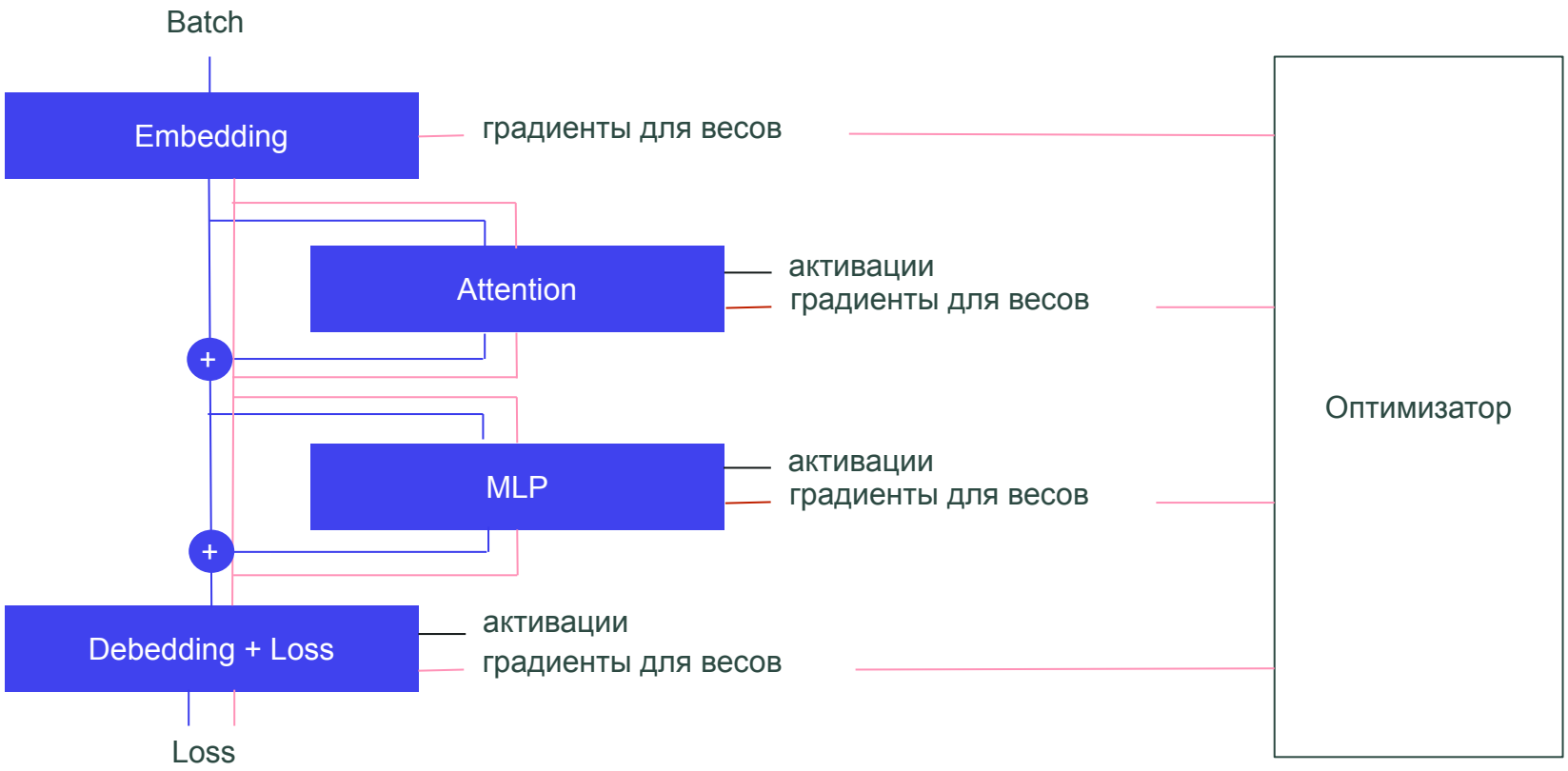

Batch
Embedding
Attention
активации
+
MLP
активации
+
Debedding + Loss
активации
Loss


A
G @ W
Linear
W
A @ G.T
A @ W.T
G

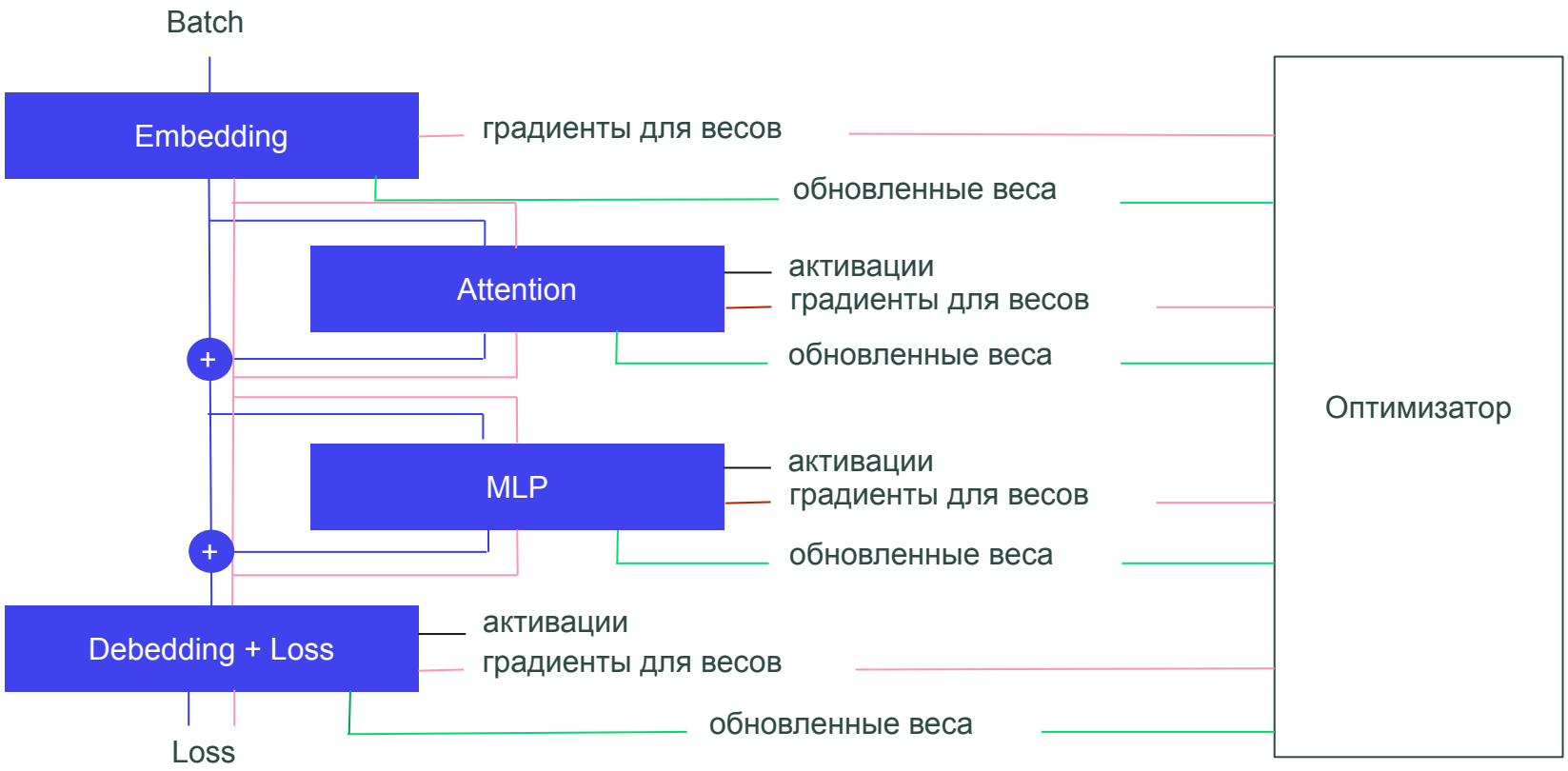
История 2


Batch
Embedding
градиенты для весов
Attention
активации
градиенты для весов
+
MLP
активации
градиенты для весов
+
Debedding + Loss
активации
градиенты для весов
Loss

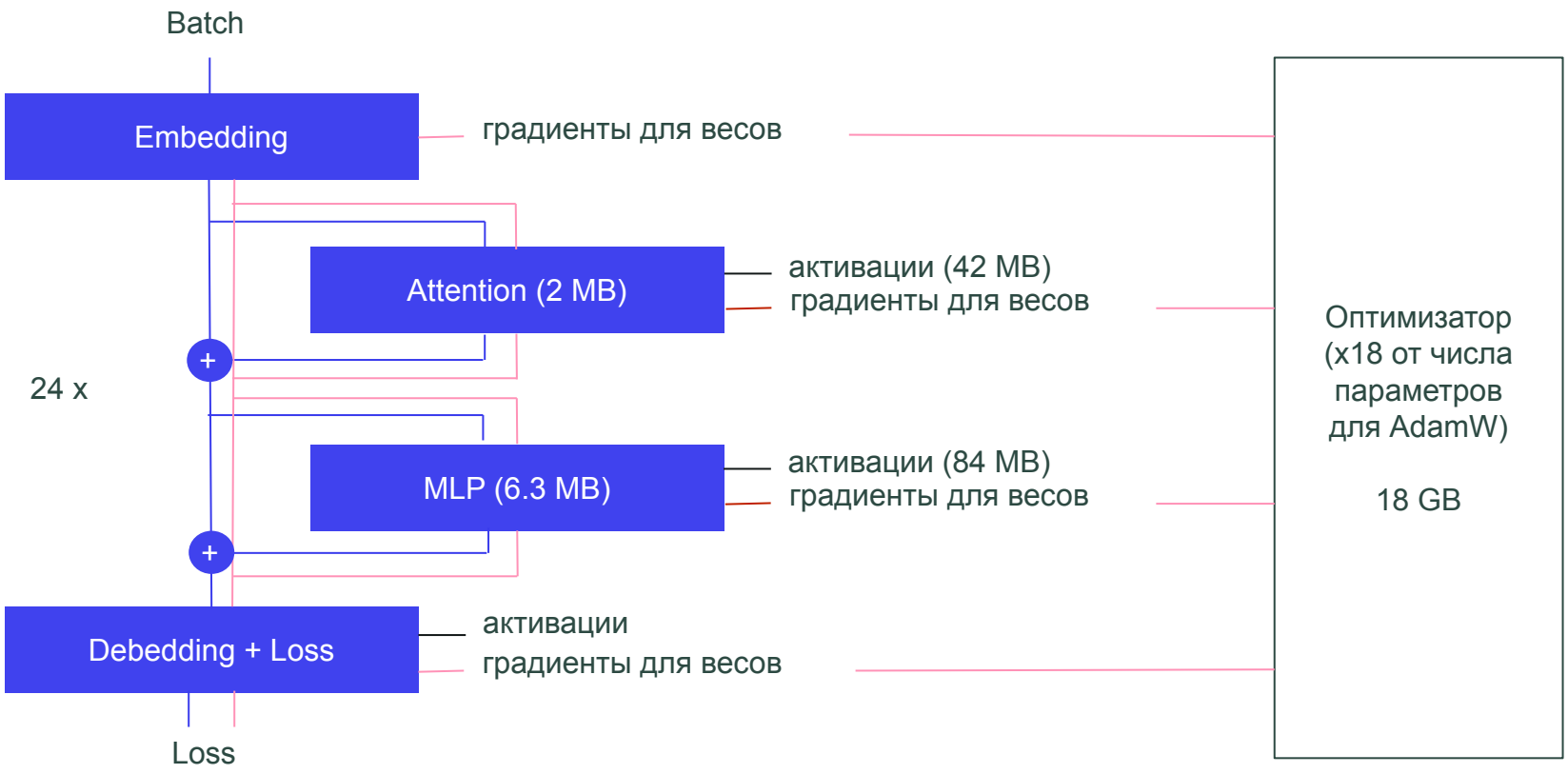
История 2


Batch
Embedding
градиенты для весов
Attention
активации
градиенты для весов
+
MLP
активации
градиенты для весов
+
Debedding + Loss
активации
градиенты для весов
Loss
Оптимизатор

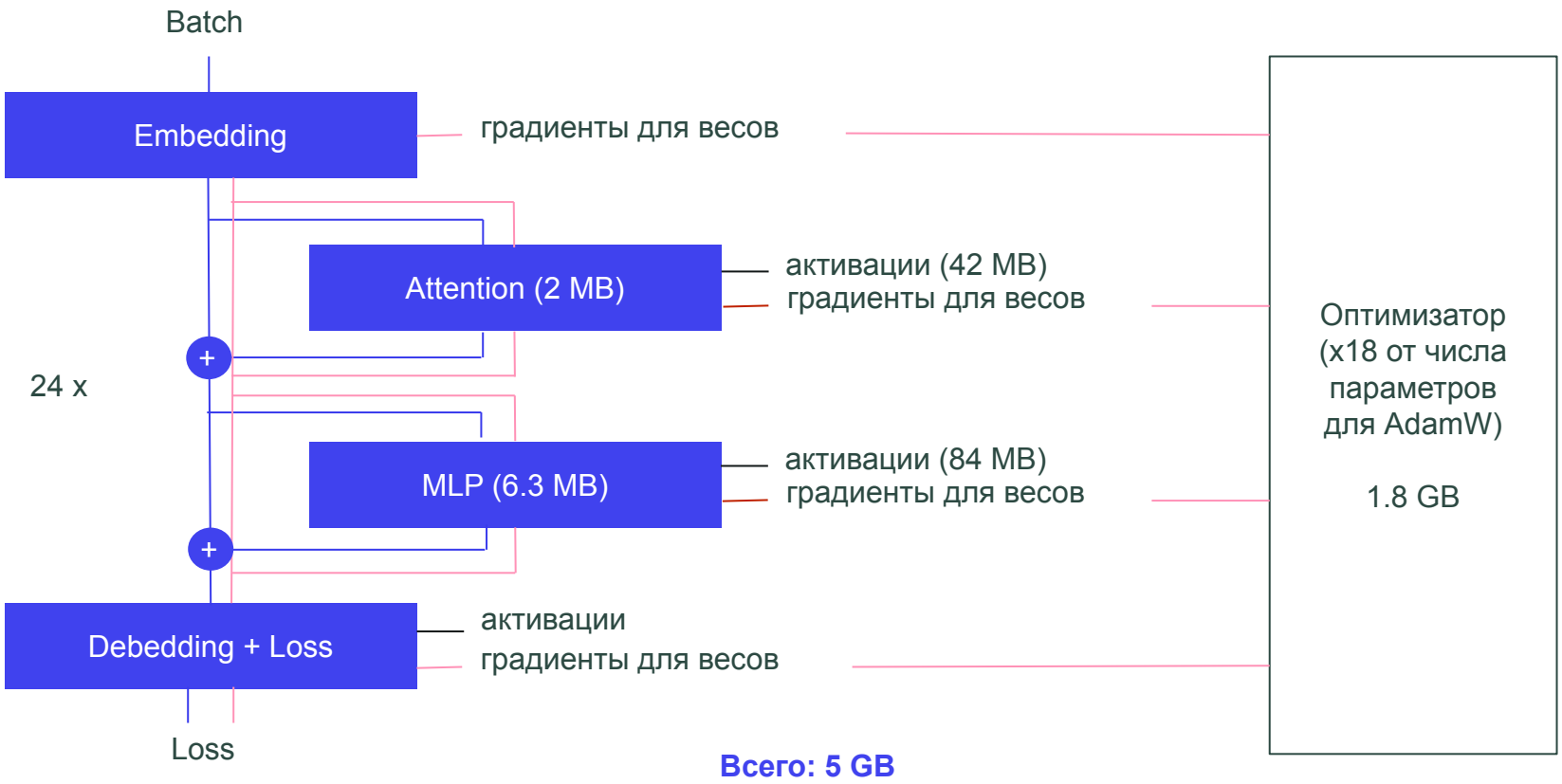
История 2


Batch
Embedding
градиенты для весов
обновленные веса
Attention
активации
градиенты для весов
обновленные веса
+
MLP
активации
градиенты для весов
обновленные веса
+
Debedding + Loss
активации
градиенты для весов
обновленные веса
Loss
Оптимизатор

100M модель

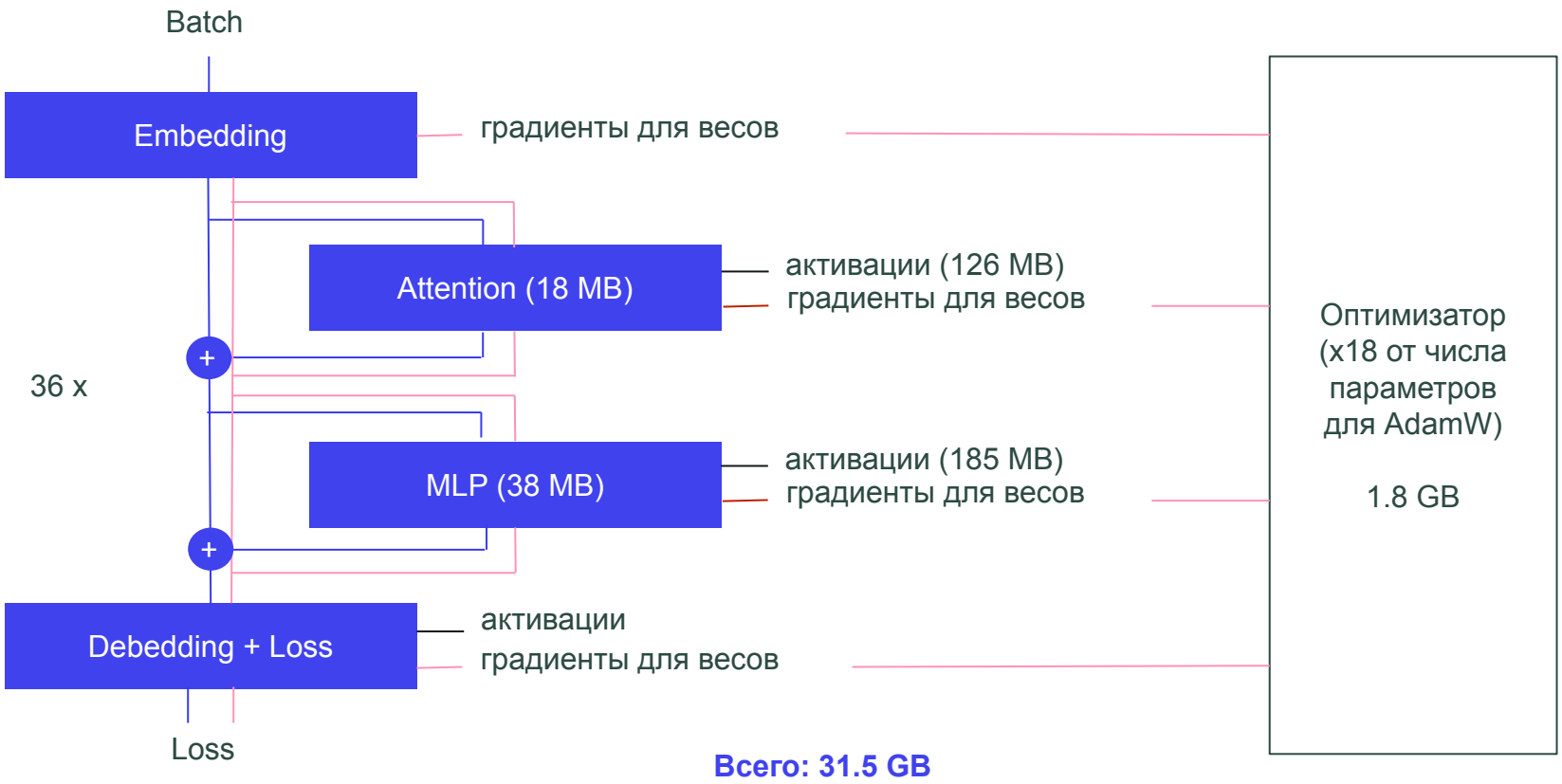

Batch
Embedding
градиенты для весов
Attention (2 MB)
активации (42 MB)
градиенты для весов
24 x
+
MLP (6.3 MB)
активации (84 MB)
градиенты для весов
+
Debedding + Loss
активации
градиенты для весов
Loss
Оптимизатор
(x18 от числа
параметров
для AdamW)
18 GB

100M модель



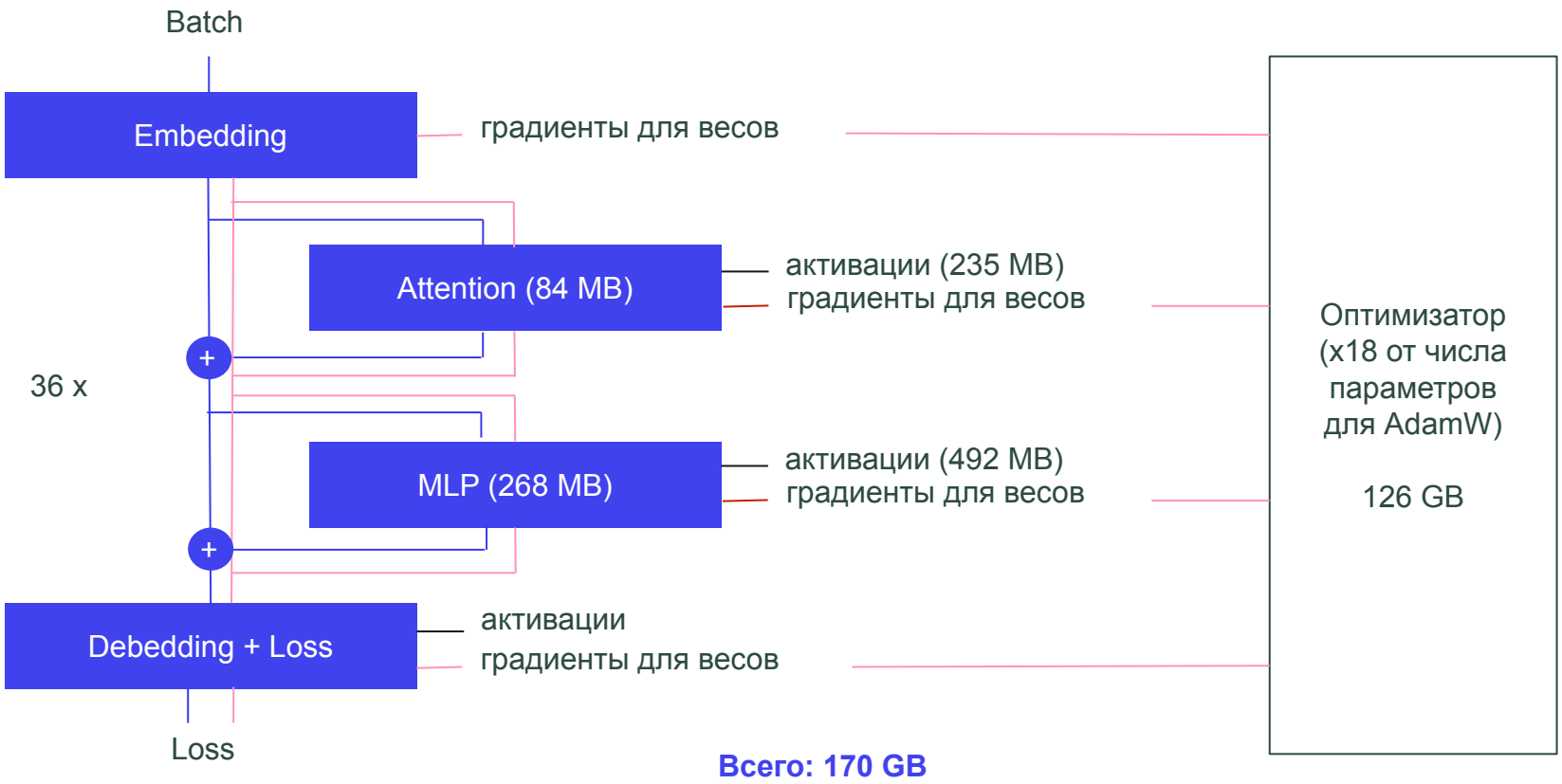
Всего: 5 GB

1B модель



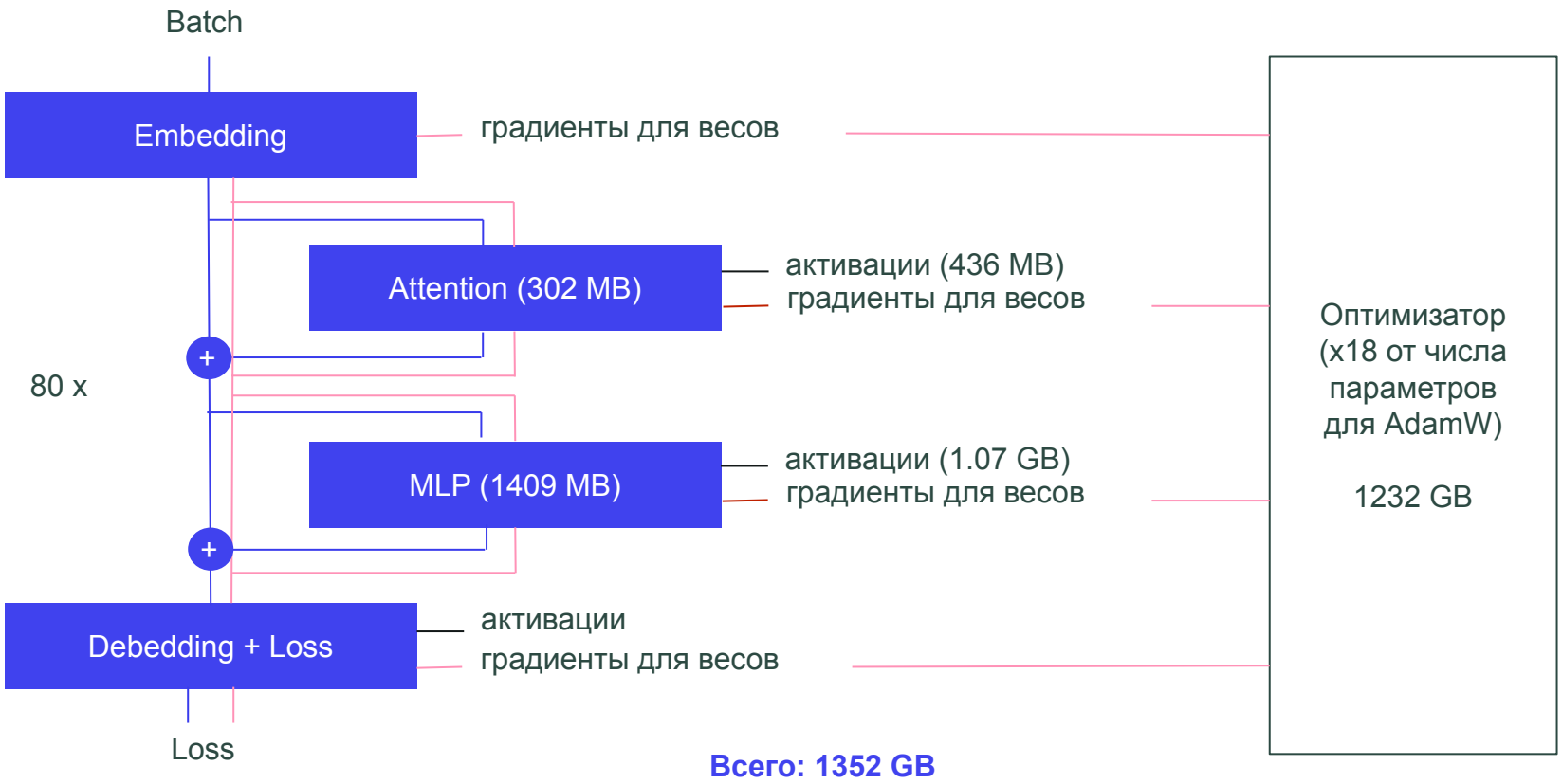
Всего: 31.5 GB

7B модель



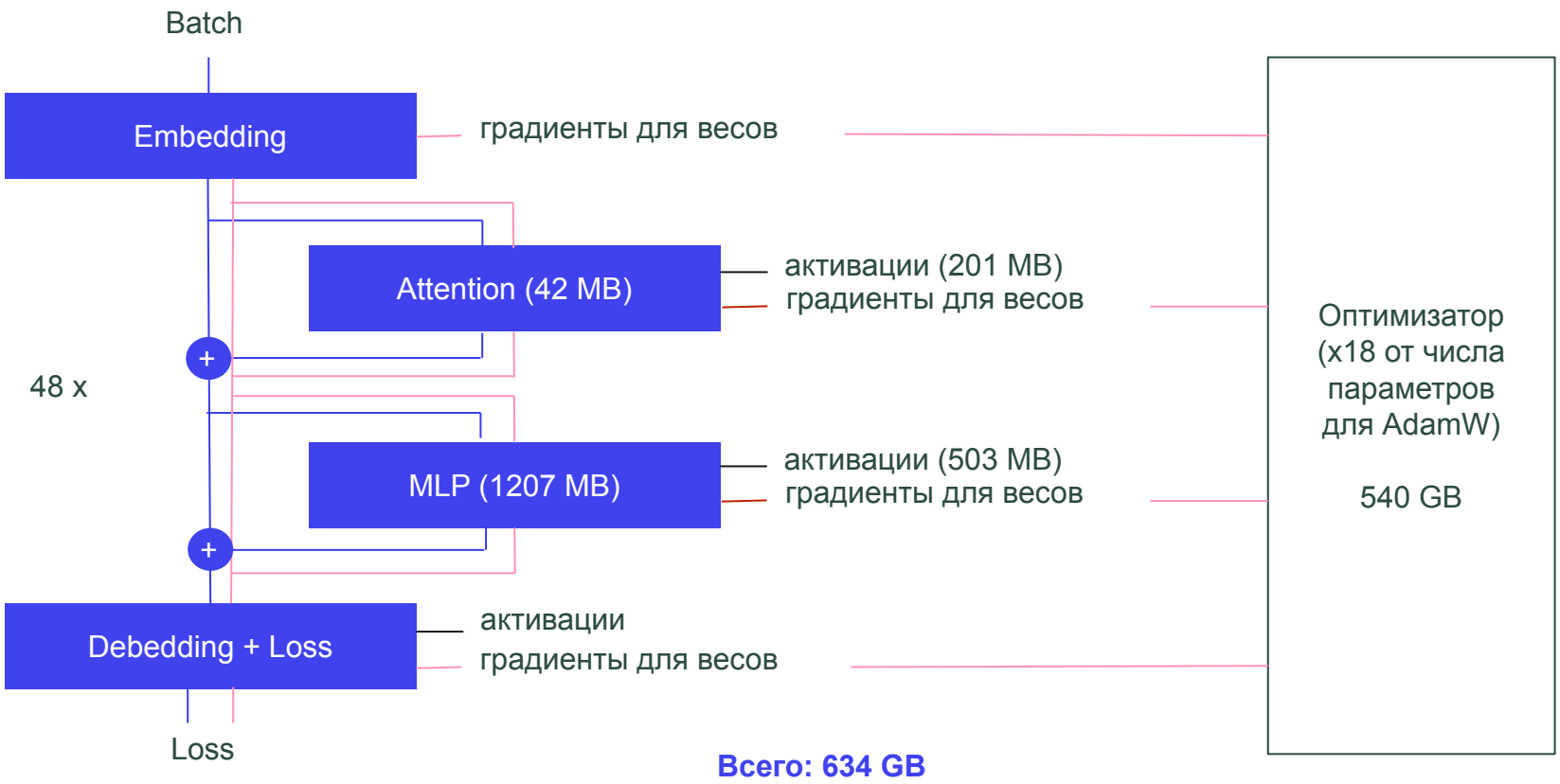
Всего: 170 GB

Llama 70B



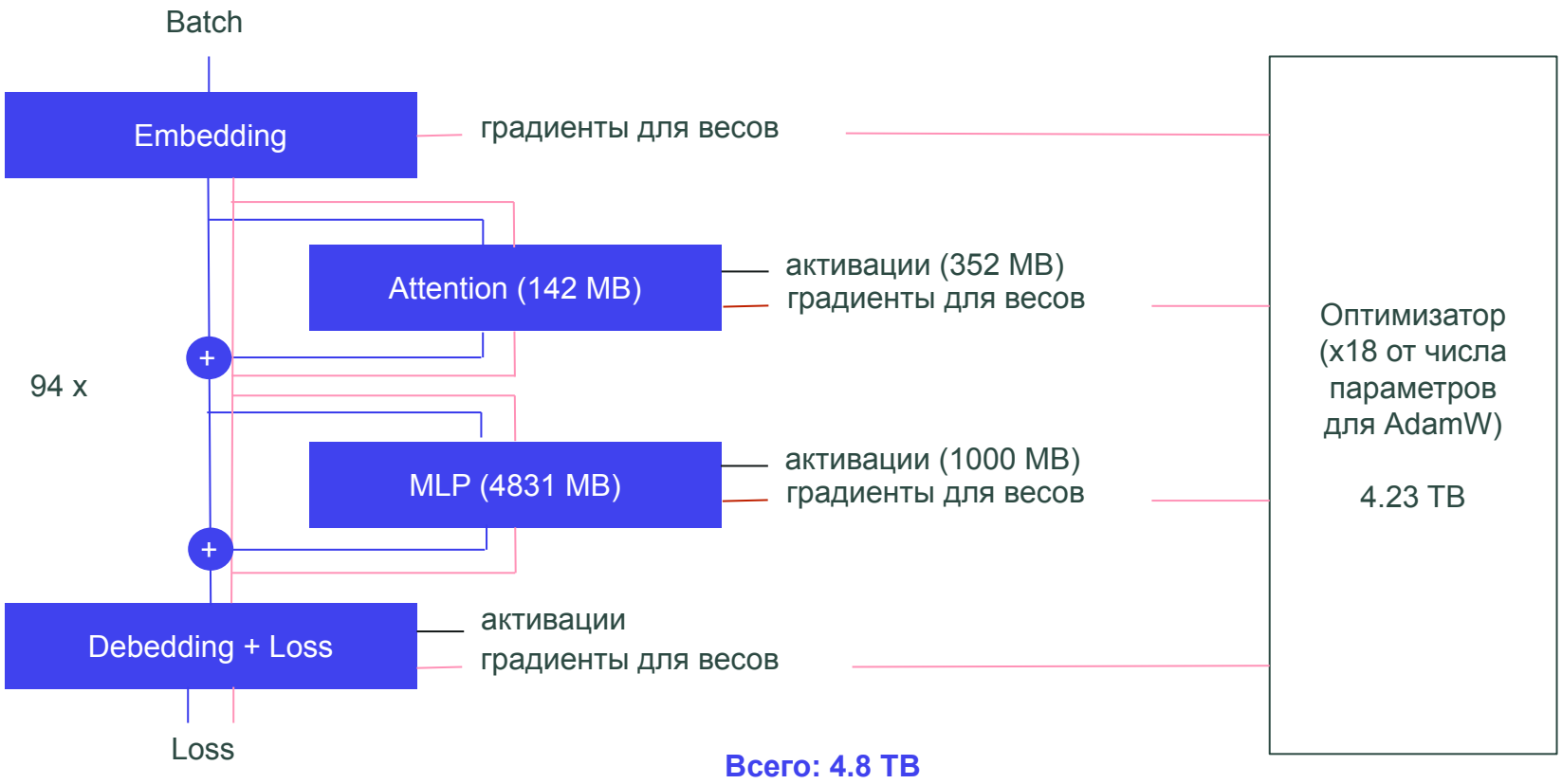
Всего: 1352 GB

Qwen 30B-A3B



Всего: 634 GB

Qwen 235B-A32B



Всего: 4.8 TB

История 2. Проблема

- Для обучения модели нужны терабайты памяти.
- Но на GPU не так много: 40GB-190GB для серверных GPU, единицы GB для игровых.

Что делать?

История 2. Проблема

- Для обучения модели нужны терабайты памяти.
- Но на GPU не так много: 40GB-190GB для серверных GPU, единицы GB для игровых.

Что делать?
Хранить где-то еще!

Как использовать веса/активации/состояния оптимизатора, если они лежат где-то?

История 2. Проблема

- Для обучения модели нужны терабайты памяти.
- Но на GPU не так много: 40GB-190GB для серверных GPU, единицы GB для игровых.

Что делать?
Хранить где-то еще!

Как использовать веса/активации/состояния оптимизатора, если они лежат где-то?
Организовать пересылку.

История 2. Проблема

- Для обучения модели нужны терабайты памяти.
- Но на GPU не так много: 40GB-190GB для серверных GPU, единицы GB для игровых.

Что делать?
Хранить где-то еще!

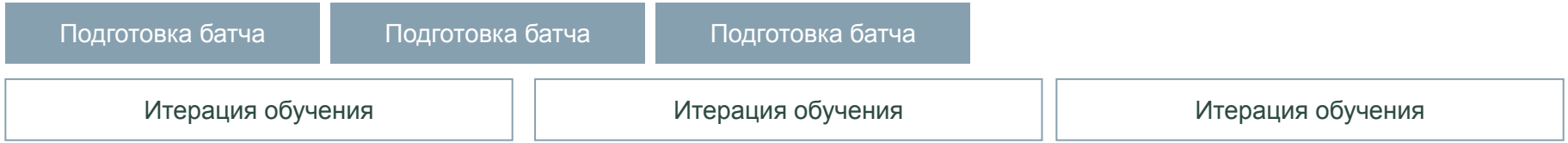
Как использовать веса/активации/состояния оптимизатора, если они лежат где-то?
Организовать пересылку.

Но GPU будут простаивать, если пересылать данные?
Нет, если пересылка асинхронная.

История 1

Перед итерацией обучения нам нужно вычитать и подготовить батч.
Обычно подготовка происходит на CPU:

Что можно сделать: Выносим чтение и подготовку батча в отдельный процесс.



Эффективное обучение – логистическая проблема

- Нужно разнести состояния оптимизатора, веса и активации на разные ресурсы: другие GPU, RAM, SSD.
- Нужно обеспечить сборку этих данных в момент использования на каждой GPU.
- Нужно организовать логистику так, чтобы GPU простаивала минимально.

Эффективное обучение – логистическая проблема

- Нужно разнести состояния оптимизатора, веса и активации на разные ресурсы: другие GPU, RAM, SSD.
- Нужно обеспечить сборку этих данных в момент использования на каждой GPU.
- Нужно организовать логистику так, чтобы GPU простаивала минимально.

Этой задачей занимаются многие исследователи DL уже больше 7 лет.

Содержание

01 Определяемся с логистикой

02 Локальная логистика на GPU

03 Оптимизация elementwise

04 Коммуникации между GPU. FSDP

05 Кейс с MoE

06 Заключение

Определяемся с логистикой

Логистика

Что нужно для правильного построения логистики

- Нужна карта маршрутов.
- Нужно понимание того, сколько стоит пройти тот или иной маршрут.

Что нужно для правильного построения логистики

- Нужна карта маршрутов.
- Нужно понимание того, сколько стоит пройти тот или иной маршрут.

Как получить такие данные?

- Спросить у экспертов
- Почитать спецификацию
- Замерить самим

Что нужно для правильного построения логистики

- Нужна карта маршрутов.
- Нужно понимание того, сколько стоит пройти тот или иной маршрут.

Как получить такие данные?

- Спросить у экспертов
- Почитать спецификацию
- Замерить самим

Смотрим в спецификацию

	H100 SXM	H100 NVL
TF32 Tensor Core*	989 teraFLOPS	835 teraFLOPs
BFLOAT16 Tensor Core*	1,979 teraFLOPS	1,671 teraFLOPS
FP8 Tensor Core*	3,958 teraFLOPS	3,341 teraFLOPS
GPU Memory	80GB	94GB
GPU Memory Bandwidth	3.35TB/s	3.9TB/s
Interconnect	NVIDIA NVLink™: 900GB/s PCIe Gen5: 128GB/s	NVIDIA NVLink: 600GB/s PCIe Gen5: 128GB/s

* With sparsity

Смотрим в спецификацию

	H100 SXM	H100 NVL
TF32 Tensor Core*	989 teraFLOPS	835 teraFLOPs
BFLOAT16 Tensor Core*	1,979 teraFLOPS	1,671 teraFLOPS
FP8 Tensor Core*	3,958 teraFLOPS	3,341 teraFLOPS
GPU Memory	80GB	94GB
GPU Memory Bandwidth	3.35TB/s	3.9TB/s
Interconnect	NVIDIA NVLink™: 900GB/s PCIe Gen5: 128GB/s	NVIDIA NVLink: 600GB/s PCIe Gen5: 128GB/s

* With sparsity

Смотрим в спецификацию

	H100 SXM	H100 NVL
TF32 Tensor Core*	~~989 teraFLOPS~~ 400 TFLOPS	835 teraFLOPs
BFLOAT16 Tensor Core*	~~1,979 teraFLOPS~~ 800 TFLOPS	1,671 teraFLOPS
FP8 Tensor Core*	~~3,958 teraFLOPS~~ 1600 TFLOPS	3,341 teraFLOPS
GPU Memory	80GB	94GB
GPU Memory Bandwidth	~~3.35TB/s~~ 2.4 TB/sec	3.9TB/s
Interconnect	NVIDIA NVLink™~~: 900GB/s~~ 400 + 400 GB/sec PCIe Gen5: 128GB/s	NVIDIA NVLink: 600GB/s PCIe Gen5: 128GB/s

* With sparsity

Что нужно для правильного построения логистики

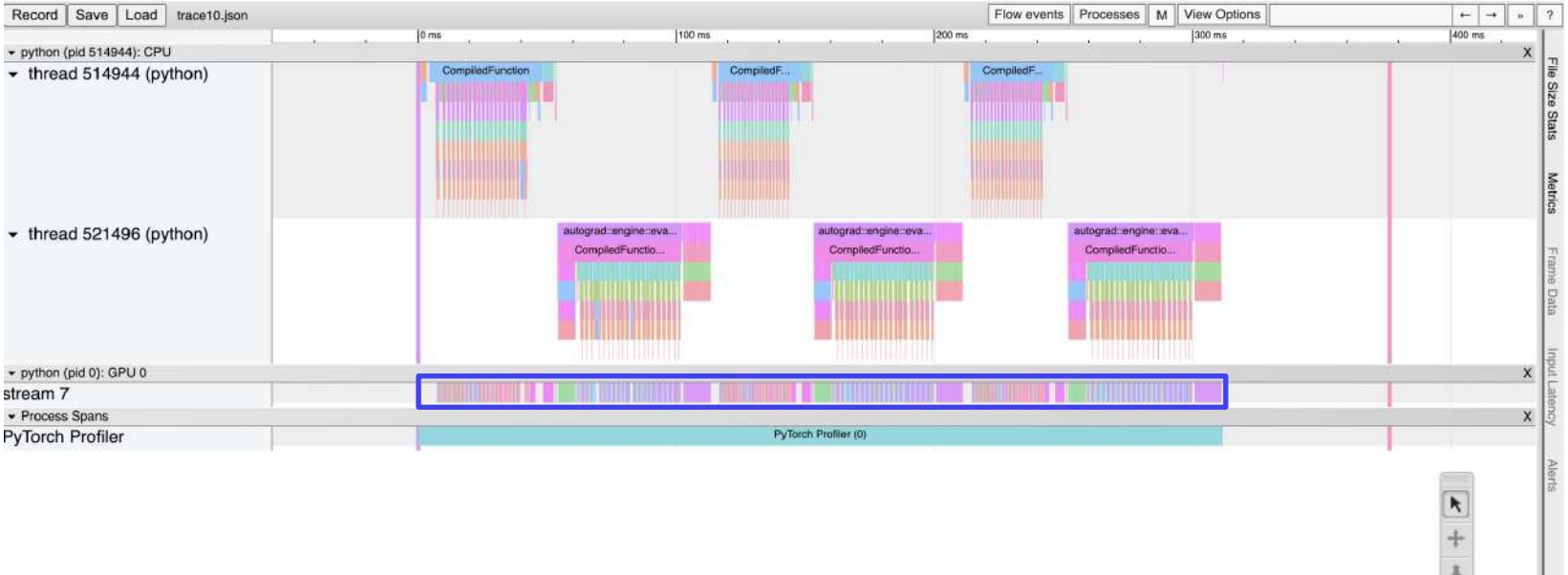
- Нужна карта маршрутов.
- Нужно понимание того, сколько стоит пройти тот или иной маршрут.

Как получить такие данные?

- Спросить у экспертов
- Почитать спецификацию
- Замерить самим

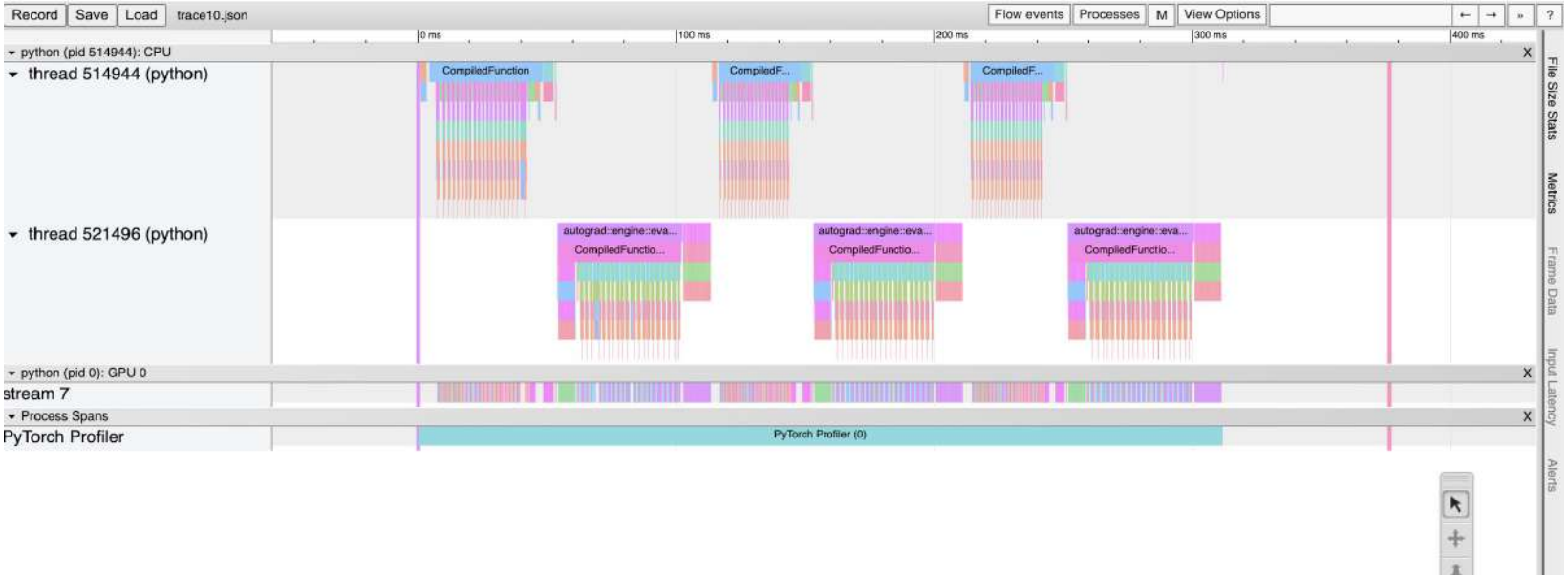
Как замерить время и память?

- Использовать torch.profiler или nsys.



Как замерить время и память?

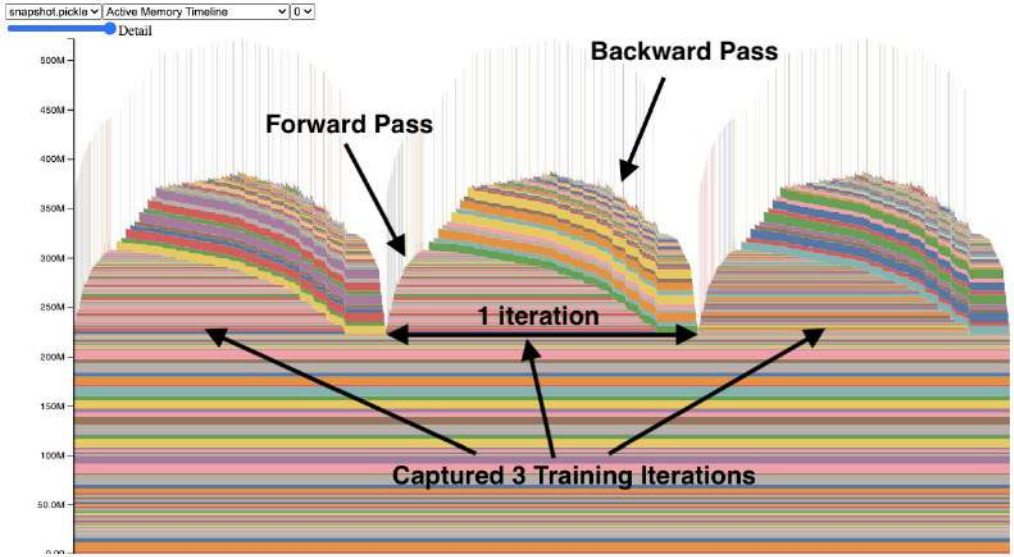
- Использовать torch.profiler или nsys.
  Важно: время CPU искажается.



Как замерить время и память?

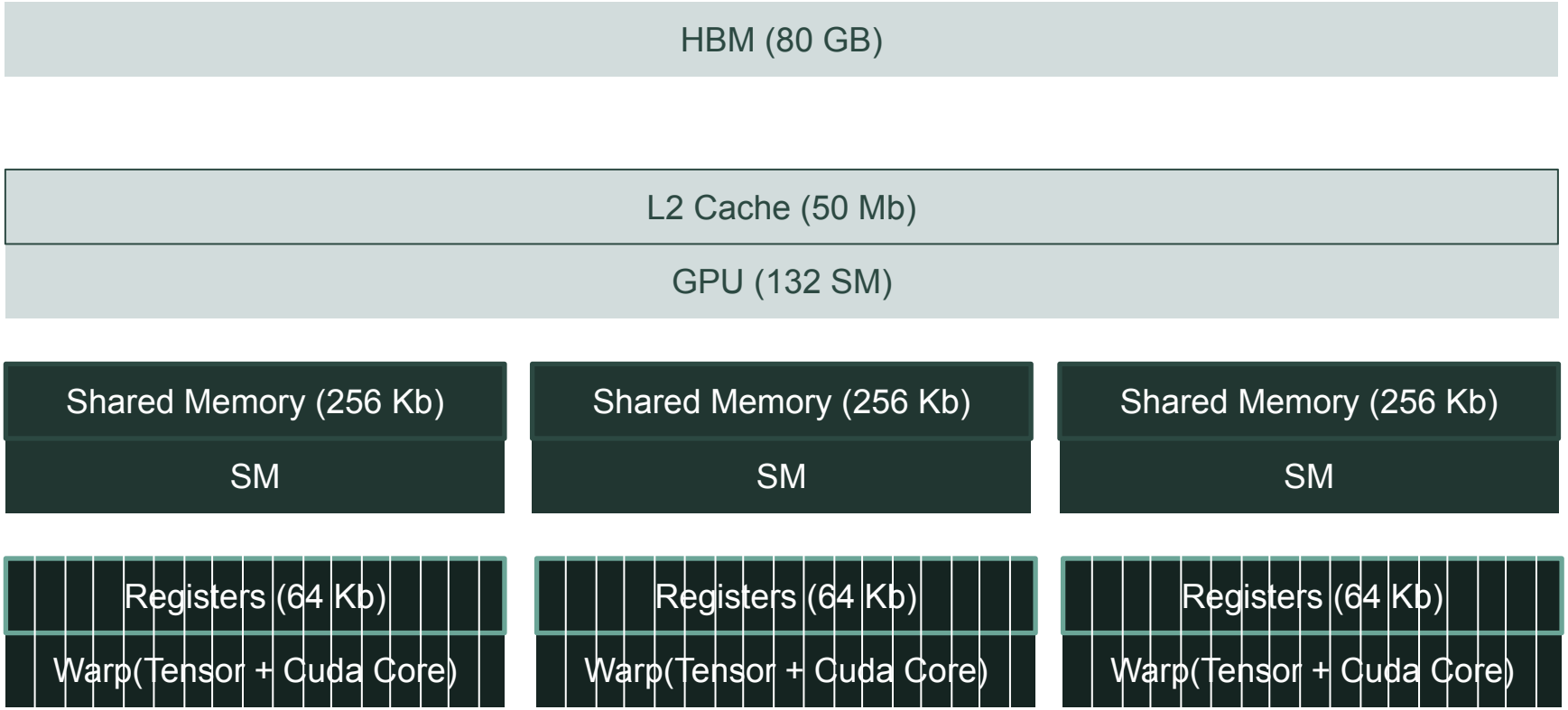
- Использовать torch.profiler или nsys.
  Важно: время CPU искажается.
- Снять профиль памяти

```
torch.cuda.empty_cache()
torch.cuda.memory._record_memory_history()

# ...

torch.cuda.memory._dump_snapshot(snapshot_path)
```

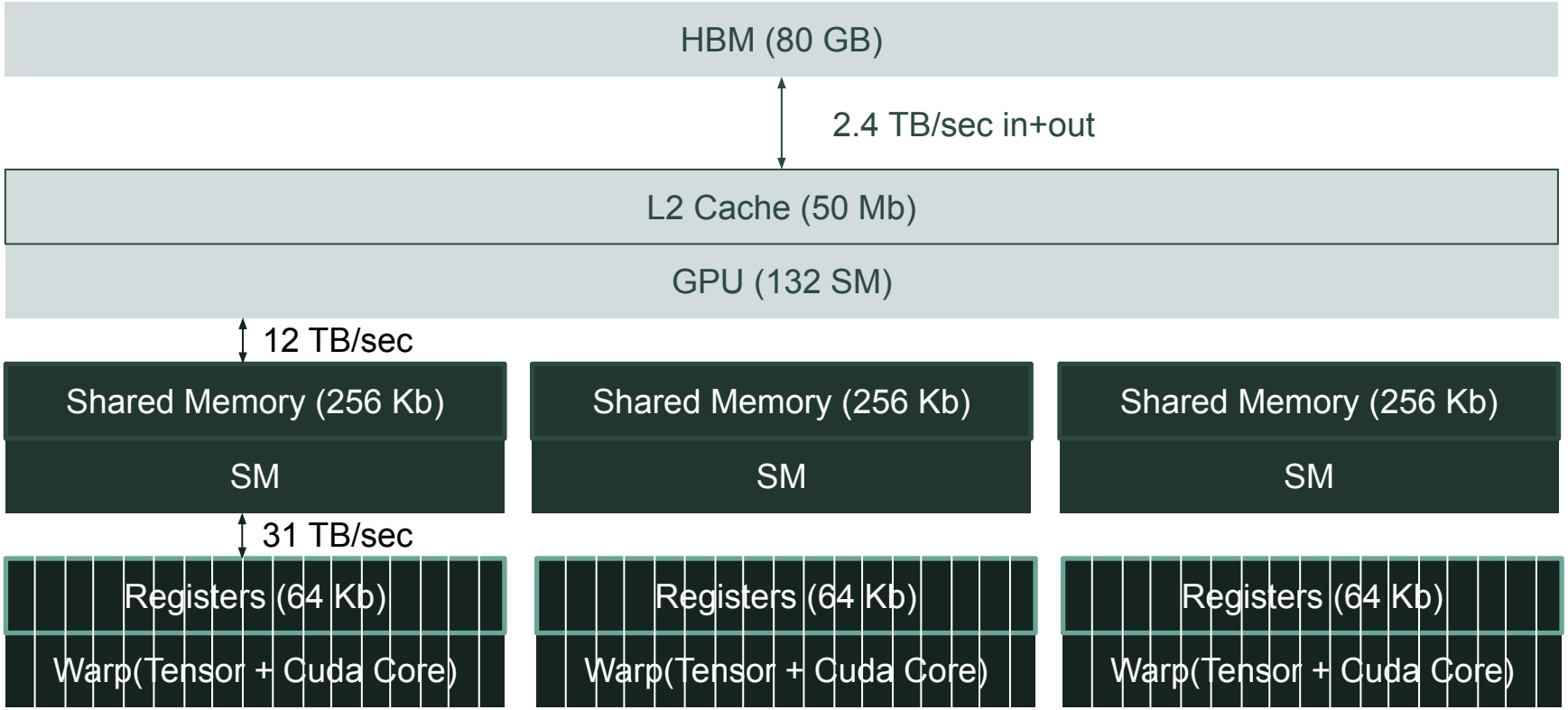


https://pytorch.org/memory_viz

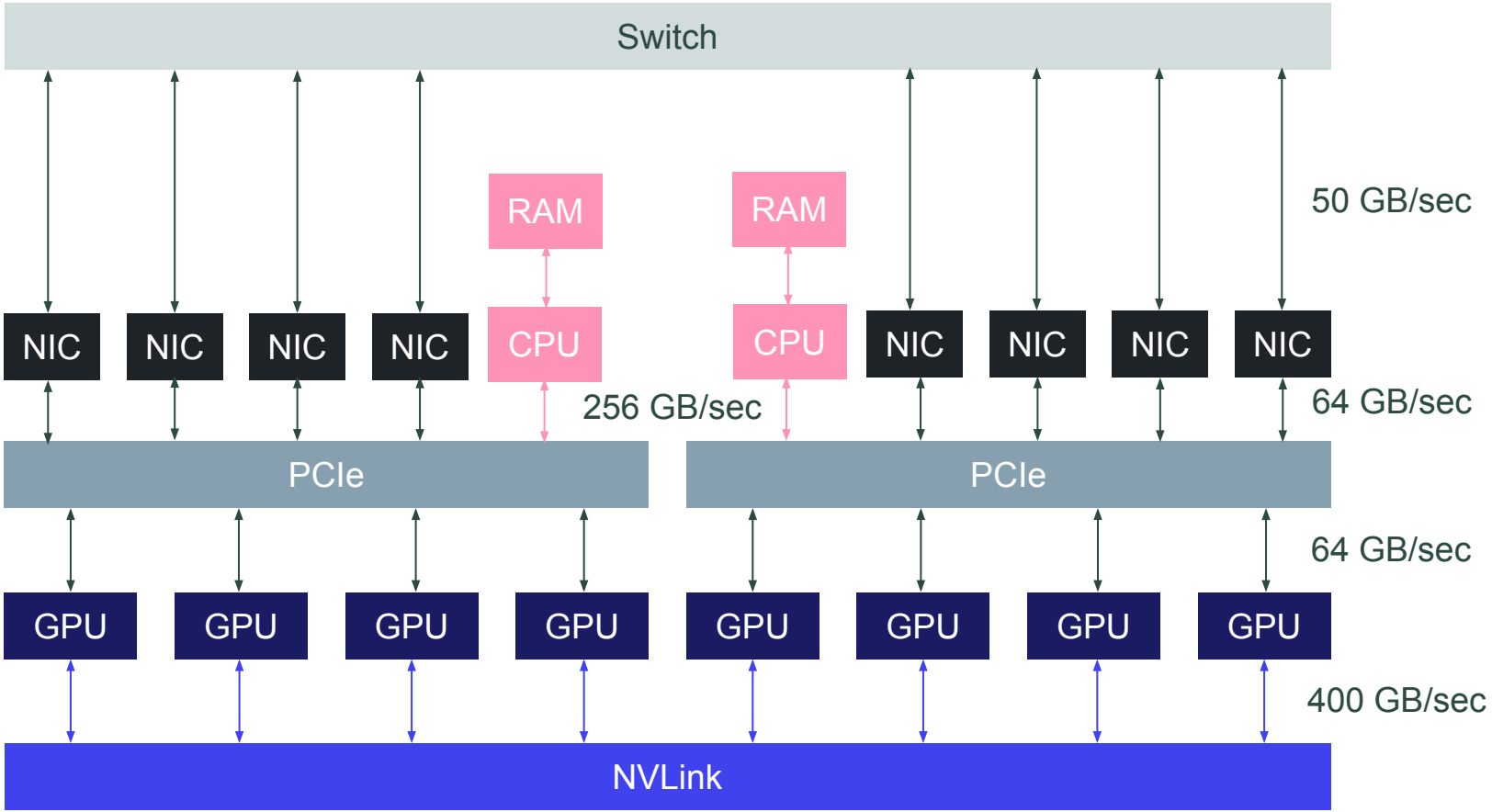
Карта GPU (H100)



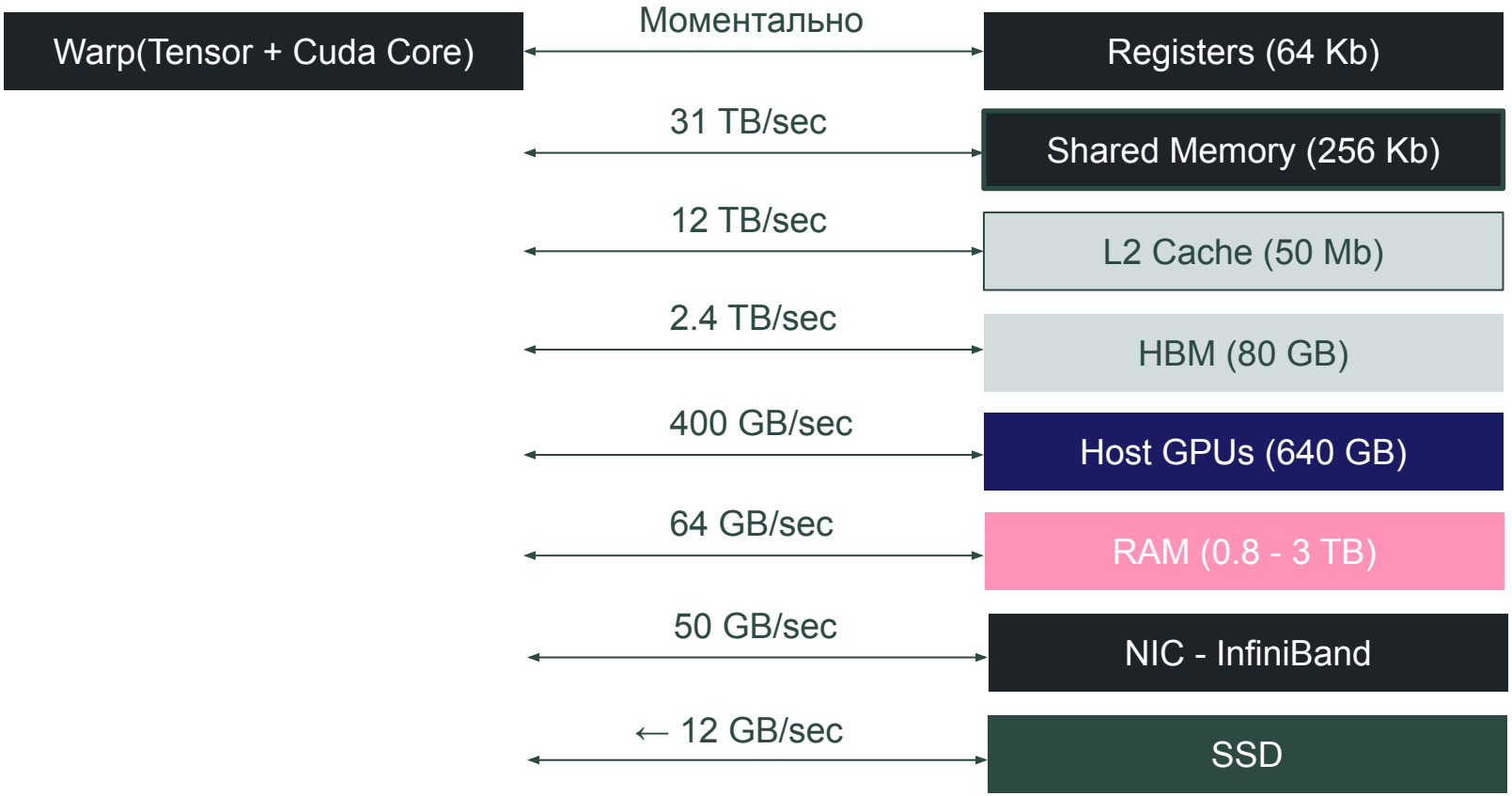
Карта GPU (H100)



Карта Хоста (H100 SXM)



Доступность памяти (H100)



Достаточно ли вводных?

С логистикой разобрались, осталось понять, сколько времени занимают вычисления на GPU:

- Вычисления на Tensor Cores (H100): 800 TFLOPS в bf16.
- Вычисления на CUDA Cores (H100): < 25 TFLOPS в bf16.

Достаточно ли вводных?

С логистикой разобрались, осталось понять, сколько времени занимают вычисления на GPU:

- Вычисления на Tensor Cores (H100): 800 TFLOPS в bf16.
- Вычисления на CUDA Cores (H100): < 25 TFLOPS в bf16.
- Запуск кернела 6-12 мкрсек.

Достаточно ли вводных?

С логистикой разобрались, осталось понять, сколько времени занимают вычисления на GPU:

- Вычисления на Tensor Cores (H100): 800 TFLOPS в bf16.
- Вычисления на CUDA Cores (H100): < 25 TFLOPS в bf16.
- Запуск кернела 6-12 мкрсек.

Мы построили карту логистики данных.
Пора делать что-то конкретное!

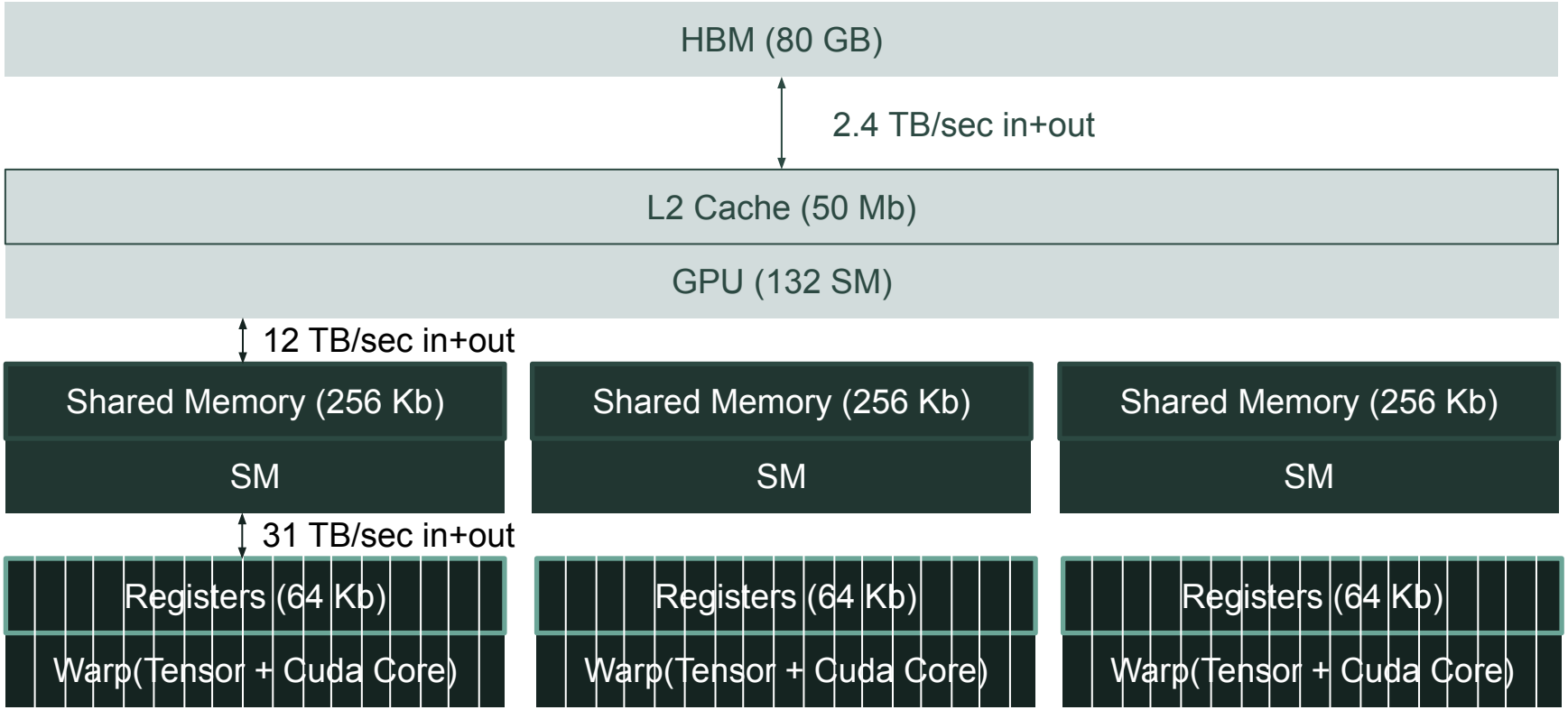
Локальная логистика на GPU

Упражнение 1

Пусть скорость HBM-памяти - 2.4 GB/sec, а скорость GPU в BF16 – 800TFLOPS/sec. Сколько по времени займет такая операция в BF16 (размерности тензоров – 8192x8192):

$$C = A + B$$

Карта GPU (H100)



Упражнение 1

Пусть скорость HBM-памяти - 2.4 GB/sec, а скорость GPU в BF16 – 800TFLOPS/sec. Сколько по времени займет такая операция в BF16 (размерности тензоров – 8192x8192):

C = A + B

- Время на загрузку и выгрузку: $$\frac{8192 \cdot 8192 \cdot 2 \cdot (2+1)}{2.4 \cdot 10^{12}} = 168\text{мкрсек}$$

Скорость вычислений на GPU

- Вычисления на Tensor Cores (H100): 800 TFLOPS в bf16.
- Вычисления на CUDA Cores (H100): < 25 TFLOPS в bf16.
- Запуск кернела 6-12 мкрсек.

Упражнение 1

Скорость HBM-памяти - 2.4 TB/sec, а скорость GPU в BF16 – 800 TFLOPS. Сколько по времени займет такая операция в BF16 (размерности тензоров – 8192x8192):

C = A + B

- Время на загрузку и выгрузку: $$\frac{8192 \cdot 8192 \cdot 2 \cdot (2+1)}{2.4 \cdot 10^{12}} = 168\text{мкрсек}$$
- Время на вычисление: $$\frac{8192 \cdot 8192}{2.5 \cdot 10^{12}} = 2.6\text{мкрсек}$$

Упражнение 1

Скорость HBM-памяти - 2.4 TB/sec, а скорость GPU в BF16 – 800 TFLOPS. Сколько по времени займет такая операция в BF16 (размерности тензоров – 8192x8192):

C = A + B

- Время на загрузку и выгрузку: $$\frac{8192 \cdot 8192 \cdot 2 \cdot (2+1)}{2.4 \cdot 10^{12}} = 168\text{мкрсек}$$
- Время на вычисление: $$\frac{8192 \cdot 8192}{2.5 \cdot 10^{12}} = 2.6\text{мкрсек}$$
- Итоговое время выполнения: 168+12 мкрсек – операция memory bound.

Упражнение 2

Скорость HBM-памяти - 2.4 TB/sec, а скорость GPU в BF16 – 800 TFLOPS. Сколько по времени займет такая операция в BF16 (размерности тензоров – 1024x1024):

C = A @ B

- Время на загрузку и выгрузку: $$\frac{1024 \cdot 1024 \cdot 2 \cdot (2+1)}{2.4 \cdot 10^{12}} = 2.6\text{мкрсек}$$
- Время на вычисление: $$\frac{1024 \cdot 1024 \cdot 1024 \cdot 2}{800 \cdot 10^{12}} = 2.6\text{мкрсек}$$
- Ожидаемое время выполнения: 2.6+**12** мкрсек.

Упражнение 3

Скорость HBM-памяти - 2.4 TB/sec, а скорость GPU в BF16 – 800 TFLOPS. Сколько по времени займет такая операция в BF16 (размерности тензоров – A - 4x8192, B – 8192x8192):

C = A @ B

- Время на загрузку и выгрузку: $$\frac{8192 \cdot 8192 \cdot 2 + 8192 \cdot 4 \cdot 2}{2.4 \cdot 10^{12}} = 56\text{мкрсек}$$
- Время на вычисление: $$\frac{4 \cdot 8192 \cdot 8192 \cdot 2}{800 \cdot 10^{12}} = 0.6\text{мкрсек}$$
- Ожидаемое время выполнения: 56+12 мкрсек.

Упражнение 3

Скорость HBM-памяти - 2.4 TB/sec, а скорость GPU в BF16 – 800 TFLOPS. Сколько по времени займет такая операция в BF16 (размерности тензоров – A - 4x8192, B – 8192x8192):

C = A @ B

- Время на загрузку и выгрузку: $$\frac{8192 \cdot 8192 \cdot 2 + 8192 \cdot 4 \cdot 2}{2.4 \cdot 10^{12}} = 56\text{мкрсек}$$
- Время на вычисление: $$\frac{4 \cdot 8192 \cdot 8192 \cdot 2}{800 \cdot 10^{12}} = 0.6\text{мкрсек}$$
- Ожидаемое время выполнения: 56+12 мкрсек.

При большом дисбалансе матричные умножения могут быть Memory Bound.

Пример Attention

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

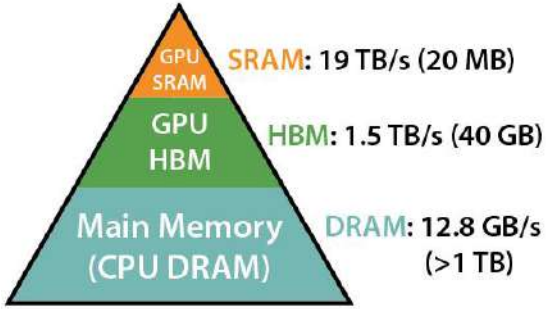
Упражнение 4. Attention

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

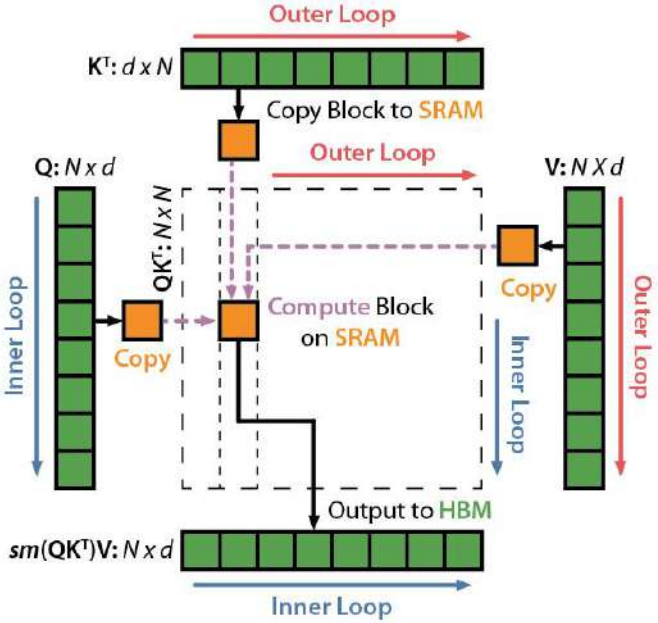
$S = 8192, H = 4096, nh = 64.$

- Линейные слои умножения: $\frac{4 \cdot S \cdot H \cdot H \cdot 2}{800 \cdot 10^{12}} = 1.4ms.$
- Матричные умножения в attention: $\frac{2 \cdot H \cdot S \cdot S \cdot (2+1)}{800 \cdot 10^{12}} = 2ms.$
- Scaling, softmax: $\frac{2 \cdot nh \cdot S \cdot S \cdot 2}{2.4 \cdot 10^{12}} = 7.1ms.$

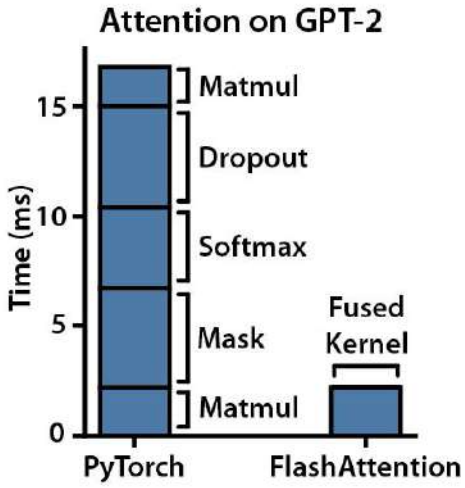
Flash Attention



Memory Hierarchy with Bandwidth & Memory Size

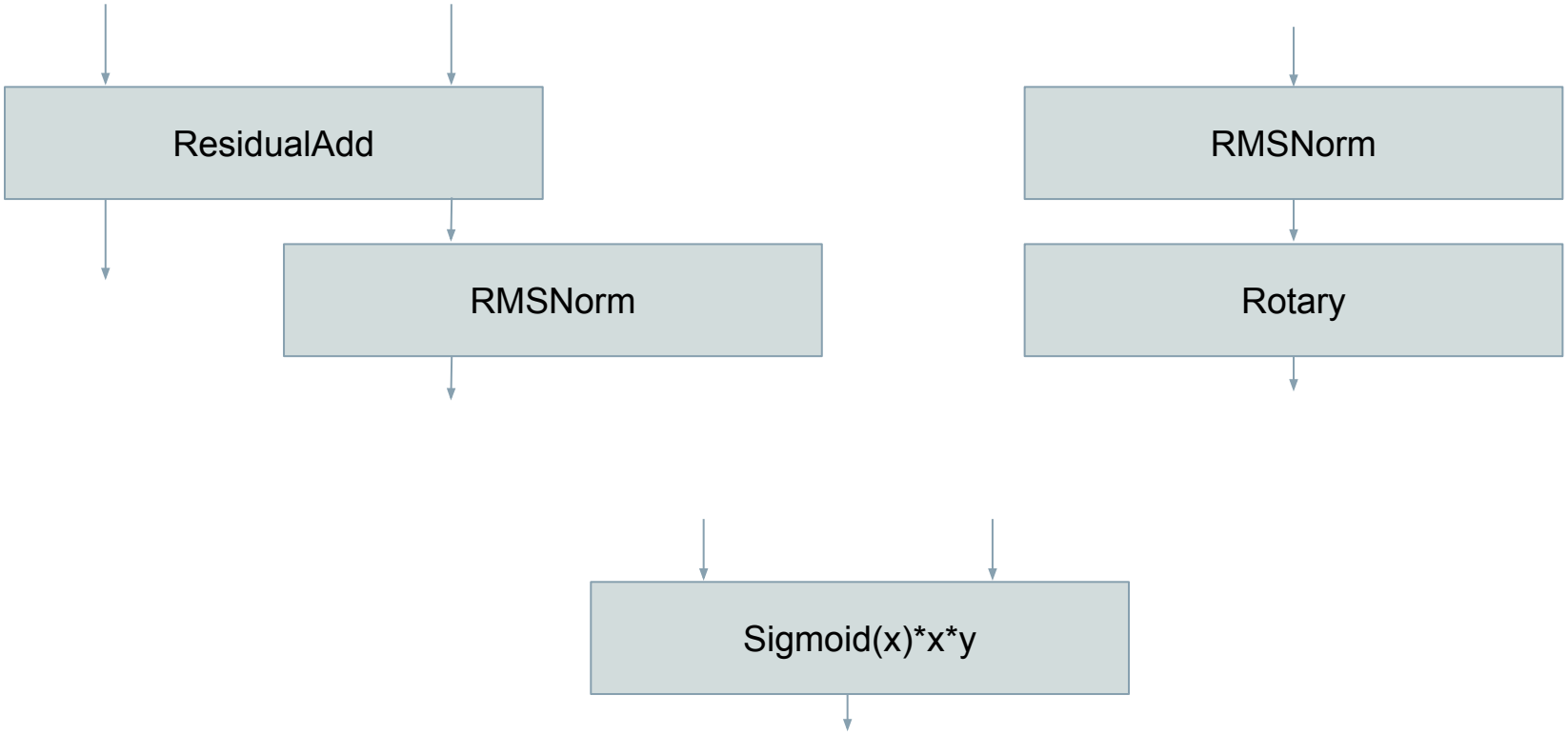


FlashAttention



Оптимизации elementwise

Последовательность elementwise-операций



Что делать с elementwise?

- torch.compile - собирает последовательности elementwise в triton-кернелы. Хорошо ужимает неэффективный код, но руками пока можно лучше.
- triton - позволяет писать код, близкий к torch, который выполняет матричные операции на SM’ках.
- CuTe - мощный (но непростой) инструмент для написания сложной логики.
- CUDA - C на уровне SM-thread.

Где можно достать хорошие кернелы?

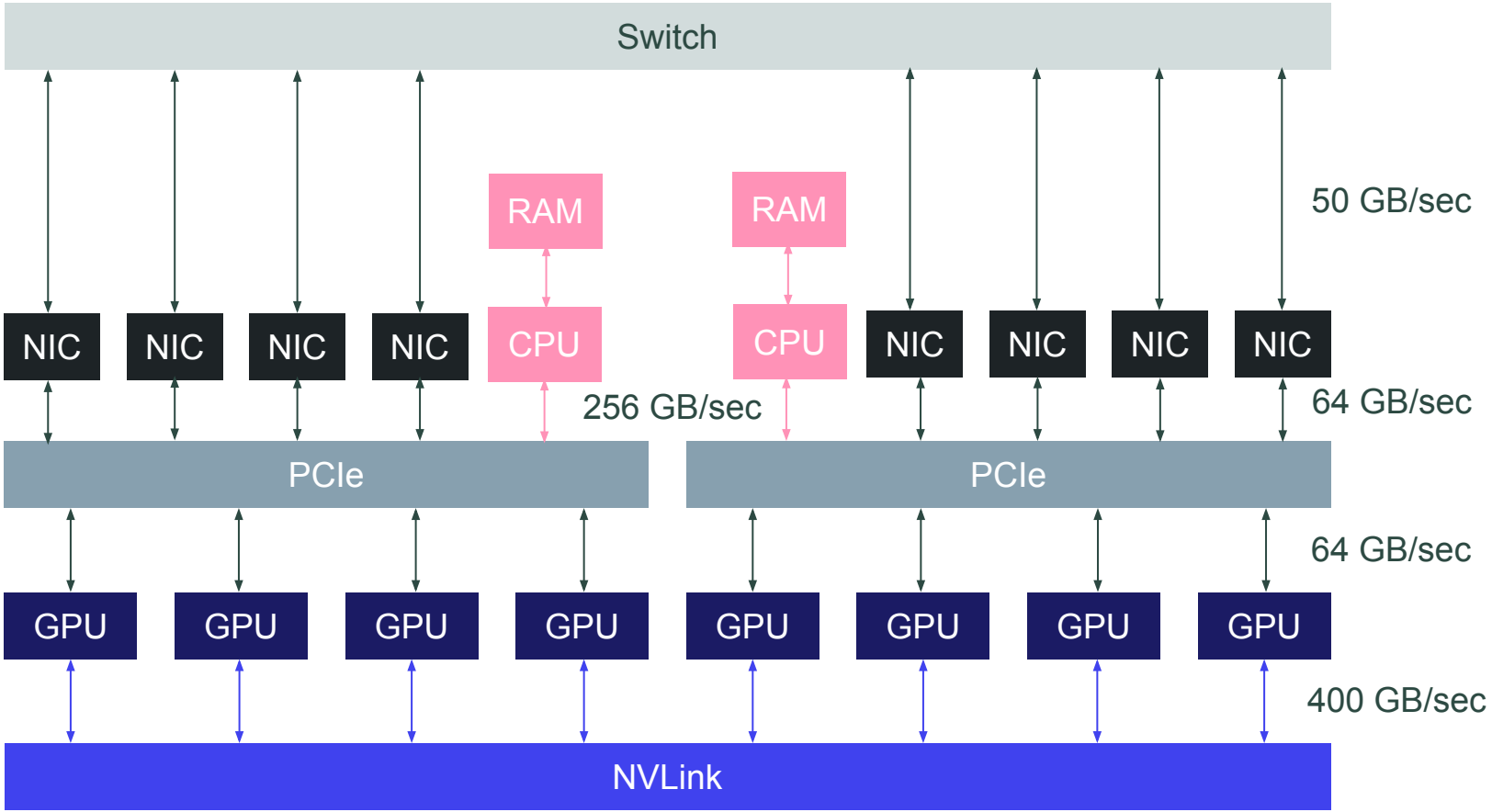
- Liger-Kernel
  - Все базовые сценарии
  - Эффективная реализация Cross Entropy Loss
- Apex
  - Старый, но memory-efficient RMSNorm
- TransformerEngine
  - Большой набор инструментов для быстрого обучения и квантизации
- FlashAttention, SonicMoE

Выводы

- Даже на одной GPU важно просчитывать логистику.
- Memory bound операции могут занимать существенное время, не загружая GPU. Особенно, если они следуют друг за другом
- Матричные умножения могут стать неэффективными на маленьких размерностях или при дизбалансе размеров матриц.
- Для многих операций уже реализованы хорошие кернелы — их стоит изучать и использовать.

Коммуникации между GPU

Карта Хоста (H100 SXM)

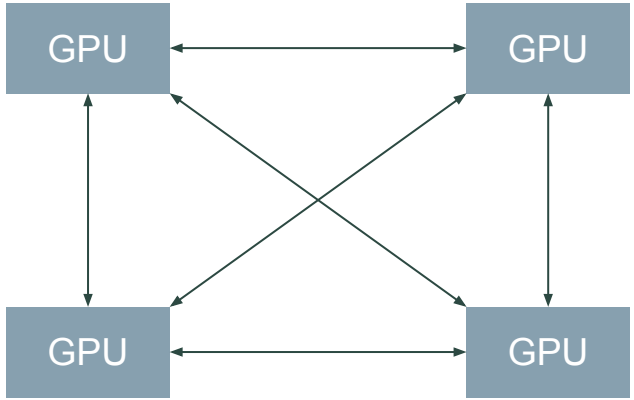


Типы коммуникаций

- Симметричные
- Ассиметричные

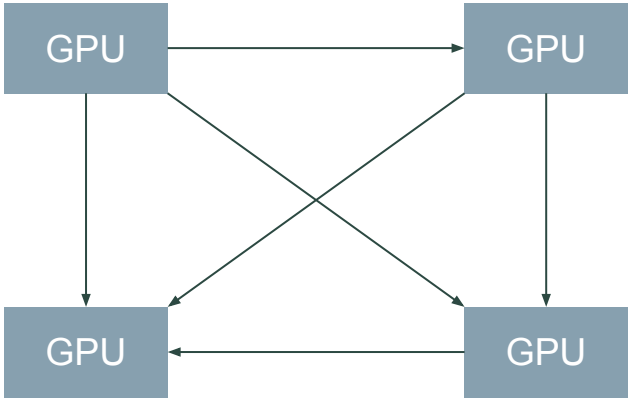
Типы коммуникаций

- **Симметричные**
- Ассиметричные



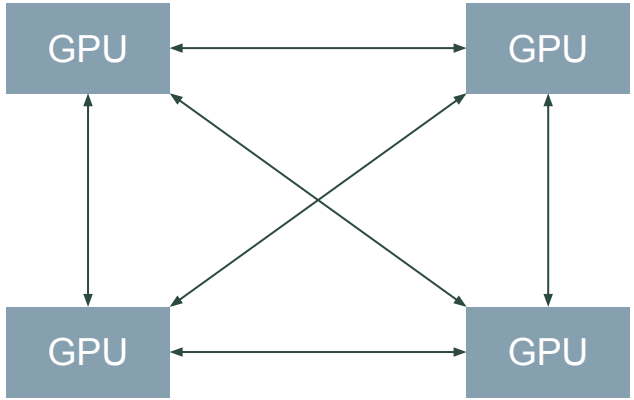
Типы коммуникаций

- Симметричные
- **Ассиметричные**

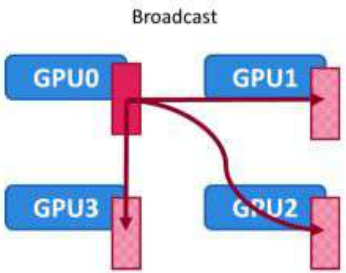


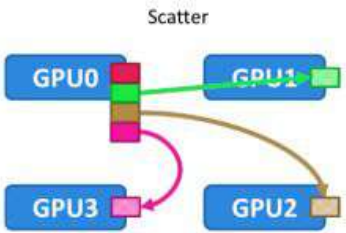
Типы коммуникаций

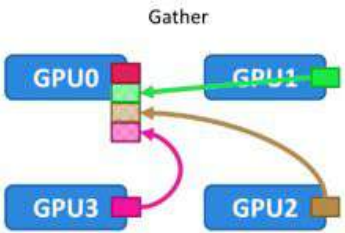
- **Симметричные**
- Ассиметричные

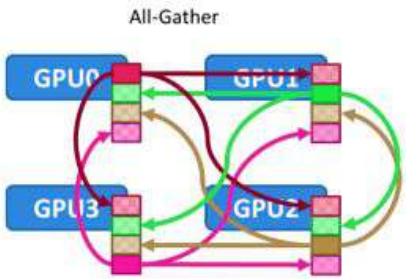


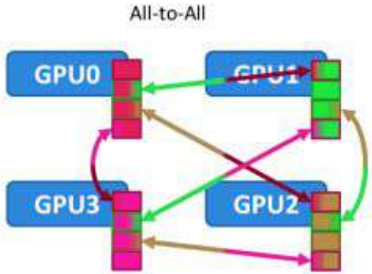
NCCL

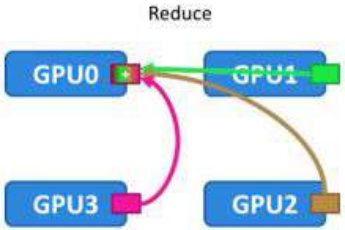

Broadcast
GPU0
GPU1
GPU3
GPU2

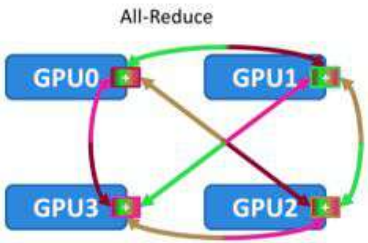

Scatter
GPU0
GPU1
GPU3
GPU2


Gather
GPU0
GPU1
GPU3
GPU2

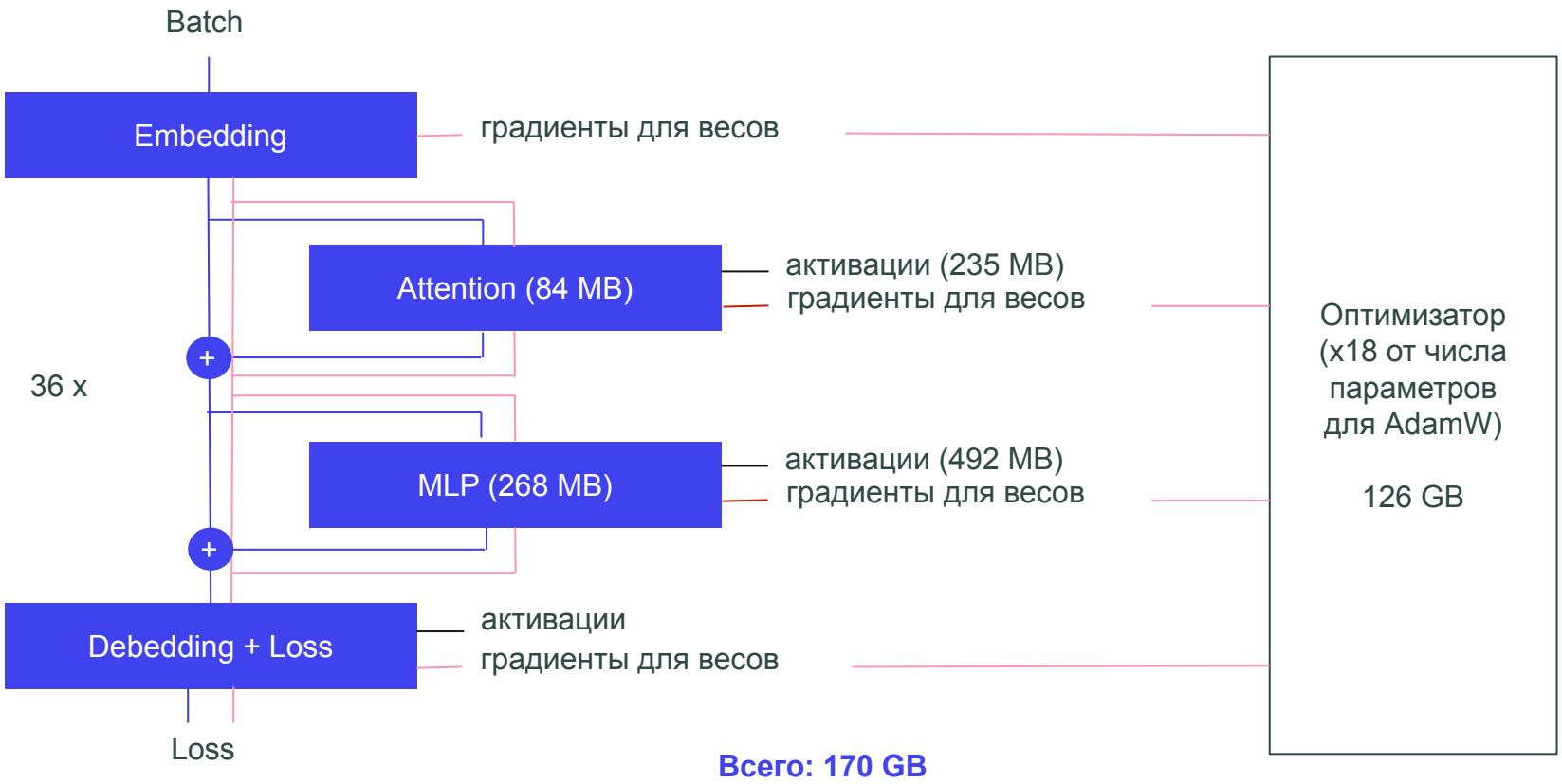

All-Gather
GPU0
GPU1
GPU3
GPU2


All-to-All
GPU0
GPU1
GPU3
GPU2


Reduce
GPU0
GPU1
GPU3
GPU2


All-Reduce
GPU0
GPU1
GPU3
GPU2

7B модель



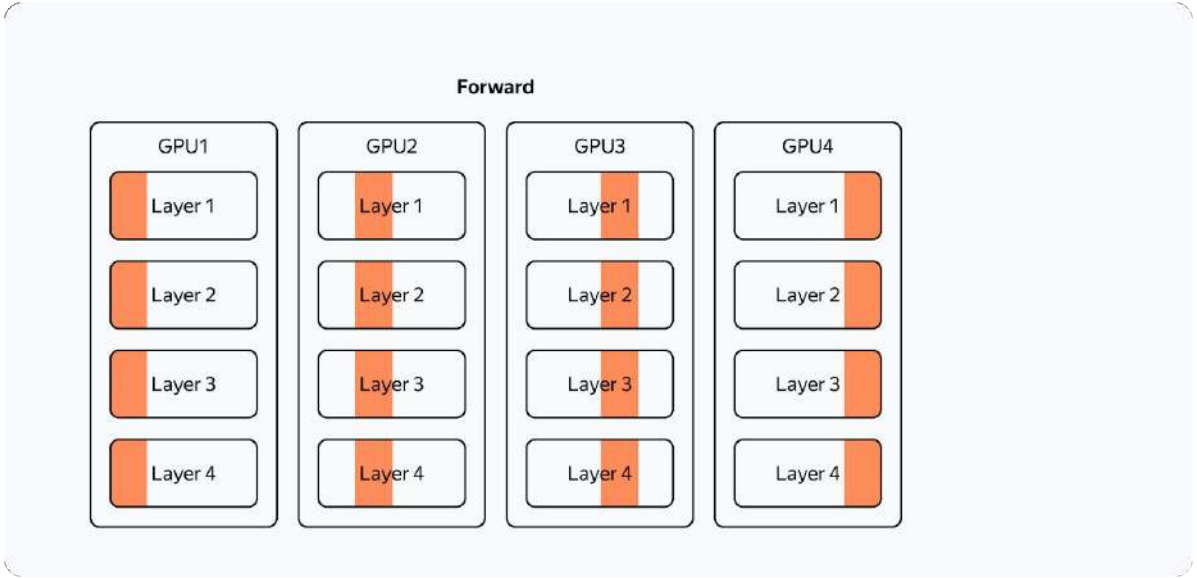
Всего: 170 GB

Бьем состояния оптимизатора на разные GPU

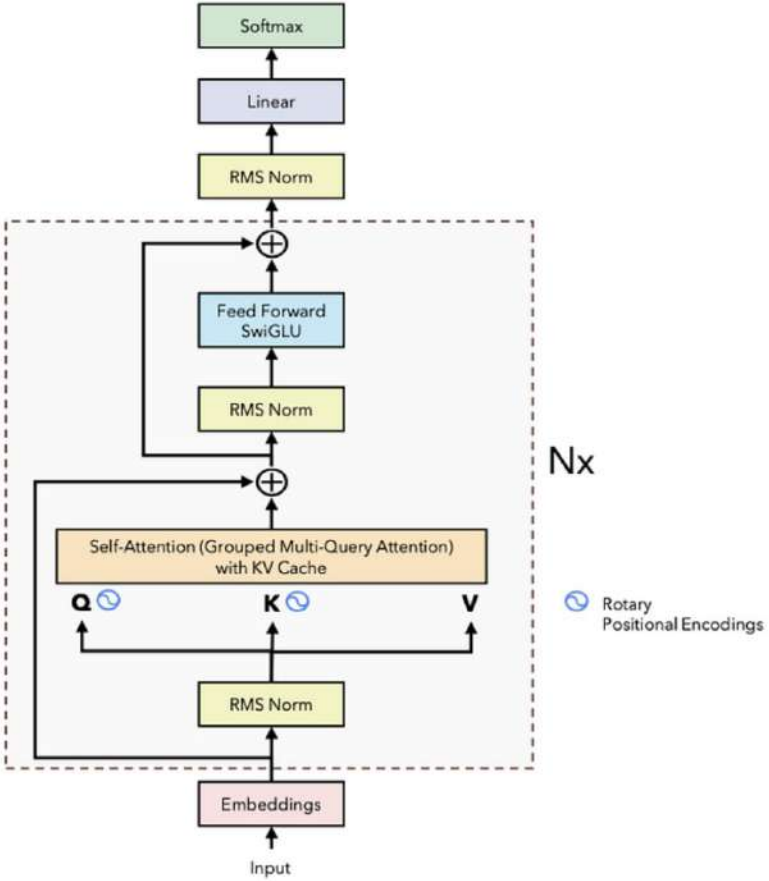
- … а заодно и веса.
- Собираем веса перед каждым forward.
- Усредняем градиенты после каждого backward.

FSDP

- Разделим веса и состояния оптимизатора между процессами.
- Сделаем коммуникации асинхронными.



Математика (Llama 7B)



Время forward одного слоя 7B

- S = 8192 токенов, H = 4096, I = 11008, $nh \cdot dh$ = 4096.
- Суммарное время матричных умножений:

$$\frac{S \cdot H \cdot (nh \cdot dh \cdot 4 \cdot 2 + I \cdot 3 \cdot 2 + 3 \cdot S)}{800 \cdot 10^{12}} = 5.1ms.$$

- Время memory bound операций (2 RMSNorm, 2 Add, 1 SwiGLU):

$$\frac{S \cdot H \cdot 2 \cdot (2 \cdot 2 + 2 \cdot 3) + S \cdot I \cdot 2 \cdot 3}{2.4 \cdot 10^{12}} = 0.5ms.$$

- Время на AllGather FSDP (в одном слое 400MB):

$$\frac{(H \cdot nh \cdot dh \cdot 4 + H \cdot I \cdot 3) \cdot 2}{400 \cdot 10^{9}} = 1.34ms.$$

Время forward одного слоя 7B

- S = 8192 токенов, H = 4096, I = 11008, $nh \cdot dh$ = 4096.
- Суммарное время матричных умножений:

$$\frac{S \cdot H \cdot (nh \cdot dh \cdot 4 \cdot 2 + I \cdot 3 \cdot 2 + 3 \cdot S)}{800 \cdot 10^{12}} = 5.1ms.$$

- Время memory bound операций (2 RMSNorm, 2 Add, 1 SwiGLU):

$$\frac{S \cdot H \cdot 2 \cdot (2 \cdot 2 + 2 \cdot 3) + S \cdot I \cdot 2 \cdot 3}{2.4 \cdot 10^{12}} = 0.5ms.$$

- Время на AllGather FSDP (в одном слое 400MB):

$$\frac{(H \cdot nh \cdot dh \cdot 4 + H \cdot I \cdot 3) \cdot 2}{400 \cdot 10^{9}} = 1.34ms.$$

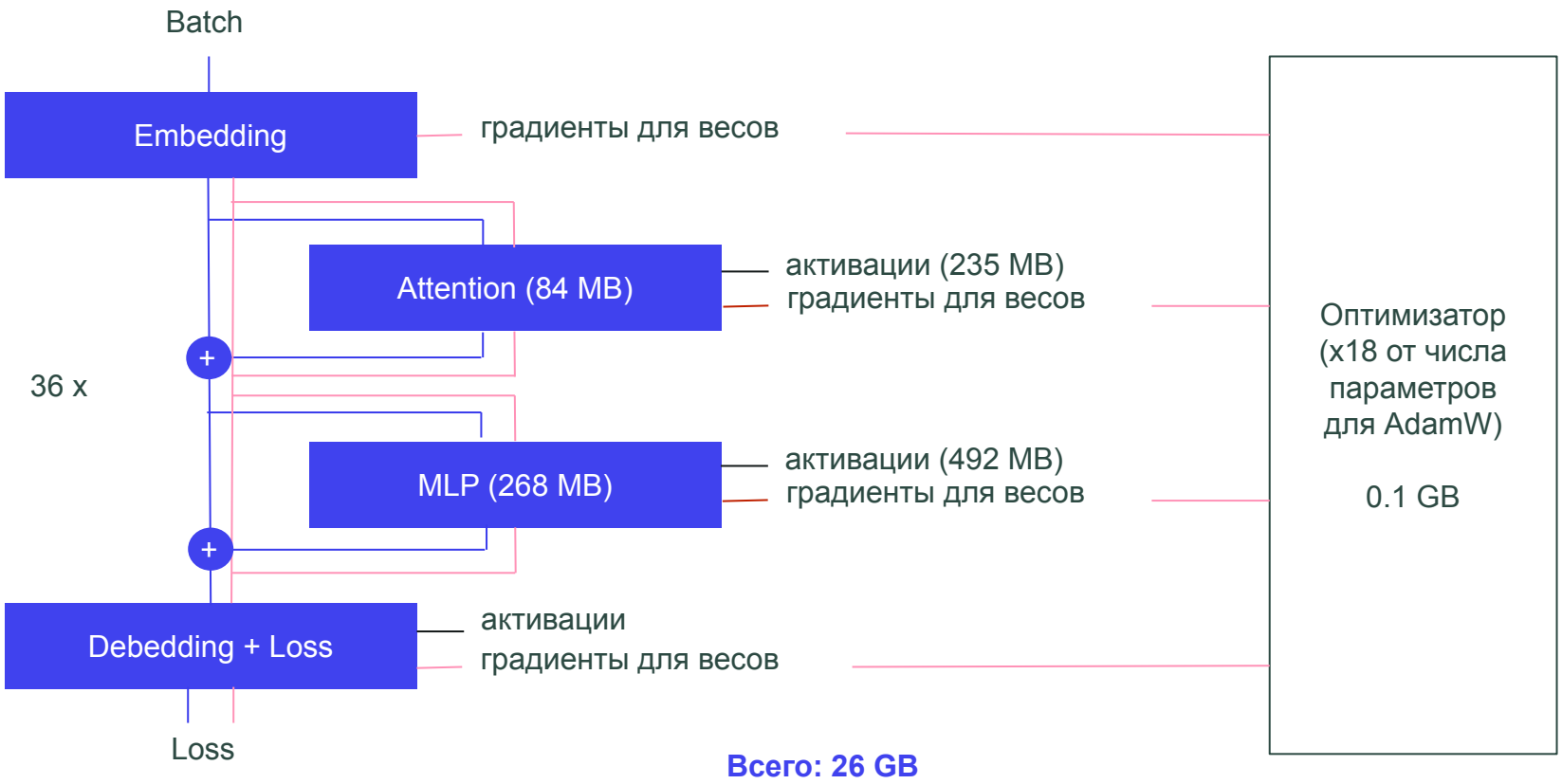
Особенности коммуникаций в NCCL

```python
def all_to_all_cost(bytes_gb: float, mesh_topo: MeshTopoInfo, mesh_dim: int) -> float:
    num_devices_on_mesh_dim = mesh_topo.mesh_dim_devices[mesh_dim]
    mesh_dim_bandwidth = mesh_topo.mesh_dim_bandwidth[mesh_dim]
    num_hops = num_devices_on_mesh_dim - 1
    # base latency + comm latency
    latency = 6.6 + num_hops * mesh_topo.mesh_dim_latency[mesh_dim]  # us
    bw = (bytes_gb * num_hops / num_devices_on_mesh_dim) / mesh_dim_bandwidth  # s
    total_time = latency + bw * 1e6  # rescale to us
```

Особенности коммуникаций в NCCL

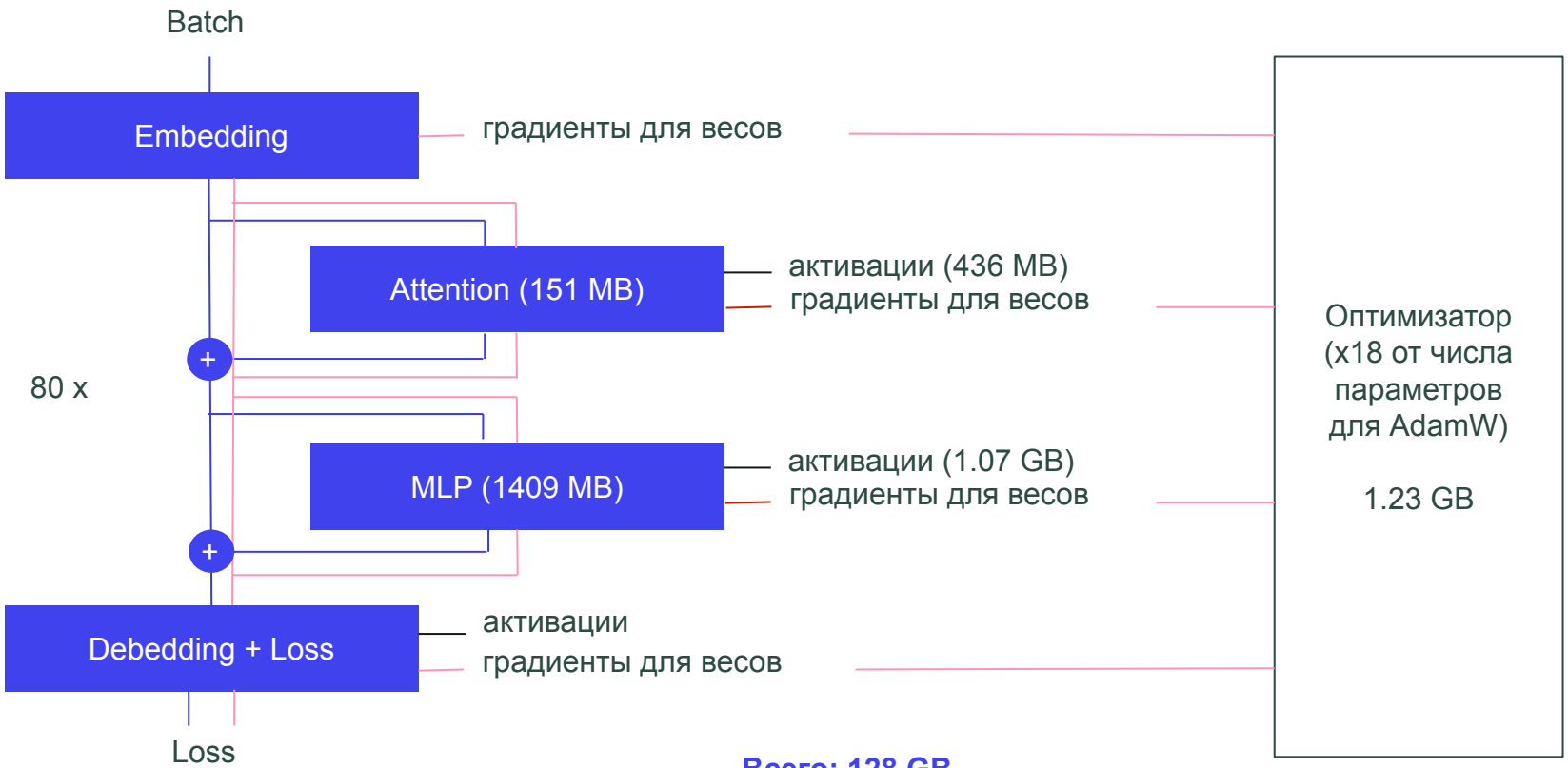
```python
def all_to_all_cost(bytes_gb: float, mesh_topo: MeshTopoInfo, mesh_dim: int) -> float:
    num_devices_on_mesh_dim = mesh_topo.mesh_dim_devices[mesh_dim]
    mesh_dim_bandwidth = mesh_topo.mesh_dim_bandwidth[mesh_dim]
    num_hops = num_devices_on_mesh_dim - 1
    # base latency + comm latency
    latency = 6.6 + num_hops * mesh_topo.mesh_dim_latency[mesh_dim]  # us
    bw = (bytes_gb * num_hops / num_devices_on_mesh_dim) / mesh_dim_bandwidth  # s
    total_time = latency + bw * 1e6  # rescale to us
    # FIXME: this is a hack, we need to spend some more effort on the cost model
    total_time *= 5
    return total_time
```

7B модель на 1024 GPU



Всего: 26 GB

Llama 70B на 1024 GPU

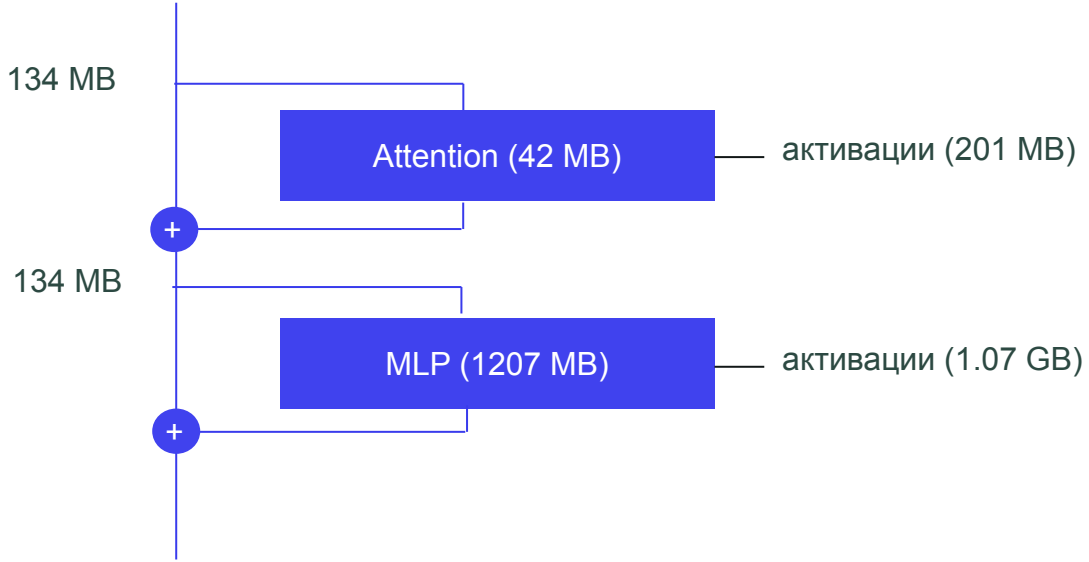


Всего: 128 GB
Активации: 120.8 GB

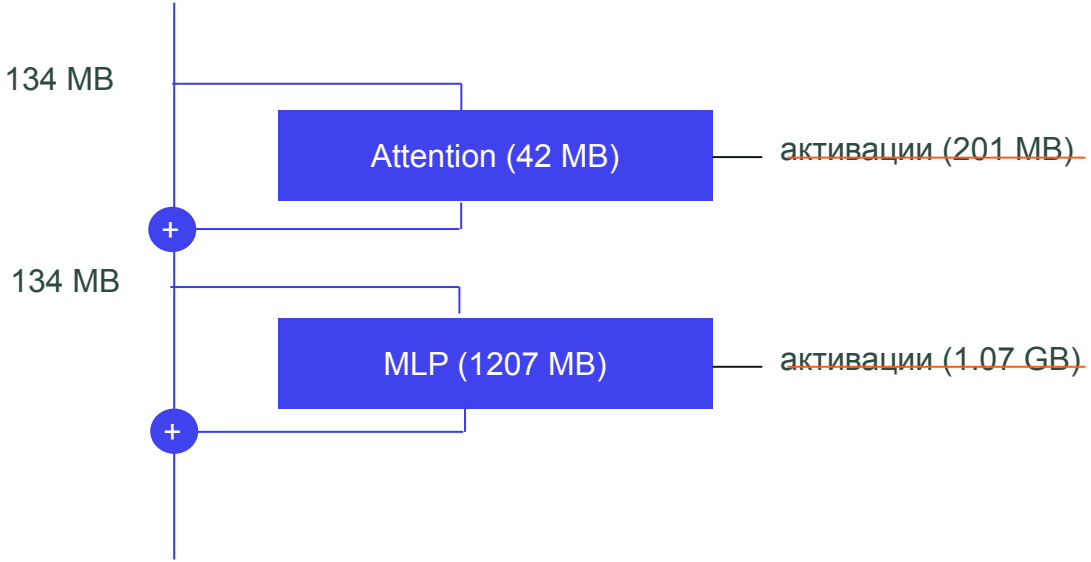
2 варианта логистики активаций

- Через перевычисление (чекпоинт активаций).
- Через пересылку.

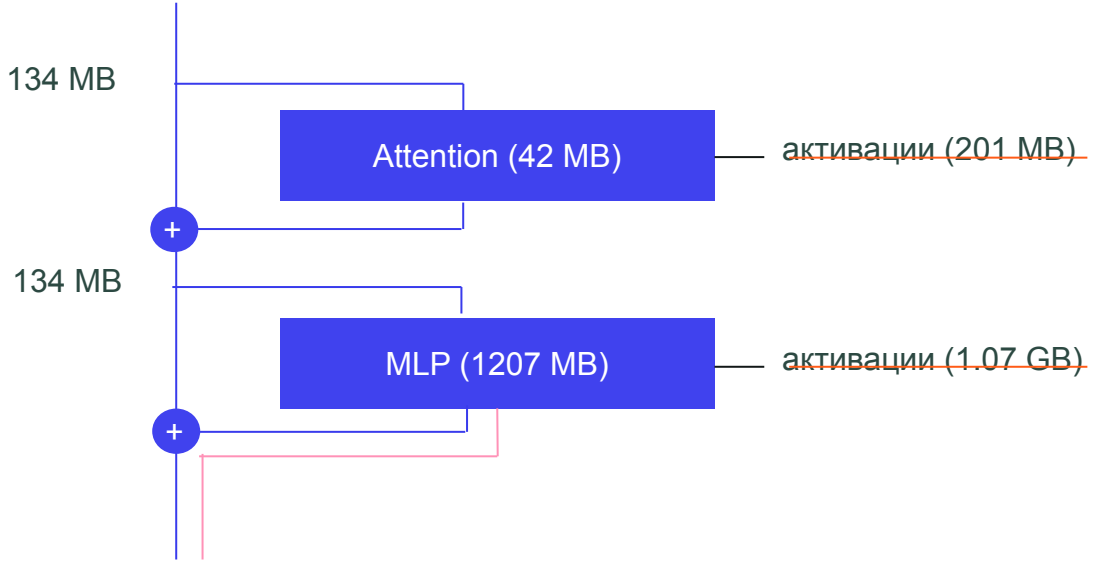
Чекпоинт активаций



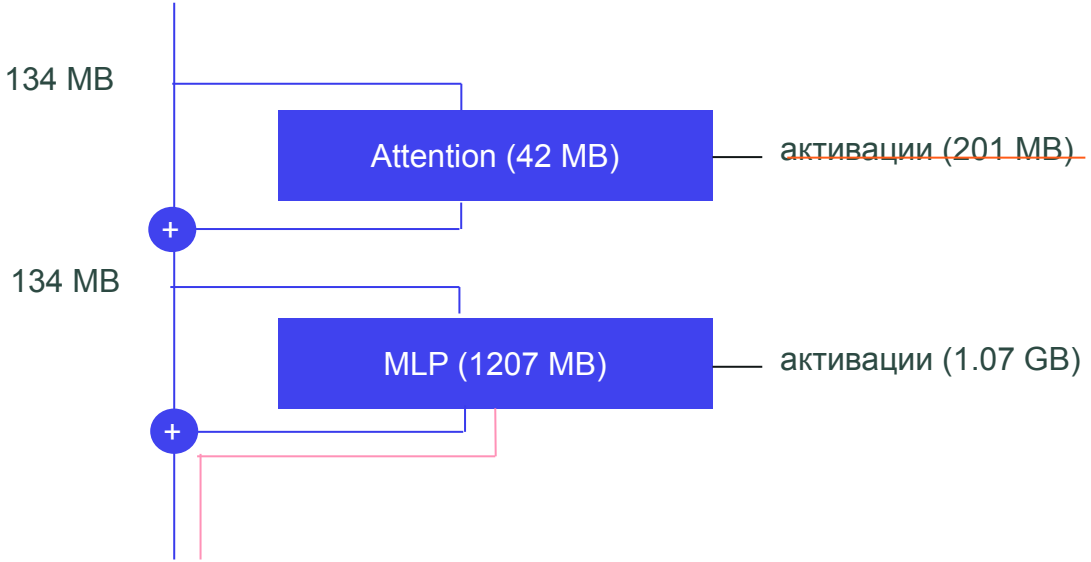
Чекпоинт активаций



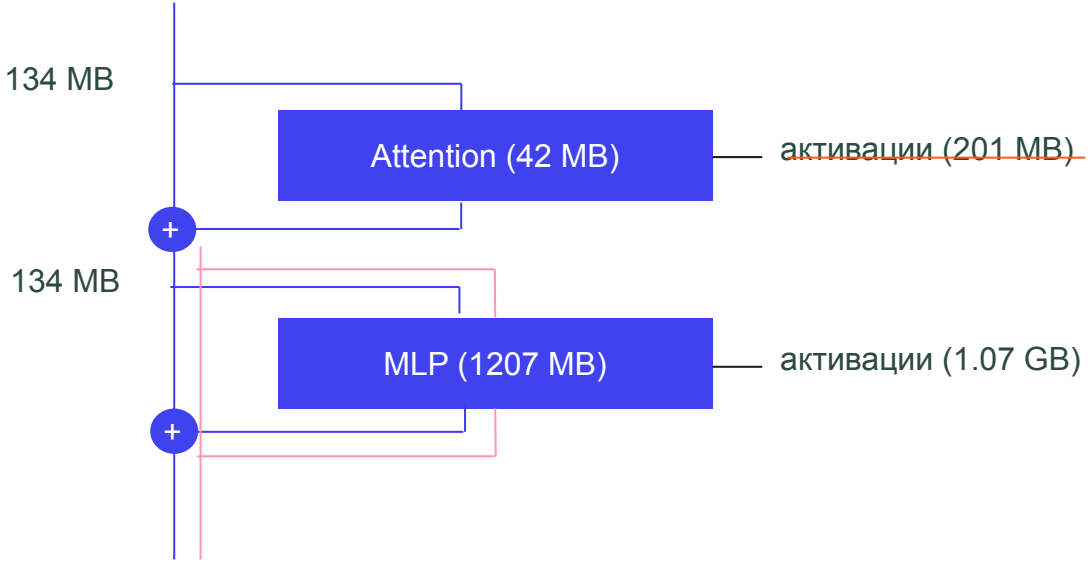
Чекпоинт активаций



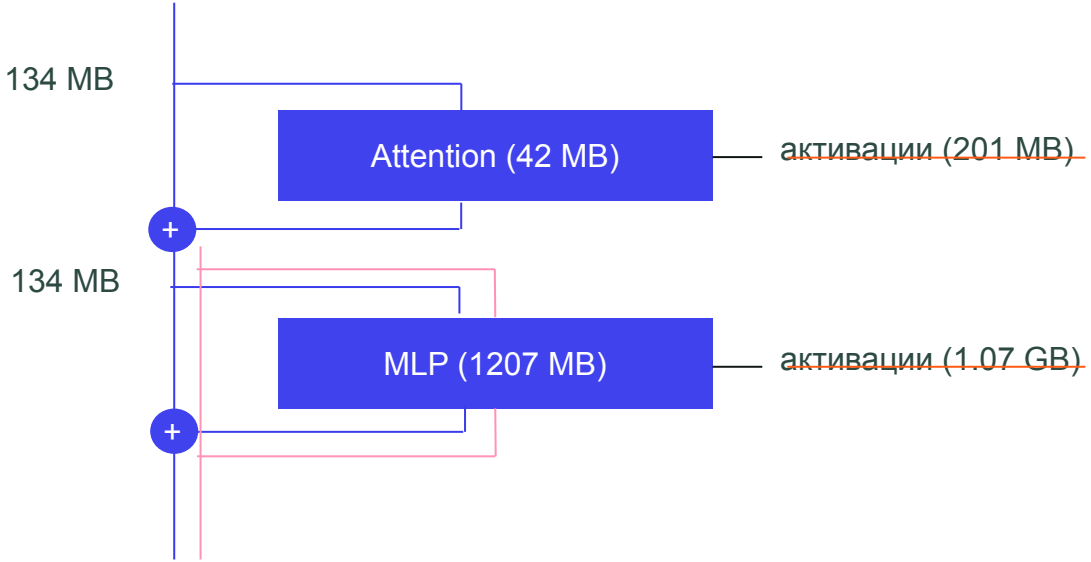
Чекпоинт активаций



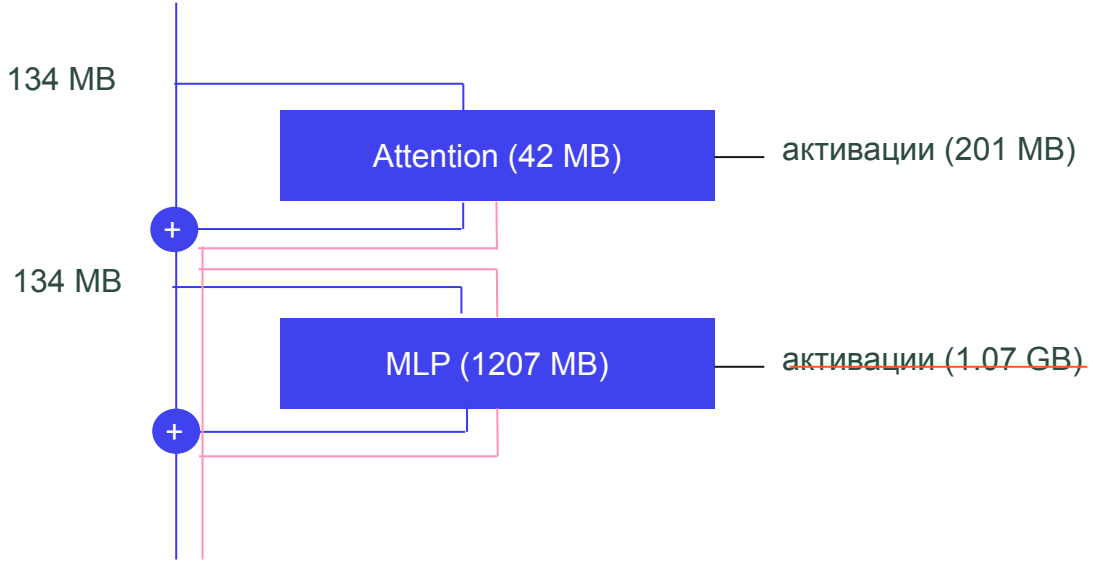
Чекпоинт активаций



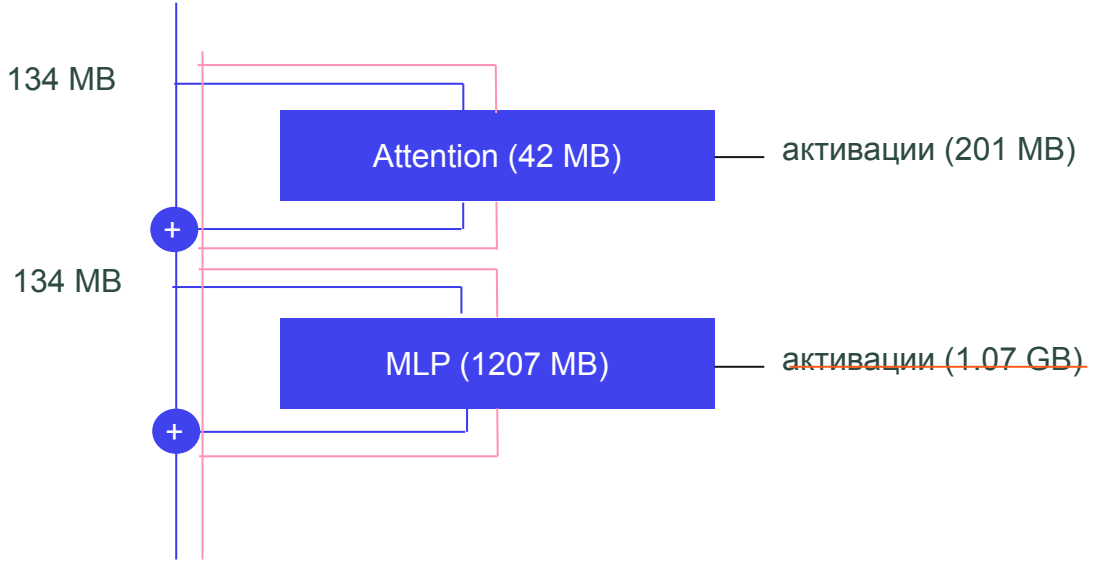
Чекпоинт активаций



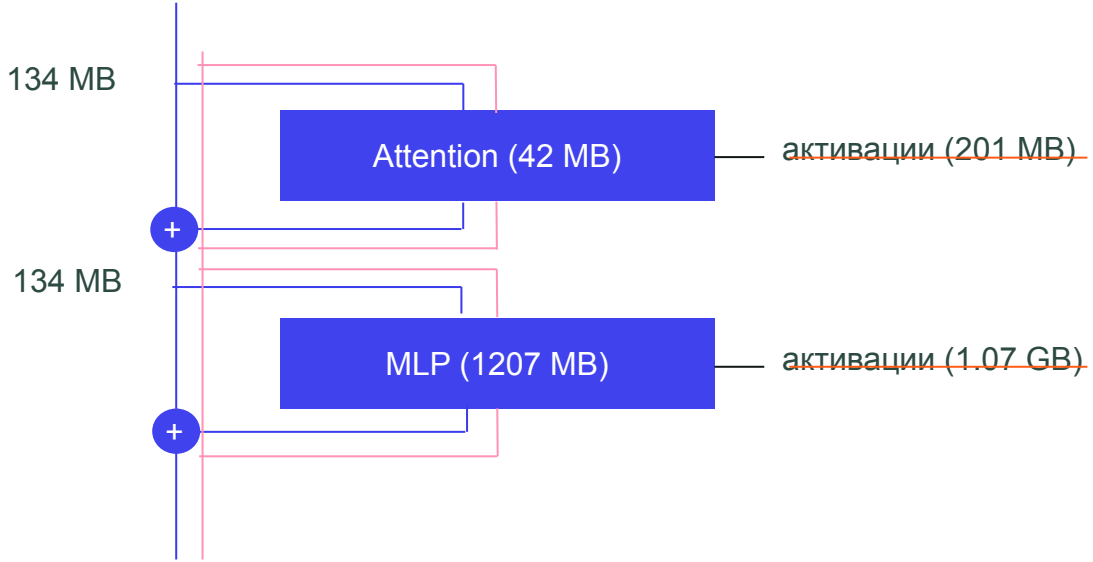
Чекпоинт активаций



Чекпоинт активаций

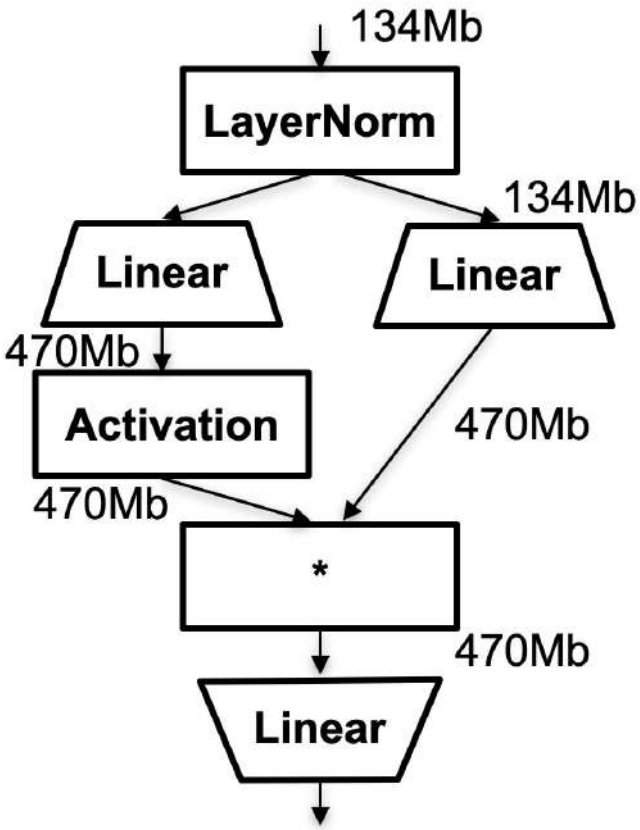


Чекпоинт активаций



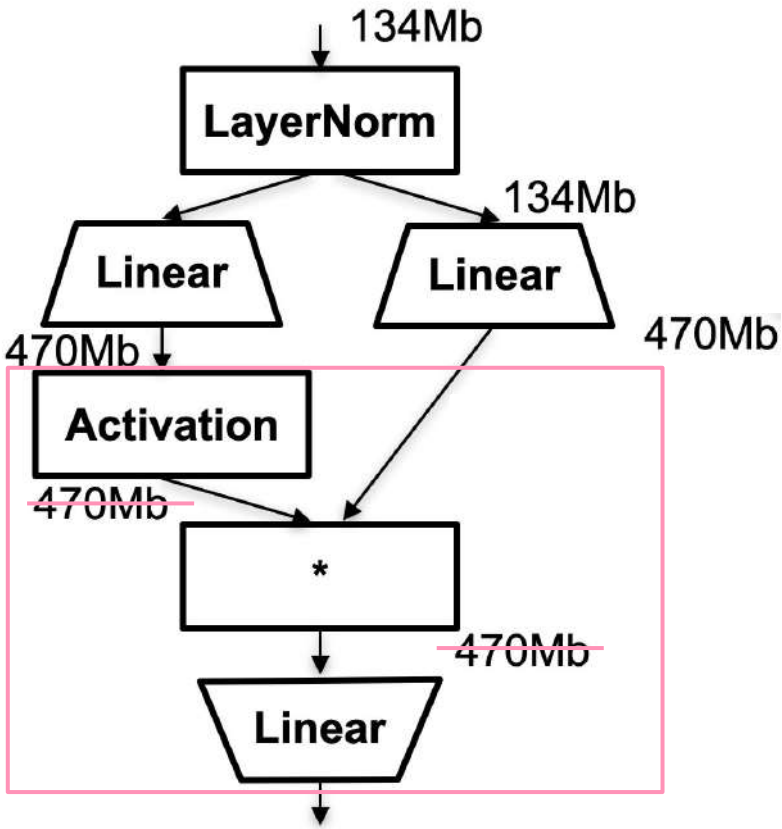
Частичный рекомпьют

- Не все активации одинаково дорого вычислять



Частичный рекомпьют

- Не все активации одинаково дорого вычислять



2 варианта логистики активаций

- Через перевычисление (чекпоинт активаций).
- Через пересылку (Тензорный параллелизм).

Тензорный параллелизм

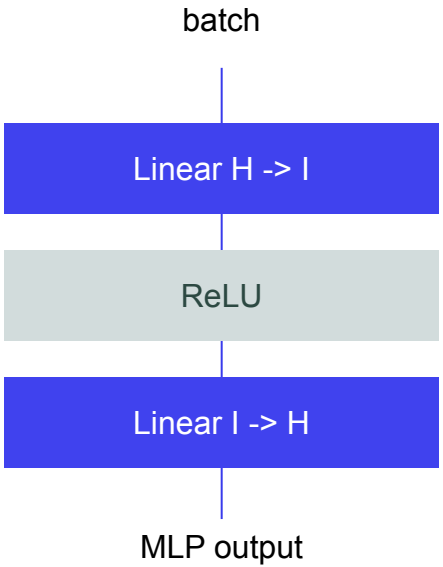
Как было раньше:

- Шардируем только веса, собираем их по необходимости.

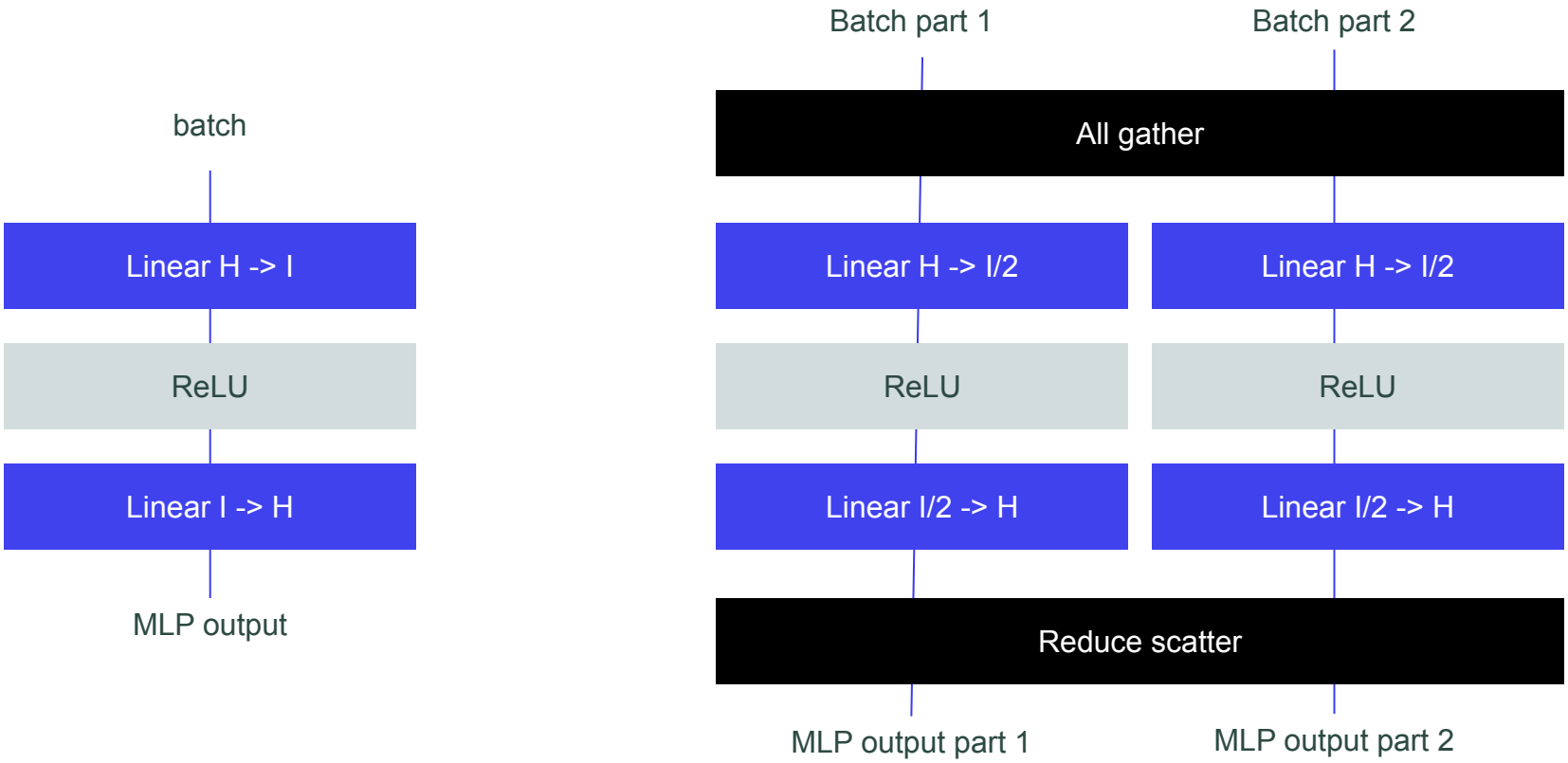
Как хотим:

- Шардируем веса и активации, собираем их.

Тензорный параллелизм



Тензорный параллелизм



Время forward одного слоя 70B (TP=2)

- S = 4096 токенов, H = 8192, I = 28672, $nh \cdot dh$ = 8192, kvh = 4.

- Суммарное время матричных умножений:

$$\frac{S \cdot H \cdot \Big((nh + kvh) \cdot dh \cdot 2 + I \cdot 3 + S \cdot (2+1)\Big) \cdot 2}{800 \cdot 10^{12}} = 10\,\mathrm{ms}$$

- Время memory bound операций (2 RMSNorm, 2 Add, 1 SwiGLU):

$$\frac{S \cdot H \cdot 2 \cdot (2 \cdot 2 + 2 \cdot 3) + S \cdot I \cdot 2 \cdot 3}{2.4 \cdot 10^{12}} = 0.55\,\mathrm{ms}$$

- Время на AllGather FSDP (в одном слое 1560 MB)

$$\frac{\Big(H \cdot (nh + kvh) \cdot dh \cdot 2 + H \cdot I \cdot 3\Big) \cdot 2}{400 \cdot 10^{9}} = 4\,\mathrm{ms}$$

- Время TP коммуникаций (NVLink)

$$\frac{S \cdot H \cdot TP \cdot 2 \cdot 2}{400 \cdot 10^{9}} = 1.3\,\mathrm{ms}$$

Время forward одного слоя 70B (TP=2)

- S = 4096 токенов, H = 8192, I = 28672, $nh \cdot dh$ = 8192, kvh = 4.

- Суммарное время матричных умножений:

$$\frac{S \cdot H \cdot \Big((nh + kvh) \cdot dh \cdot 2 + I \cdot 3 + S \cdot (2+1)\Big) \cdot 2}{800 \cdot 10^{12}} = 10\,\mathrm{ms}$$

- Время memory bound операций (2 RMSNorm, 2 Add, 1 SwiGLU):

$$\frac{S \cdot H \cdot 2 \cdot (2 \cdot 2 + 2 \cdot 3) + S \cdot I \cdot 2 \cdot 3}{2.4 \cdot 10^{12}} = 0.55\,\mathrm{ms}$$

- Время на AllGather FSDP (в одном слое 1560 MB)

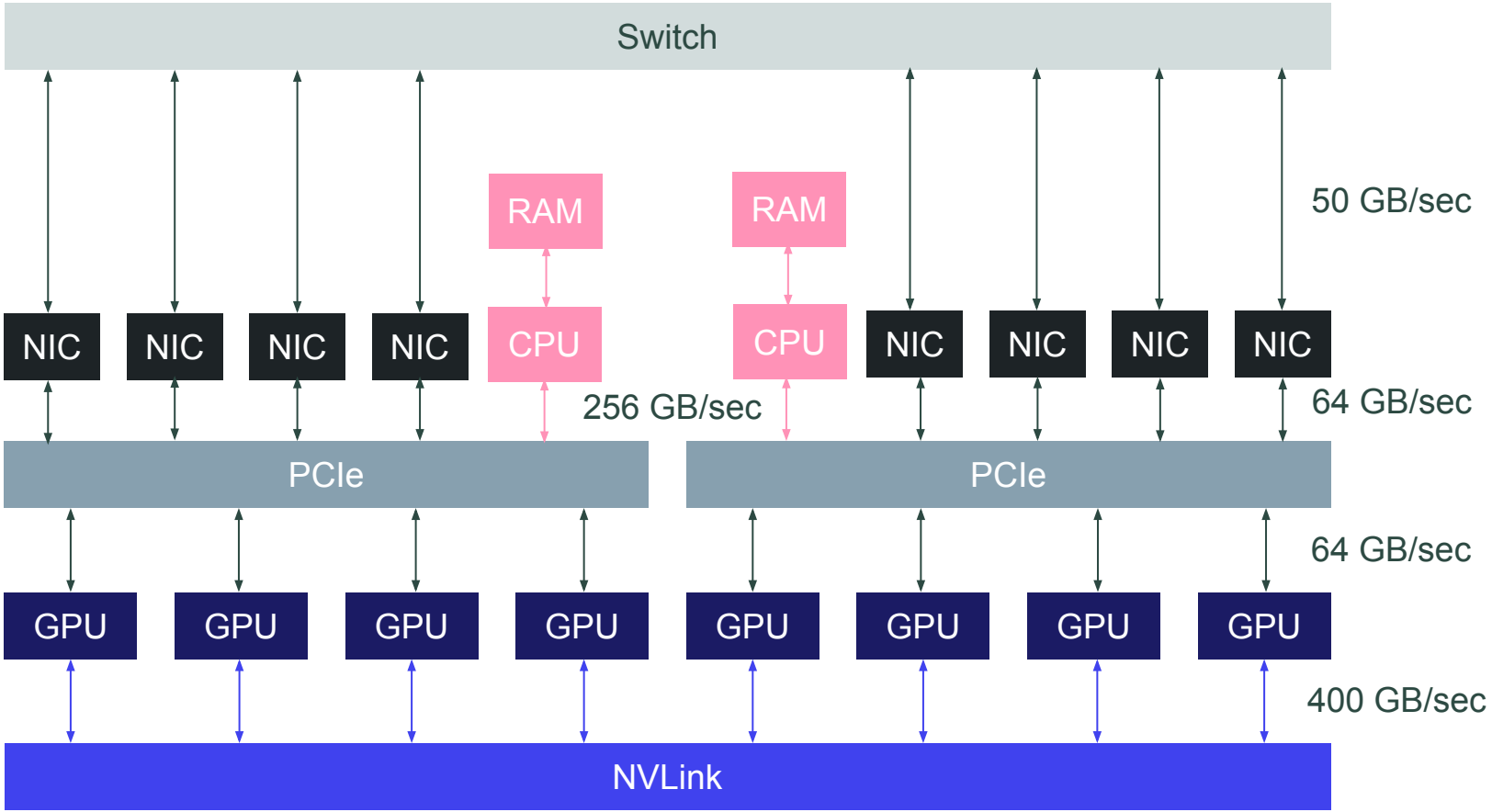
$$\frac{\Big(H \cdot (nh + kvh) \cdot dh \cdot 2 + H \cdot I \cdot 3\Big) \cdot 2}{400 \cdot 10^{9}} = 4\,\mathrm{ms}$$

Почему слой модели не стал меньше в 2 раза?

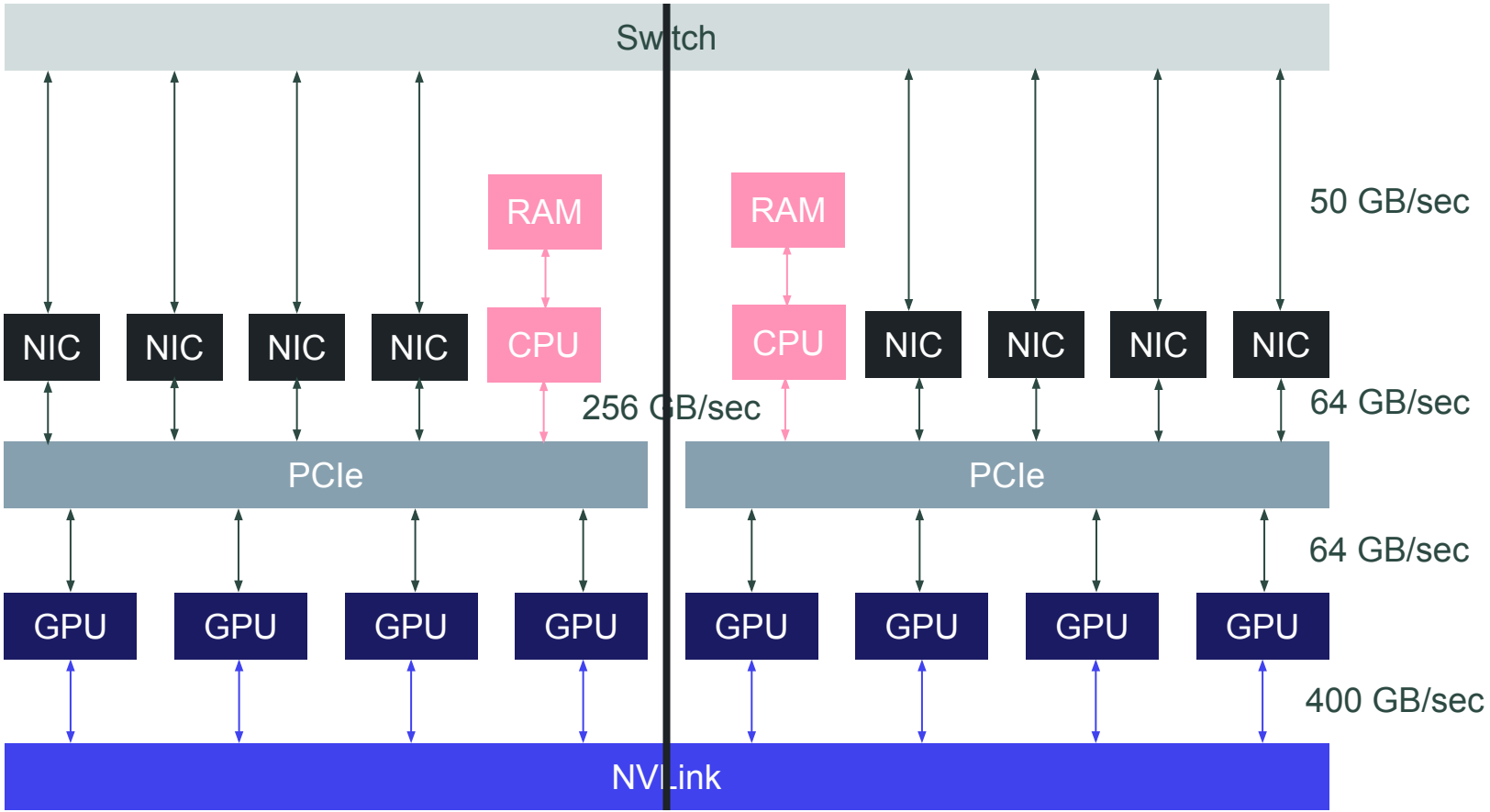
- Время TP коммуникаций (NVLink)

$$\frac{S \cdot H \cdot TP \cdot 2 \cdot 2}{400 \cdot 10^{9}} = 1.3\,\mathrm{ms}$$

Карта Хоста (H100 SXM)



Карта Хоста (H100 SXM)



Время forward одного слоя 70B (TP=2)

- S = 4096 токенов, H = 8192, I = 28672, $nh \cdot dh$ = 8192, kvh = 4.

- Суммарное время матричных умножений:

$$\frac{S \cdot H \cdot \Big((nh + kvh) \cdot dh \cdot 2 + I \cdot 3 + S \cdot (2+1)\Big) \cdot 2}{800 \cdot 10^{12}} = 10\,\mathrm{ms}$$

- Время memory bound операций (2 RMSNorm, 2 Add, 1 SwiGLU):

$$\frac{S \cdot H \cdot 2 \cdot (2 \cdot 2 + 2 \cdot 3) + S \cdot I \cdot 2 \cdot 3}{2.4 \cdot 10^{12}} = 0.55\,\mathrm{ms}$$

- Время на AllGather FSDP (в одном слое 1560 MB)

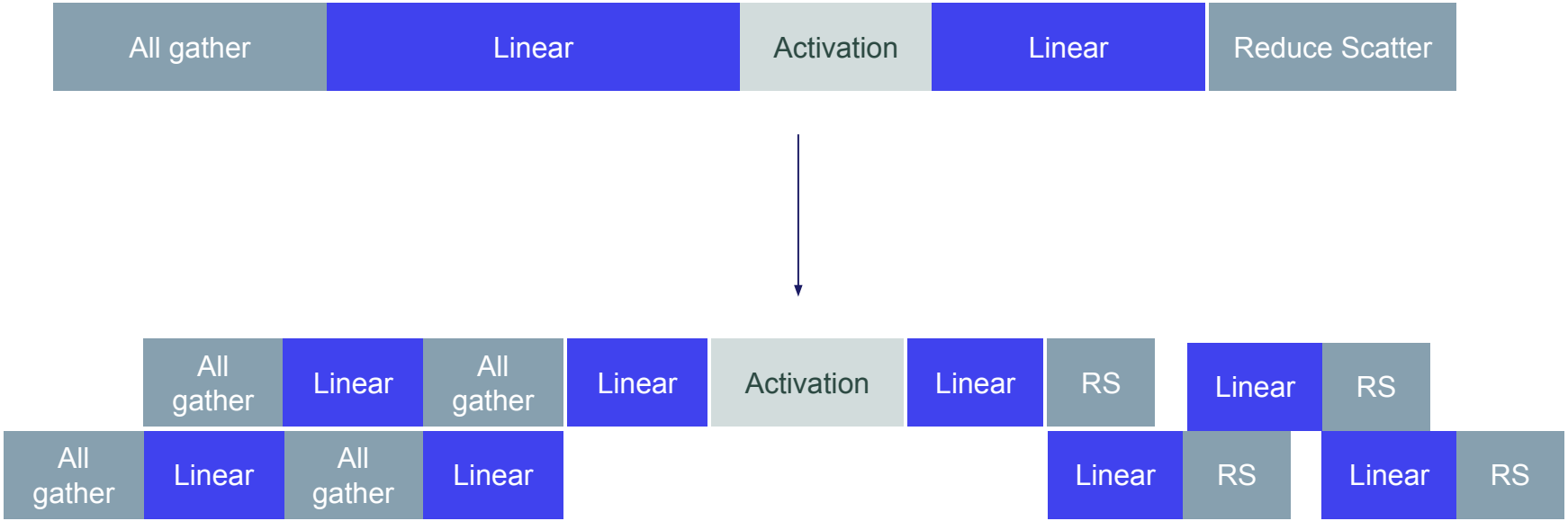
$$\frac{\Big(H \cdot (nh + kvh) \cdot dh \cdot 2 + H \cdot I \cdot 3\Big) \cdot 2}{400 \cdot 10^{9}} = 4\,\mathrm{ms}$$

- Время TP коммуникаций (NVLink)

$$\frac{S \cdot H \cdot TP \cdot 2 \cdot 2}{400 \cdot 10^{9}} = 1.3\,\mathrm{ms}$$

Эти коммуникации тормозят обучение?

Асинхронный TP



Асинхронный ТР. Риски

- Матричные умножения могут оказаться memory bound – это приведет к замедлению

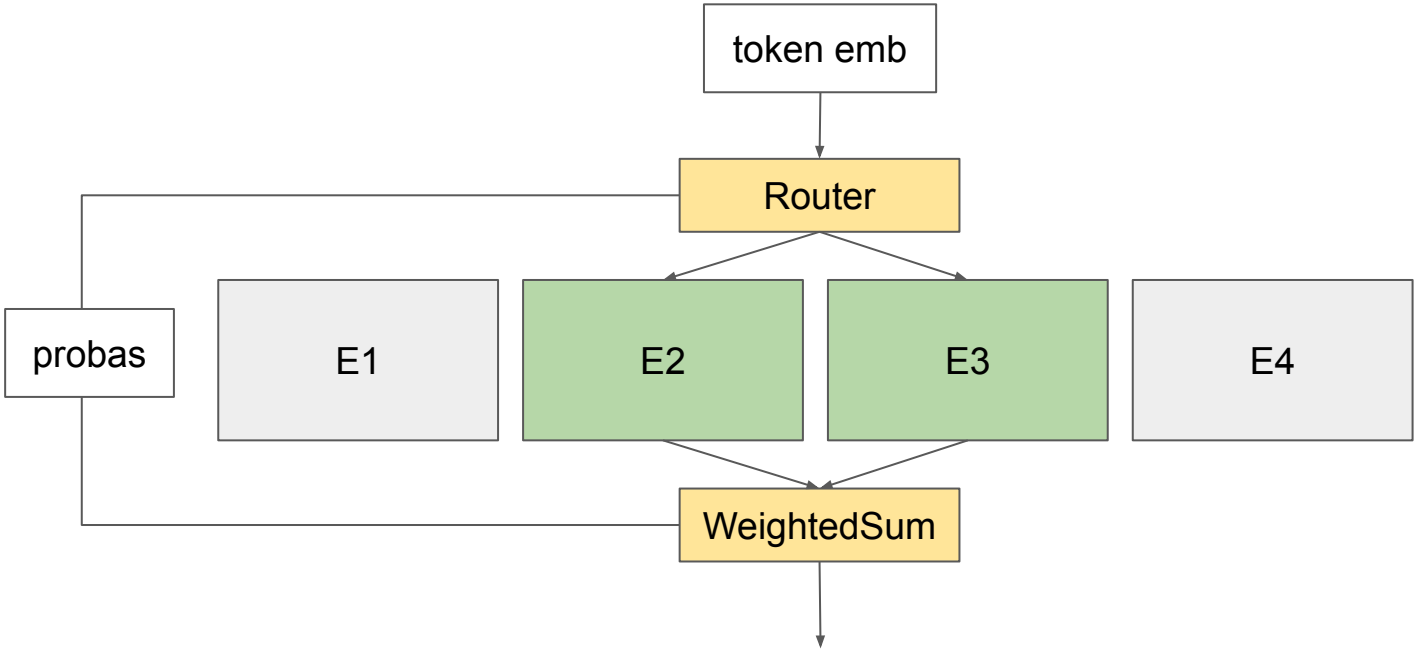
Выводы

- Симметричные коммуникации: FSDP и TP позволяют производить обучение LLM без потери скорости.
- Рекомпьют активаций – альтернатива доставке.
- Увеличение TP внутри хоста не приводит к уменьшению FSDP коммуникаций.
- Это основная причина, по которой подход FSDP+TP не является универсальным рецептом масштабирования обучения.
- Альтернативы: Context parallelism, Expert parallelism для MoE, Pipeline parallelism.

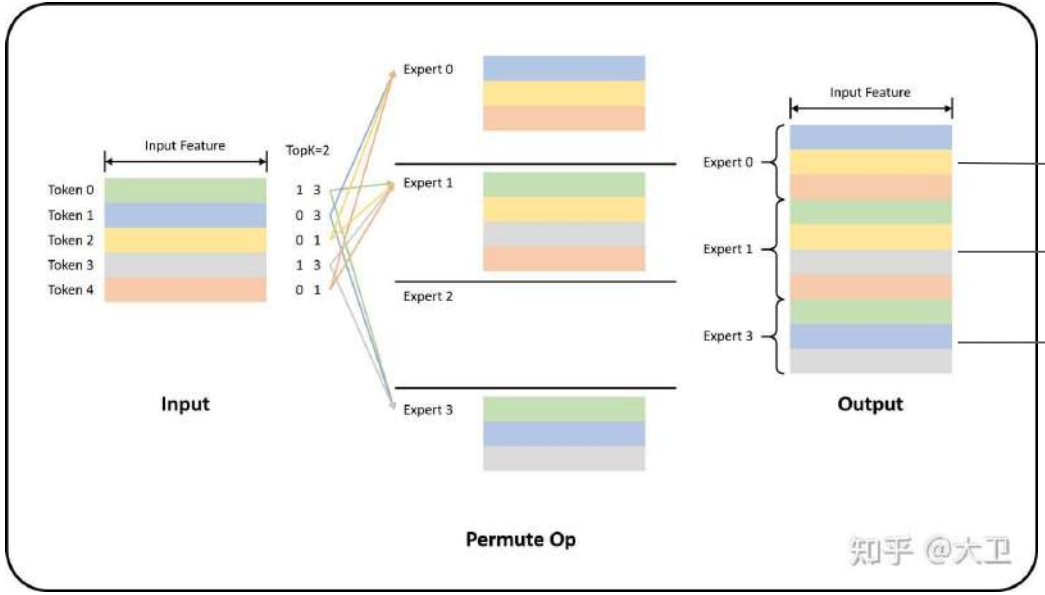
Кейс с MoE

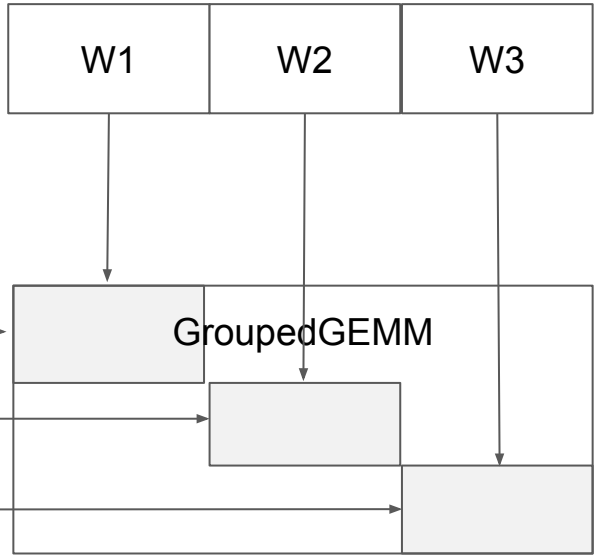
MoE

Современные модели учатся с MoE



MoE. Как это работает





А что поменялось?

В первую очередь, sparsity:

- Соотношение вычислений к параметрам существенно меньше:
  - DeepSeek v3: 1/18
  - Qwen 3, 235B: 1/11
  - Qwen 3.5: 1/23

А что поменялось?

В первую очередь, sparsity:

- Соотношение вычислений к параметрам существенно меньше:
  - DeepSeek v3: 1/18
  - Qwen 3, 235B: 1/11
  - Qwen 3.5: 1/23

Как итог: большой шанс того, что операция будет memory bound, FSDP коммуникации становятся очень дорогими.

Пример: Qwen 3, 235B

Оценки по времени на H100:

- Размер одного слоя в BF16: 5 GB
- Оптимальное время Forward (S=8k): 5 ms
- Оптимальное время FSDP-коммуникации: 12.5 ms

Пример: Qwen 3, 235B

Оценки по времени на H100:

- Размер одного слоя в BF16: 5 GB
- Оптимальное время Forward (S=8k): 5 ms
- Оптимальное время FSDP-коммуникации: 12.5 ms

Что можно делать?

- Растить локальный батч: 8k -> 16k
- Уменьшать объем FSDP-пересылок через квантизацию:
  - bf16 -> fp8

Пример 2: DeepSeek v3, 671B

Оценки по времени на H100:

- Размер одного слоя в BF16: 21 GB
- Оптимальное время Forward (S=8k): 10 ms
- Оптимальное время FSDP-коммуникации: 57 ms

Пример 2: DeepSeek v3, 671B

Оценки по времени на H100:

- Размер одного слоя в BF16: **21 GB**
- Оптимальное время Forward (S=8k): 10 ms
- Оптимальное время FSDP-коммуникации: 57 ms

Проблемы:

- 2 слоя весов и градиентов будут занимать слишком много памяти.

Пример 2: DeepSeek v3, 671B

Оценки по времени на H100:

- Размер одного слоя в BF16: 21 GB
- Оптимальное время Forward (S=8k): 10 ms
- Оптимальное время FSDP-коммуникации: **57 ms**

Проблемы:

- 2 слоя весов и градиентов будут занимать слишком много памяти.
- Коммуникации FSDP огромные и не перекрываются вычислениями.

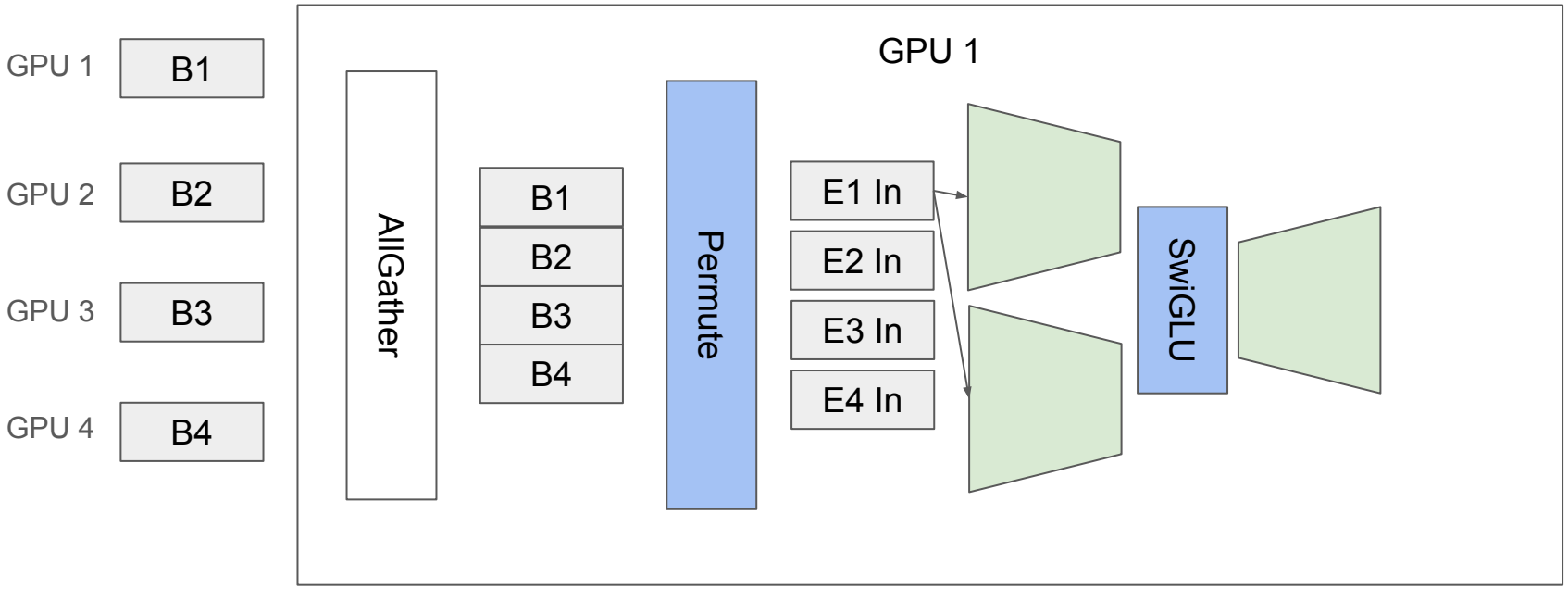
Нужно шардировать модель между хостами

- Тензорный параллелизм
- Экспертный параллелизм
- Пайплайн параллелизм

TP vs EP

Тензорный параллелизм

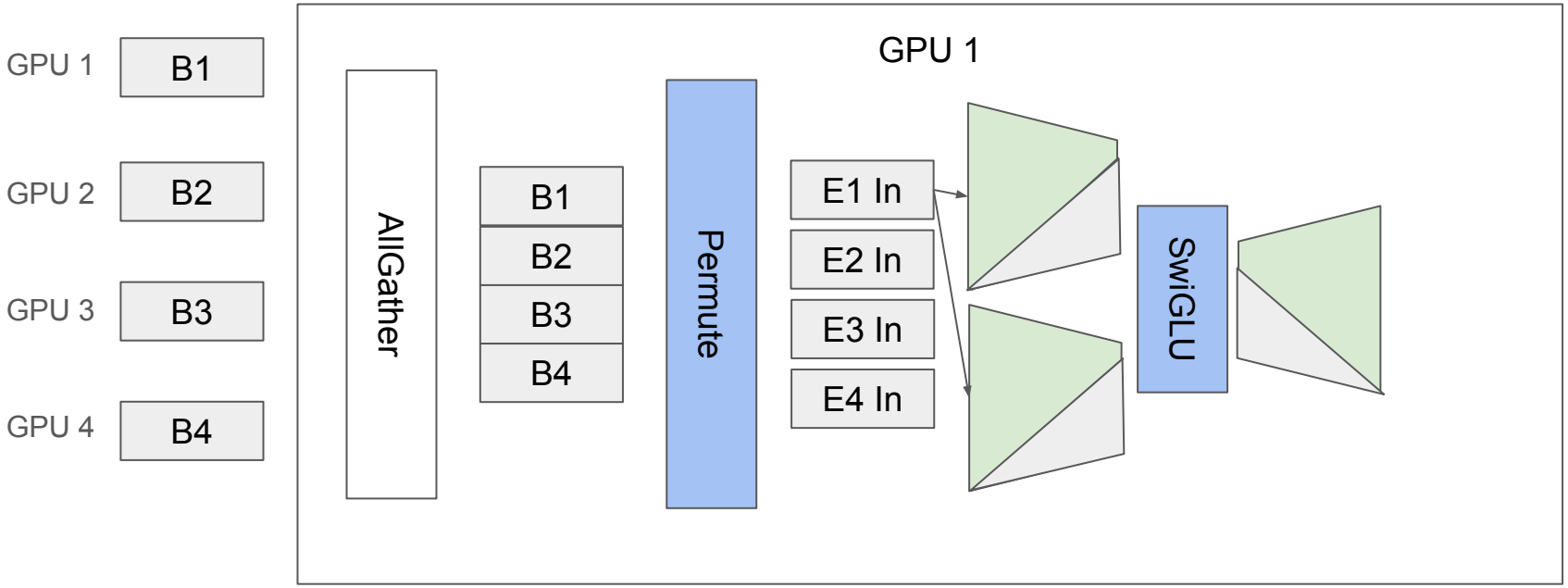
Экспертный параллелизм



TP vs EP

Тензорный параллелизм

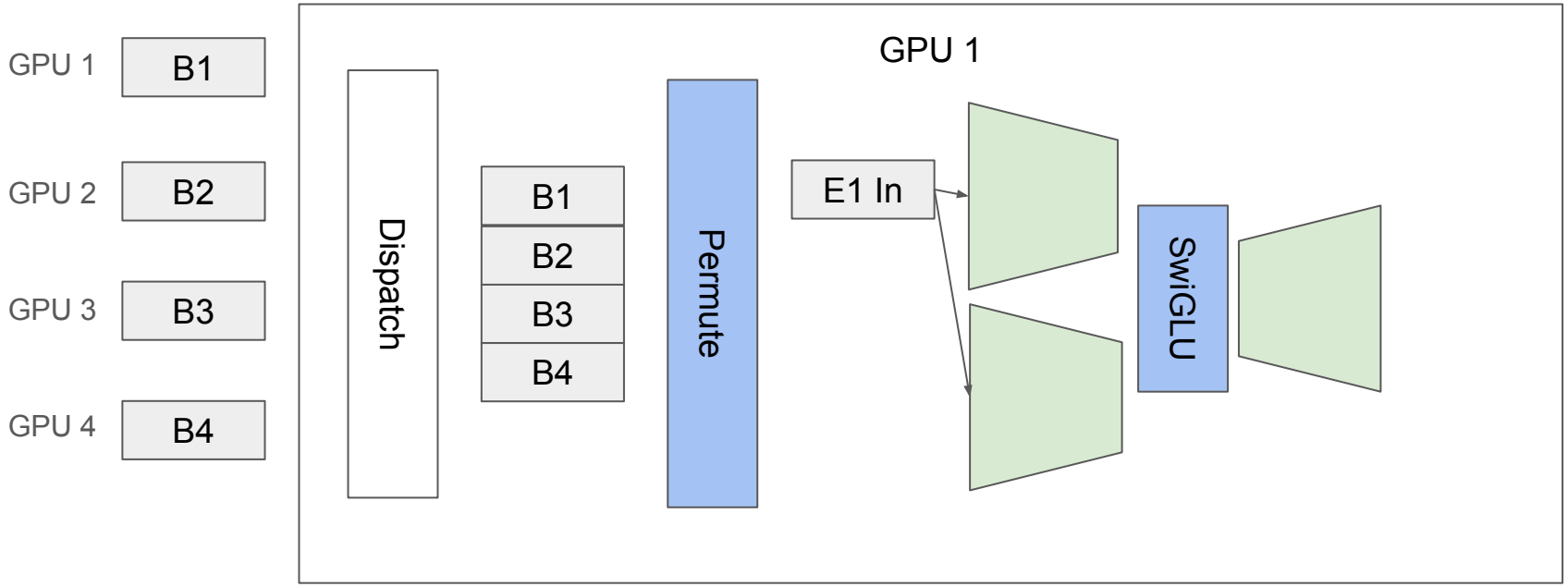
Экспертный параллелизм



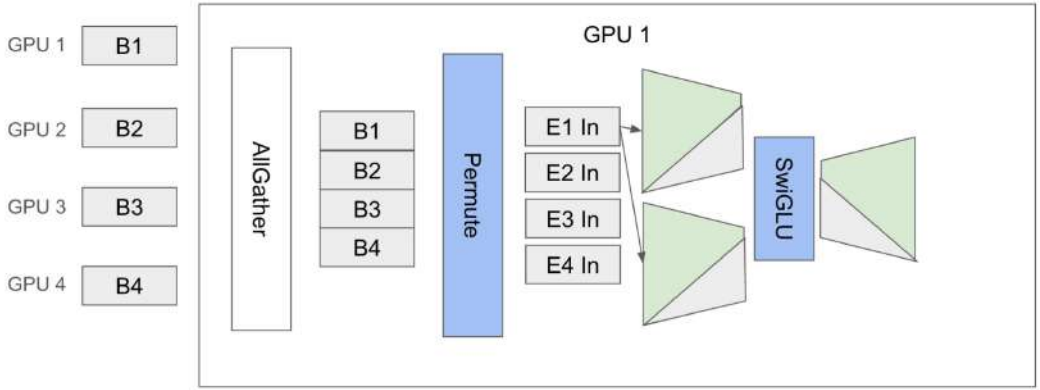
TP vs EP

Тензорный параллелизм

Экспертный параллелизм



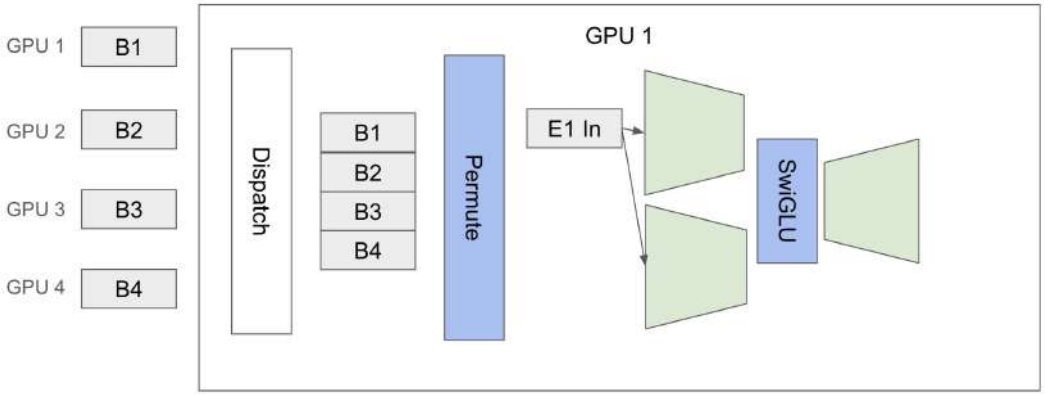
Для TP=8



S=8192, E=256, topK=8, H=7168, I=2048

- GroupedGEMM compute time:
  (S*TP)*topK*H*(I/TP)*2 / 800e12 = 2.5 ms
- GroupedGEMM memory time:
  (S*TP*top_k*H + E*H*I/TP)*2 / 2.4e12 = 3.5 ms

Для EP=8



S=8192, E=256, topK=8, H=7168, I=2048

- GroupedGEMM compute time:
  S*topK*H*I*2 / 800e12 = 2.5 ms
- GroupedGEMM memory time:
  (S*top_k*H + E/TP*H*I)*2 / 2.4e12 = 0.78 ms

Что по коммуникациям?

- Стоимость полного оптимального AllGather/RS для TP=8:
  2*S*H*TP / 400e9 = 2.2 ms
- Таких коммуникации нужно 2 на слой.

Что по коммуникациям?

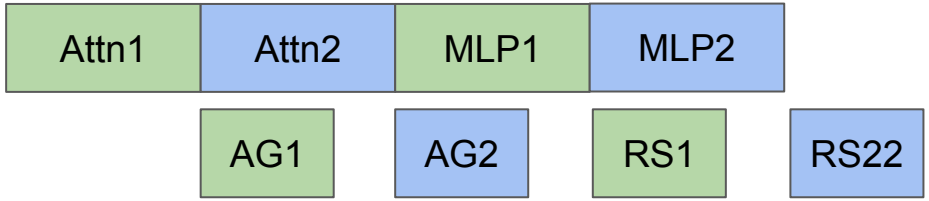
- Стоимость полного оптимального AllGather/RS для TP=8:
  2*S*H*TP / 400e9 = 2.2 ms
- Таких коммуникации нужно 2 на слой.
- EP может быть реализован дешевле, но при небольших EP оценка по времени коммуникации близка к TP.
- Время Forward одного слоя - 10 ms

Что по коммуникациям?

- Стоимость полного оптимального AllGather/RS для TP=8:
  2*S*H*TP / 400e9 = 2.2 ms
- Таких коммуникации нужно 2 на слой.
- EP может быть реализован дешевле, но при небольших EP оценка по времени коммуникации близка к TP.
- Время Forward одного слоя - 10 ms - лишь в 2 раза больше вычислений.

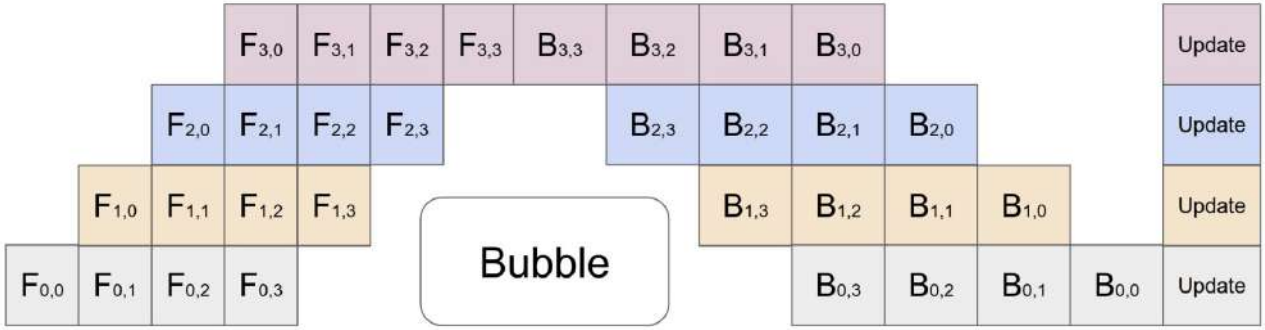
Что по коммуникациям?

- Стоимость полного оптимального AllGather/RS для TP=8:
  2*S*H*TP / 400e9 = 2.2 ms
- Таких коммуникации нужно 2 на слой.
- EP может быть реализован дешевле, но при небольших EP оценка по времени коммуникации близка к TP.
- Время Forward одного слоя - 10 ms - лишь в 2 раза больше вычислений. При больших TP/EP проигрыш будет больше.
- Вариант. Можно пробовать запускать 2 батча в параллель:



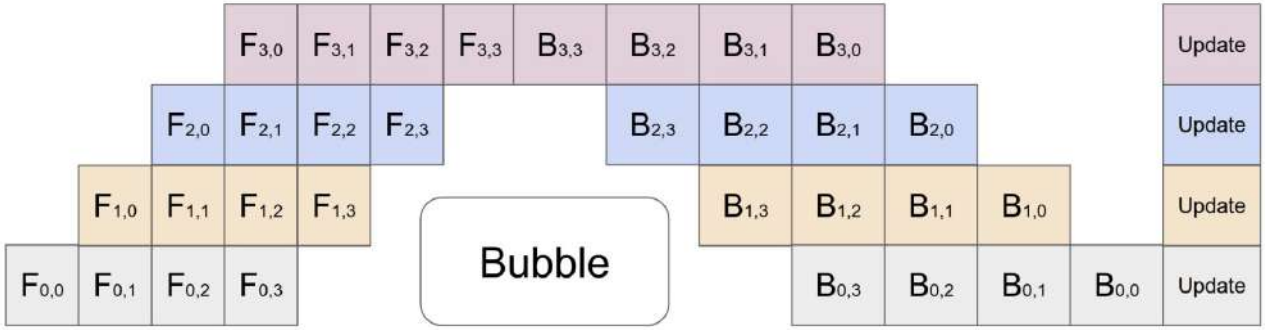
Что еще можно сделать с коммуникациями?

- Pipeline Parallelism



Что еще можно сделать с коммуникациями?

- Pipeline Parallelism
- Это ассиметричное шардирование - нужно строить сложные схемы.



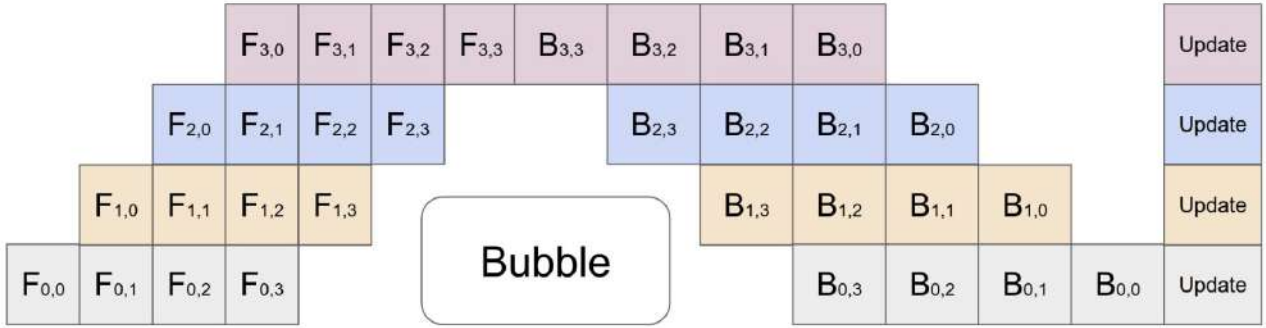
Что еще можно сделать с коммуникациями?

- Pipeline Parallelism
- Это ассиметричное шардирование - нужно строить сложные схемы.
- Но коммуникации максимально дешевы и подходят для пересылки между хостами:

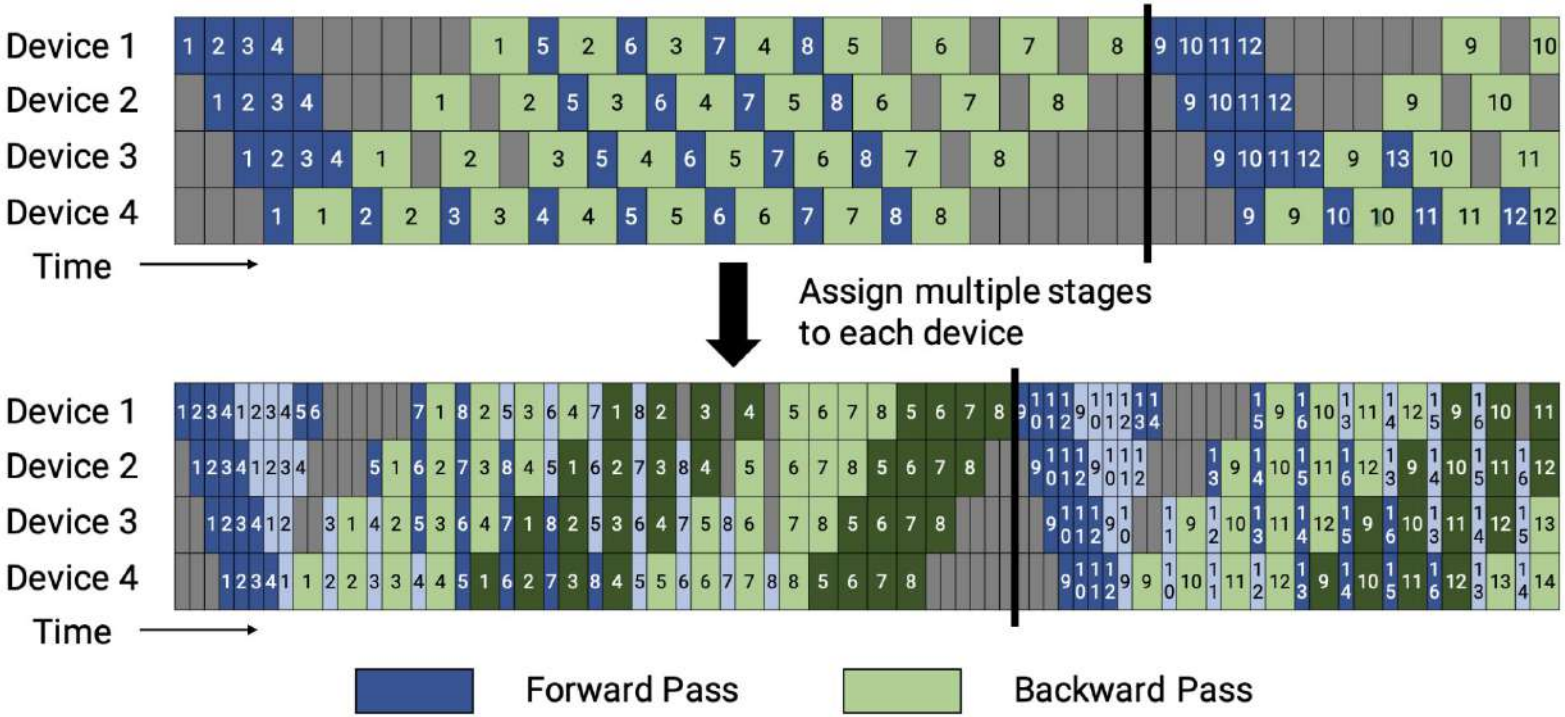
  $S*H*2 / 50e9 = 2$ ms

Pipeline parallelism

- PP = 4
- FSDP коммуникаций в PP раз меньше
- PP коммуникаций мало и они дешевые
- Но: схема ассиметричная, есть пузыри



1F1B


Device 1
Device 2
Device 3
Device 4
Time
Assign multiple stages
to each device
Device 1
Device 2
Device 3
Device 4
Time
Forward Pass
Backward Pass

ZeroBubble

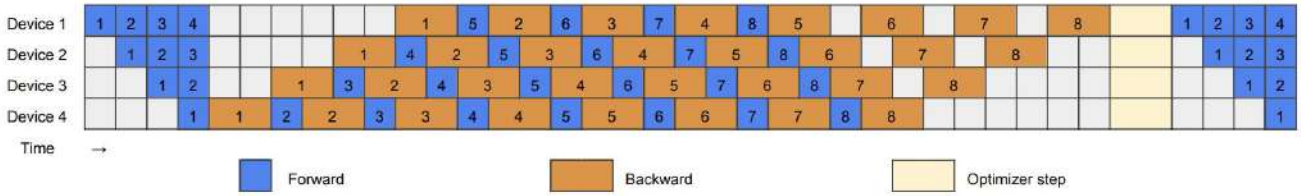


Figure 2: 1F1B pipeline schedule.

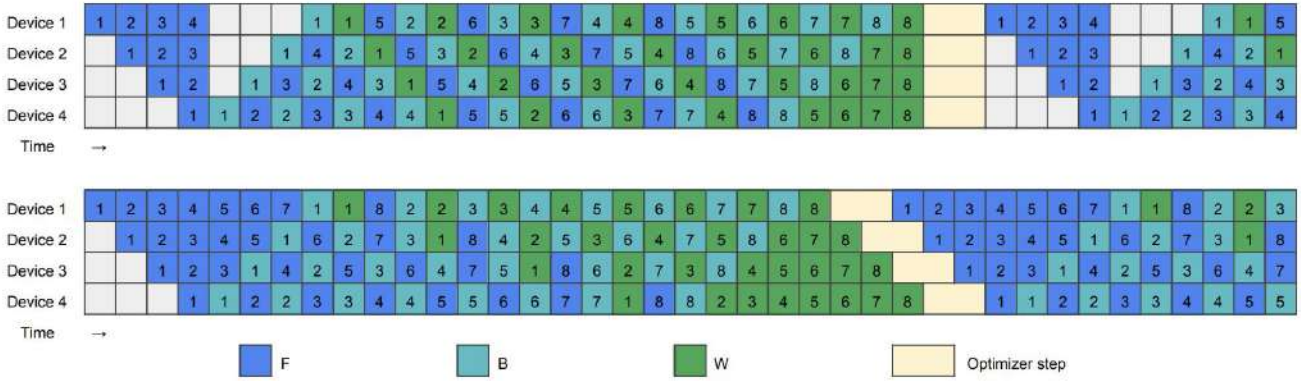
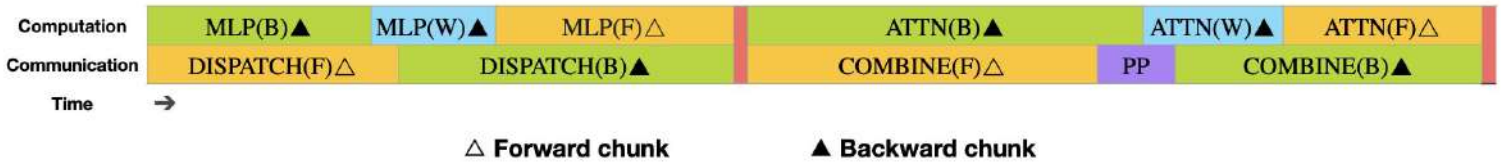


Figure 3: Handcrafted pipeline schedules, top: ZB-H1; bottom: ZB-H2

EP+PP: DualPipeV



Выводы

- С MoE работать сложнее: FSDP коммуникации становятся очень дорогими, а слои могут не помещаться на GPU после сборки.
- Иногда эту проблему можно решить увеличением плотности вычислений и ускорением коммуникаций.
- В общем случае, модель нужно шардировать по различным хостам для оптимизации FSDP.
- PP - наиболее эффективный и сложный способ сделать это. Часто для этого используют EP.

Заключение

Заключение

- Эффективное обучение LLM связано в первую очередь с правильно выстроенной логистикой данных.
- Можно пересылать веса и их градиенты через FSDP, в том числе на CPU RAM.
- Можно пересылать активации и их градиенты через TP и другие параллелизмы.
- Можно делать их рекомпьют.

Школа анализа данных

Михаил Хрущев